Electronics for Embedded Systems notes

Luca Seggiani
2026

1 Lesson 23-09-26

1.1 Introduction

The course is meant to ggive an introduction to *embedded systems*, after a review of analog and digital electronics. The topics covered are:

- Memories;
- (Programmable Logic Devices) (**PLD**s);
- Interconnection protocols and I/O;
- Embedded processors an peripherals;
- **DA** (*Digital-to-Analog*) and **AD** (*Analog-to-Digital*) conversion;
- Power management.

1.2 Review

Let's begin the review of analog and digital electronics. The review will cover:

- Electrical components, including:
  - **Passive components**, as in *resistors*, *capacitors* and *inductors*;
  - **Active components**, as in *diodes* and *transistors*;
  - **Operational amplifiers**.
- Digital electronics, including *logic gates*, *digital functions*, **FSM**s (*Finite State Machines*) and the *VHDL* language.

1.2.1 Passive components

Resitors offer *resistance* to electric current, measured in *Ohms* (Ω). By Ohm's law:

$$v_R(t) = Ri_R(t), \quad V_R(s) = RI_R(s)$$

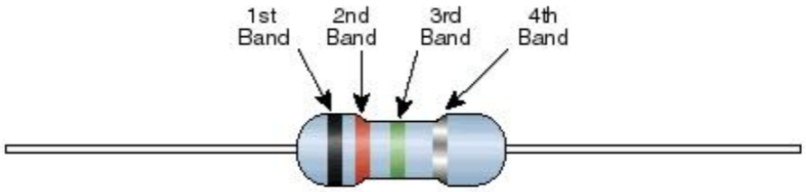
with $s = \sigma + j\omega$ in the Laplace domain. A definition via *conductance*, measured in *Siemens* (S), is also possible:

$$i_R(t) = Gv_R(t), \quad g = \frac{1}{R}$$

Resistor response doesn't depend on the frequency of periodic signals (all signals get attenuated) the same. This applies at least partly to real resistors, and we will assume the ideal model from now onwards.

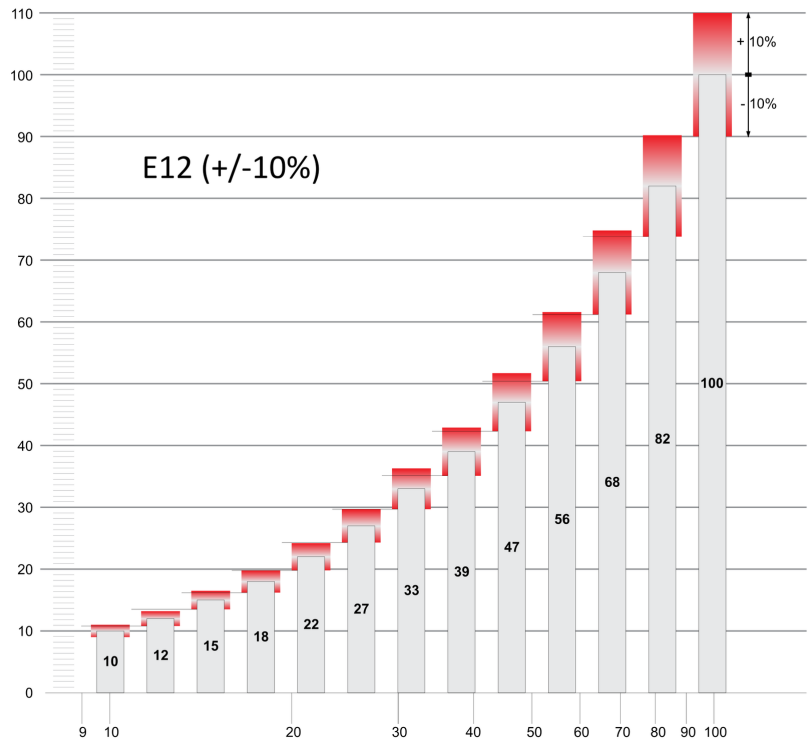
It can also be useful to remember the code used to distinguish different resistance values.

Standard EIA Color Code Table 4 Band: ±2%, ±5%, and ±10%



Color	1st Band (1st figure)	2nd Band (2nd figure)	3rd Band (multiplier)	4th Band (tolerance)
Black	0	0	10^0	
Brown	1	1	10^1	
Red	2	2	10^2	±2%
Orange	3	3	10^3	
Yellow	4	4	10^4	
Green	5	5	10^5	
Blue	6	6	10^6	
Violet	7	7	10^7	
Gray	8	8	10^8	
White	9	9	10^9	
Gold			10^{-1}	±5%
Silver			10^{-2}	±10%

Resistor tolerances are to be taken into account, as they might drive resistances in our circuits out of the feasible margins. Because of this, we preder using standard values in *E series*. E series is a system of preferred numbers (also called preferred values) derived for use in electronic components. It consists of the E3, E6, E12, E24, E48, E96, and E192 series, where the number after the "E" designates the quantity of logarithmic value "steps" per decade.



Capacitors store energy as voltage potential between conductive armatures separated by a dielecric. *Capacity*, measured in *Farad* (F) is defined as:

$$C = \frac{Q}{V}$$

The laws that govern capacitors in the time domain are differential equations:

$$i_C(t) = C\frac{d}{dt}v_C(t), \quad v_C(t) = \frac{1}{C}\int_0^t i_C(t)\,d\tau$$

It is then easier to move to the Laplace domain, where we can simply express integration as division by s:

$$V_C(s) = \frac{I_C(s)}{sC}$$

In regards to periodic signals, the behaviour of capacitors depends on frequency:

- When $f \to 0$ it behaves as an *open circuit*;
- When $f \to \infty$ it behaves as a *short circuit*;

Many types of capacitors are available:

- *Electrolytic* capacitors, which are polar;
- *Ceramic* capacitors, which are not polar.

To read the value of a capacitor we have quite a complex system:

- Two digits: it is the value in pF;
- Three digits: the first two are the value in pF, and the third is the multiplier; If the third digit is 8 the multiplier is 10^{-2}, if it is 9 the multiplier is 10^{-1};
- A digit, a character and a digit: the two digits are the one before and after the decimal point, the character is the unit of measure;
- A character follows the last digit indicates the tolerance, with $J = \pm 5\%$, $K = \pm 10\%$, $M = \pm 20\%$.

Inductors store energy as magnetic field in a winding of conductive wire around a metal core. *Indcutance*, measured in *Henry* (H) is defined as:

$$L = \frac{\Phi}{I}$$

The laws that govern indcutors in the time domain are differential equations:

$$v_L(t) = L\frac{d}{dt}i_L(t), \quad i_L(t) = \frac{1}{L}\int_0^t v_L(t)\,d\tau$$

It is then easier to move to the Laplace domain, where we can simply express derivation as multiplication by s:

$$V_L(s) = sLI_L(s)$$

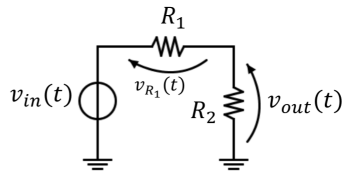
In regards to periodic signals, the behaviour of inductors also depends on frequency, on the contrary of capacitors:

- When $f \to \infty$ it behaves as an *open circuit*;
- When $f \to 0$ it behaves as a *short circuit*;

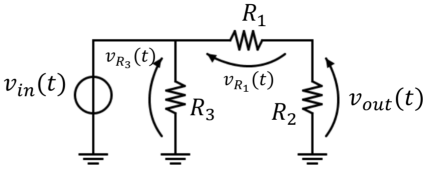
1.2.2 Voltage and current dividers

We show a chart showcasing a number of elementary divider circuits which we can use.

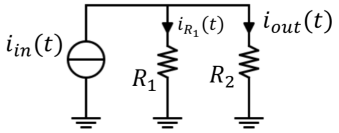
- Voltage divider with two resistances
  - $v_{out}(t) = \frac{R_2}{R_1+R_2} v_{in}(t) = \frac{g_1}{g_1+g_2} v_{in}(t)$
  - $v_{R_1}(t) = \frac{R_1}{R_1+R_2} v_{in}(t) = \frac{g_2}{g_1+g_2} v_{in}(t)$



- Voltage divider with three resistances
  - $v_{out}(t) = \frac{R_2}{R_1+R_2} v_{in}(t) = \frac{g_1}{g_1+g_2} v_{in}(t)$
  - $v_{R_3}(t) = v_{in}(t)$



- Current divider
  - $i_{out}(t) = \frac{R_1}{R_1+R_2} i_{in}(t) = \frac{g_2}{g_1+g_2} i_{in}(t)$
  - $i_{R_1}(t) = \frac{R_2}{R_1+R_2} i_{in}(t) = \frac{g_1}{g_1+g_2} i_{in}(t)$



Let us note in the second example (voltage divider with three resistances) that nothing changes by adding the third resistor. $V_{out}(t)$ is exactly the same, as the potential seen by R_1 on its left side doesn't change (and the potential across R_3 is the same as $v_{in}(t)$).

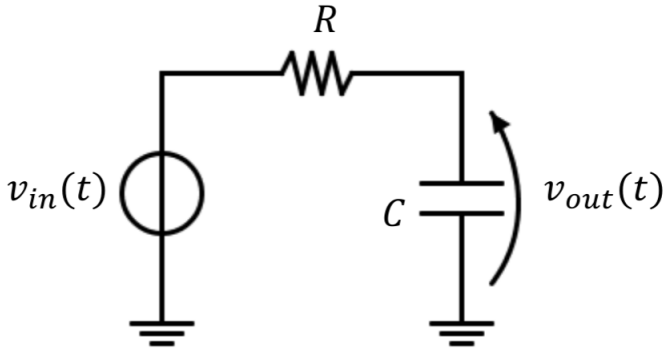
What actually does change in the example is the resistance: we have effectively built a current divider, where the input current is:

$$i_{in}(t) = \frac{v_{in}(t)}{R_1 + R_2} + \frac{v_{in}(t)}{R_3}$$

Lastly, let's note that all the equations in the Laplace domain are the same, changing the variables accordingly.

1.2.3 Simple low pass filters

A simple **low pass** filter can be seen as a special case of a divider:



By writing the *Kirchoff Voltage Law* (**KVL**) of the circuit, we get:

$$v_{in}(t) - R \cdot C \frac{d}{dt} v_{out}(t) - v_{out}(t) = 0$$

with the first term being the voltage gain on the power supply, the second term being the voltage drop on the resistor, and the last term being the output voltage taken across the capacitor.

Time domain solution This is a first order linear differential equation, which we can express as:

$$\frac{d}{dt}v_{out}(t) + \frac{1}{\tau}v_{out}(t) = \frac{1}{\tau}v_{in}(t), \quad \tau = RC$$

We refer to $\tau = RC$ as the *time constant* of the circuit. A general solution is:

$$v_{out}(t) = e^{-\frac{t}{\tau}}\left(k + \int\left(\frac{v_{in}(t)}{\tau}e^{\frac{t}{\tau}}\right)dt\right)$$

It can be useful to carry out the full calculation. We start from the general differential equation obtained from KVL:

$$v_{in}(t) - RC\frac{d}{dt}v_{out}(t) - v_{out}(t) = 0$$

We can easily rearrange this into a standard first-order form:

$$\frac{d}{dt}v_{out}(t) + \frac{1}{RC}v_{out}(t) = \frac{1}{RC}v_{in}(t)$$

Now we take a function $\mu(t)$ constructed such that:

$$\mu(t)\frac{1}{RC} = \frac{d}{dt}\mu(t)$$

While the purpose of this might at this point be unclear, we can clearly see that a function that satisifies the equation is:

$$\mu(t) = e^{\frac{t}{RC}}$$

Let's multiply both sides of the original differential equation by $\mu(t)$:

$$\mu(t)\frac{d}{dt}v_{out}(t) + \mu(t)\frac{1}{RC}v_{out}(t) = \mu(t)\frac{1}{RC}v_{in}(t)$$

By applying the property above this leaves us with:

$$\mu(t)\frac{d}{dt}v_{out}(t) + \frac{d}{dt}\mu(t)v_{out}(t) = \frac{\mu(t)}{RC}v_{in}(t)$$

It's clear that the left hand side is the expansion of the derivative of a product, so we can write:

$$\frac{d}{dt}\left(\mu(t)v_{out}(t)\right) = \frac{\mu(t)}{RC}v_{in}(t)$$

Recall that we were interested in $v_{out}(t)$. We can take it out the derivative by integrating both sides:

$$\mu(t)v_{out}(t) = \int\frac{\mu(t)}{RC}v_{in}(t) + k$$

where the antiderivative operation requires that we include a $\pm k$ factor (sign indifferent).

At this point we just have to make $v_{out}(t)$ explicit and substitute our original definition of $\mu(t)$:

$$v_{out}(t) = \frac{1}{\mu(t)} \left(\int \frac{\mu(t)}{RC} v_{in}(t) + k \right) = e^{-\frac{t}{RC}} \left(k + \int \left(\frac{v_{in}(t)}{RC} e^{\frac{t}{RC}} \right) dt \right)$$

Which is our original expression (given $\tau = RC$). □

With $v_{in}(t)$ taken as a step of amplitude V_M, or equivalently by taking the solution given with zero initial conditions and a costant power source, we get the usual form:

$$v_{out}(t) = V_M(1 - e^{-\frac{t}{\tau}})$$

where the time constant determines the time taken to reach saturation. We recall that in one time constant we reach around 63.2% of full saturation, and in three time constants we reach 95%.

Laplace domain solution Let's carry out the same calculation in the Laplace domain. By expressing the voltage divider we get:

$$V_{out}(S) = \frac{\frac{1}{sC}}{\frac{1}{sC} + R} V_{in}(S) = \frac{1}{sRC + 1} V_{in}(s)$$

where in the first expression we simply applied the voltage divider law to the impedance definitions of the capacitor and the resistor. We can then derive a *transfer function*:

$$\frac{V_{out}(s)}{V_{in}(s)} = \frac{1}{sRC + 1}$$

We can shift to the Fourier domain, or in equivalent terms take a projection of the Laplace domain in only the frequency direction, by setting $s = j\omega$:

$$\frac{V_{out}(j\omega)}{V_{in}(j\omega)} = \frac{1}{j\omega RC + 1}, \quad \left| \frac{V_{out}(j\omega)}{V_{in}(j\omega)} \right| = \frac{1}{\sqrt{\omega^2 R^2 C^2 + 1}}$$

A *pole* in this expression is a zero of the denominator, so:

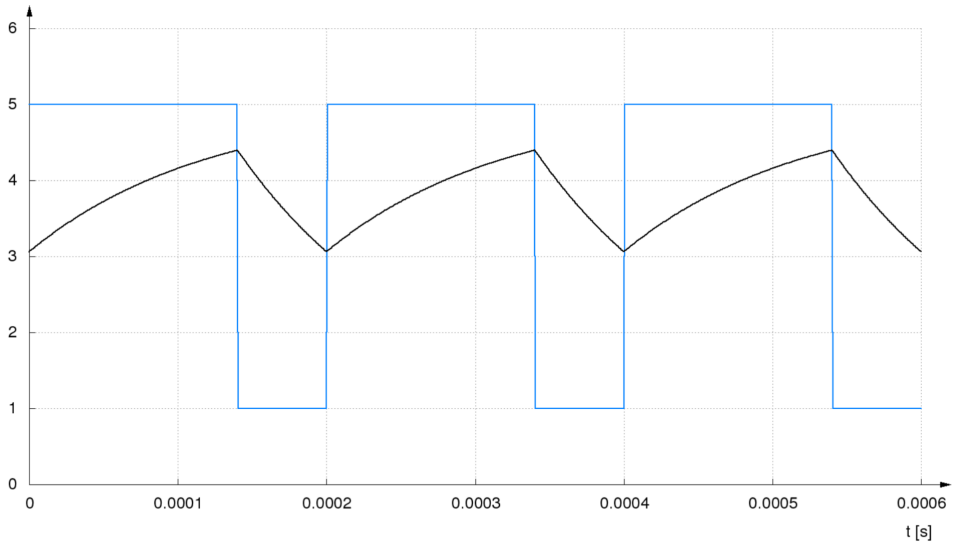
$$s_p = -\frac{1}{RC}, \quad \omega_p = \frac{1}{RC}$$

with the frequency representation being bilateral, as always. Let's note that the *pole pulsation* ω_p is not really a zero of the denominator, but just the characteristic frequency of the system, which we find useful for plotting and solving circuit responses.

Time domain response Fixing V_{in} to a periodic square wave, let's compute the time domain response. Each fragment in the response is an exponential:

$$v_{out}(t) = A + Be^{-\frac{t}{\tau}}$$

where $\tau = RC$.



We can proceed with the following approach:

1. Evaluate the waveforms at the extreme, fixing initial conditions;
2. Determine A and B for the rising and falling cases;
3. Determine the maximum and minimum voltages.

And the result we obtain should be valid at the steady-state. We say *steady-state*, as when the oscillators first begins oscillating there will be a transient, which will temporarily not obey to our prediction.

Let's start with the rising edge. By remembering our first equation, we can compute V_{out} at the first time instant:

$$V^r_{out}(0) = A + B = V_{min}$$

If at this point the square wave never went to a low value, the capacitor would continue charging and we would approach saturaiton (at V_H, the maximum value of the square wave). From this we can derive $A = V_H$, as the B term vanishes to 0 approaching infinity, and it follows that $B = V_{min} - V_H$. This gives us:

$$V^r_{out}(t) = V_H + (V_{min} - V_H)e^{-\frac{t}{\tau}}$$

What happens in reality is that at some time T_H we switch to the falling edge, reaching a maximum value V_{max}.

$$V^r_{out}(T_H) = A + Be^{-\frac{t}{\tau}} = V_H + (V_{min} - V_H)e^{-\frac{T_H}{\tau}} = V_{max}$$

To fix V_{min} and V_{max}, we switch to anayzing the falling edge. With similar calculations, we can derive the equation:

$$V^f_{out}(T_L) = V_L + (V_{max} - V_L)e^{-\frac{T_L}{\tau}} = V_{min}$$

with V_L at the lowest point of the square wave.

We can easily solve the resulting system of equations, to figure out the minimum and maximum bounds of the response of the circuit:

$$\begin{cases} V_H + (V_{min} - V_H)e^{-\frac{T_H}{\tau}} = V_{max} \\ V_L + (V_{max} - V_L)e^{-\frac{T_L}{\tau}} = V_{min} \end{cases}$$

By setting:

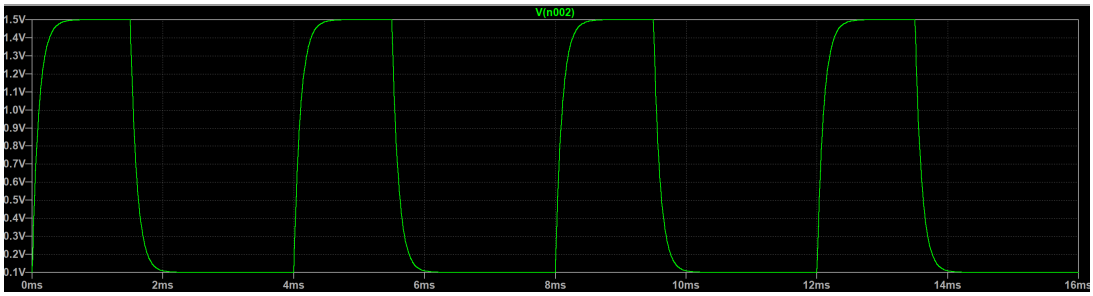
$$\alpha = e^{-\frac{T_H}{T}}, \quad \beta = e^{-\frac{T_L}{T}}$$

we get the solutions (calculations omitted because they're boring):

$$V_{max} = \frac{V_H (1-\alpha) + \alpha V_L (1-\beta)}{1-\alpha\beta}, \quad V_{min} = \frac{\beta V_H (1-\alpha) + V_L (1-\beta)}{1-\alpha\beta}$$

We can then trace the actual response by drawing exponential branches with the time constant given by $\tau = RC$.

Lastly, let's see an LTSPICE (simulation program) plot of the square wave response of this circuit, which confirms our mathematical model:



Frequency domain response In the frequency domain, we can consider the effect on a periodic signal such as a sine wave. Let's have:

$$V_{in}(t) = M \sin(\omega t) = M \sin(2\pi f t)$$

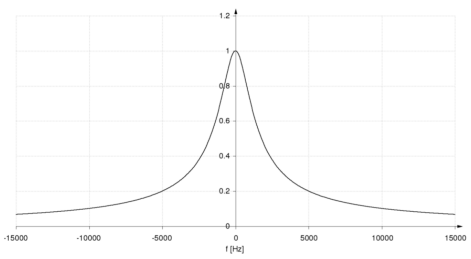
In a linear system at the steady state (which we have) if the input is sinusoidal, the output is sinusoidal at the same frequency: only the amplitude and the phase can be shifted:

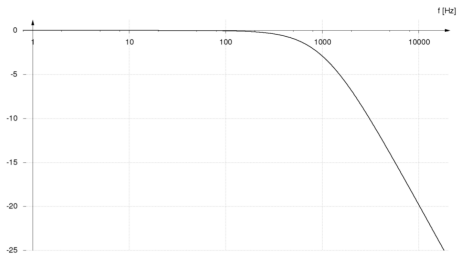
$$V_{out}(t) = M' \sin(\omega t + \phi)$$

By using the Fourier domain equations calculated earlier, we can actually express M' and ϕ:

$$M' = M \frac{\omega_p}{\sqrt{\omega^2 + \omega_p^2}}, \quad \phi = -\tan^{-1}\left(\frac{\omega}{\omega_p}\right)$$

For arbitrary frequencies, we can plot the linear and logarithmic Bode diagrams of amplitude:





Let us note that at $f_p = \frac{\omega_p}{2\pi}$ we have the $-3\,\mathrm{dB}$ band for our transfer function, which we can also refer to as the *cutoff frequency* or *pole frequency* of the lowpass.

1.2.4 Superposition of effects

Theorem 1.1: Superposition of effects

In a linear system with more than one input (independent ideal sources, voltage or current), the output is the algebraic sum of the effects of each input with the other set to 0.

This means:

- Voltage source at zero $\rightarrow$ short circuit;
- Current source at zero $\rightarrow$ open circuit;
- Dependent sources must not be set to 0.

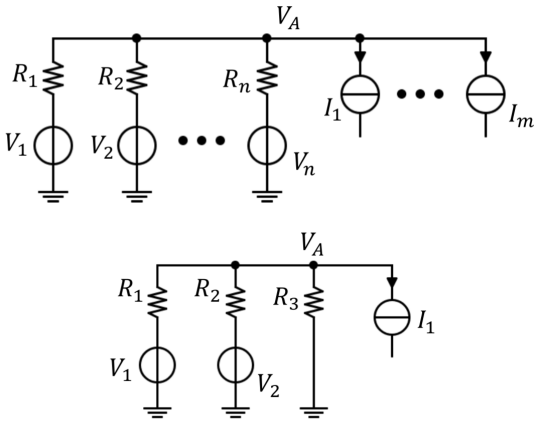
Superposition applies to *linear* systems, that is circuits which feature:

- Sources (independent and dependent), resistors, capacitors, inductors, transformers;
- Non linear system

If diodes, transistors, operational amplifiers or other active elements are present, the superposition of effects is not applicable. In these cases we need to first *linearize* the system around a bias point.

1.2.5 Millman's theorem

Let us take the circuti:



Theorem 1.2: Millman's Theorem

In the general network on the right, the voltage V_A is given by the expression:

$$V_A = \frac{\sum_{i=1}^{n} \frac{V_i}{R_i} - \sum_{j=1}^{m} I_j}{\sum_{i=1}^{n} \frac{1}{R_i}}$$

The theorem can be proved using the superposition of effects and voltage dividers with conductances. Let us work through it.

The idea is to:

1. Calculate the contribution on voltage V_A given by just a voltage generator V_j; then we can sum the contribution over all voltage generators to get the cumulative voltage generator contribution, which we'll call V_A^a;

2. Calculate the contribution on voltage V_A given by just a current generator I_j; then we can sum the contribution over all current generators to get the cumulative current generator contribution, which we'll call V_A^b.

Superposition is applied in both steps, and one final time to sum the contributions of the voltage and current generators and get the final expression for V_A.

- For the voltage generators, we take the contribution of a single voltage generator V_j as:

$$V_A^{a_j} = \frac{\|_{i\neq j}^n R_i}{\|_{i\neq j}^n R_i + R_j} V_j$$

where $\|_{i\neq j}^n R_i$ is notation soup for the parallel of all resistance but R_j.
We can sum over these to get the total voltage generator contribution V_A^a:

$$V_A^a = \sum_{j=1}^n \frac{\|_{i\neq j}^n R_i}{\|_{i\neq j}^n R_i + R_j} V_j$$

This sum is a bit hard to compute, as the denominator changes at each value of j and we don't have a clean way to expand it. Things would be quite a bit easier if we had the same denominator at each value of j. We can make this happen by dividing the numerator and the denominator by $R_j \, \|_{i\neq j}^n R_i$, which gives:

$$V_A^a = \sum_{j=1}^n \frac{\frac{1}{R_j}}{\frac{1}{R_j} + \frac{1}{\|_{i\neq j}^n R_i}} V_j$$

To our convenience, we note that $\frac{1}{\|_{i\neq j}^n R_i}$ is equal to $\sum_{i\neq j}^n \frac{1}{R_i}$, so:

$$V_A^a = \sum_{j=1}^n \frac{\frac{1}{R_j}}{\frac{1}{R_j} + \sum_{i\neq j}^n \frac{1}{R_i}} V_j = \sum_{j=1}^n \frac{\frac{1}{R_j}}{\sum_{i=1}^n \frac{1}{R_i}} V_j = \frac{\sum_{i=1}^n \frac{V_i}{R_i}}{\sum_{i=1}^n \frac{1}{R_i}}$$

- Current generators are thankfully simpler to handle. We take the contribution of a single current generator I_j as:

$$V_A^{b_j} = I_j \, \|_{i=1}^n$$

We can sum over these to get the total voltage generator contribution V_A^b:

$$V_A^b = \sum_{j=1}^m I_j \, \|_{i=1}^n = \sum_{j=1}^m \frac{I_j}{\sum_{i=1}^n \frac{1}{R_i}} = \frac{\sum_{j=1}^m I_i}{\sum_{i=1}^n \frac{1}{R_i}}$$

At this point we just sum to get the final result. □

1.2.6 Active components

Diodes are non-linear active components, built today using solid state technologies, and more specifically *PN junctions* (as opposite to tubes, which were historically used).

By combining two doped regions of semiconductors (usually Silicon), respectively P (boron) and N (arsenic or phosphorous) doped, we obtain an initial condition where:

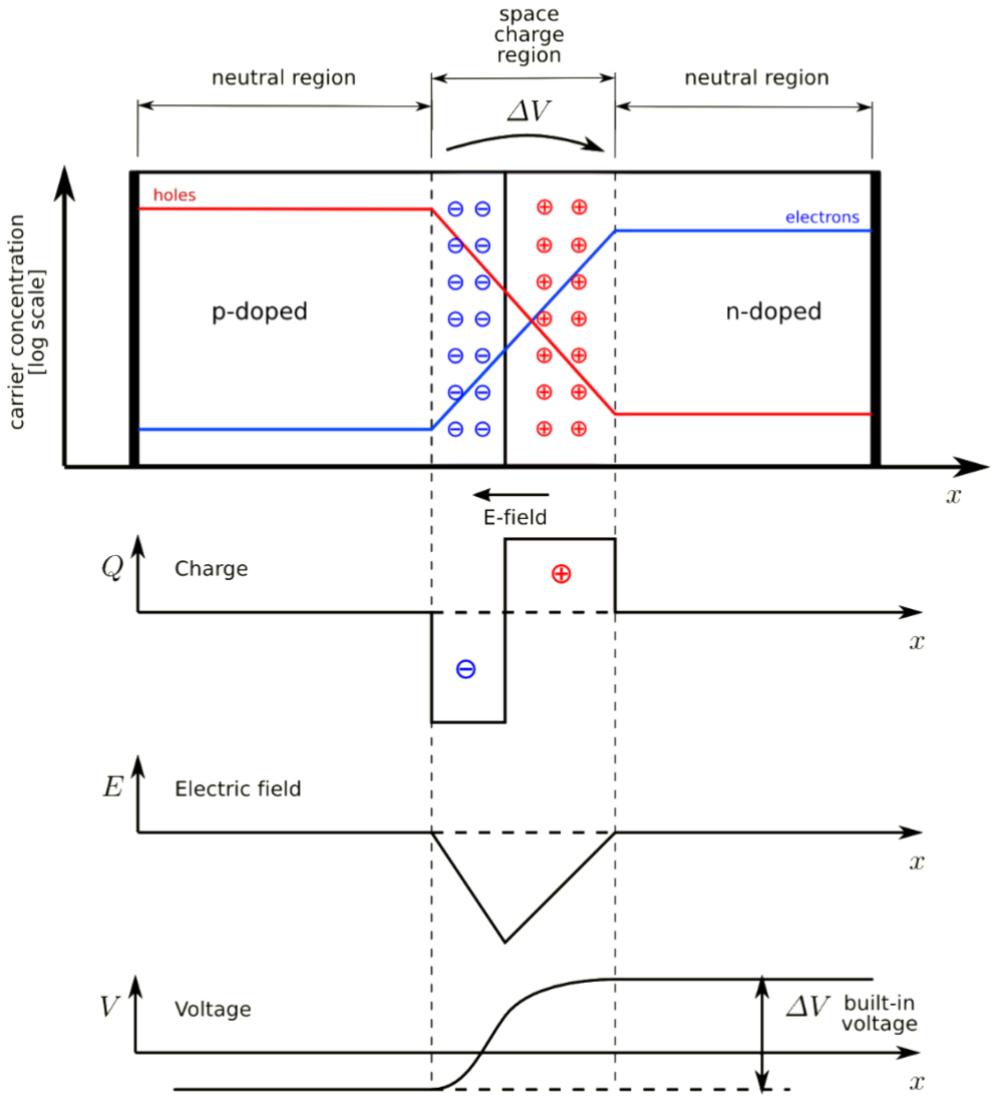
- Electrons from the n side flow by diffusion to the p side: fixed positive ions are left in the n side.

- Holes from the p side flow by diffusion to the n side: fixed negative ions are left in the p side.

At thermal equilibrium:

- Fixed ions in the two sides create an electrical field which opposes the further diffusion of electrons and holes, this is called a *depletion layer* or *space charge region*;
- A built-in voltage V_{bi} is established;
- The built-in voltage opposes current, acting as a barrier, or *threshold*, which must be overcome in order to have current flowing.

The usual value of the voltage needed to get current flowing through a diode is around $V_\gamma \approx 0.7V$.



Applying a voltage to the junction, that is applying a *forward bias*, we get mutliple effects around V_γ:

- No current flows if $v_D < V_\gamma$;
- Current starts flowing if $v_D > V_\gamma$.

In *reverse bias*, no current flows and depletion layer width increases. For large reverse voltages, we see *breakdown* of the junction, followed by a large reverse current.

This effect is the combined result of *tunneling effect* or *avalanche effect*.

There is no damages to the diode, but the large current may increase power dissipation and heat, causing damages. This effect is exploited in Zener diodes, which break at specific reference voltages.

2 Lesson 24-09-26

We continue treating *active components*, starting from the **current-voltage** characteristic of the Diode:

Definition 2.1: Schottky's law

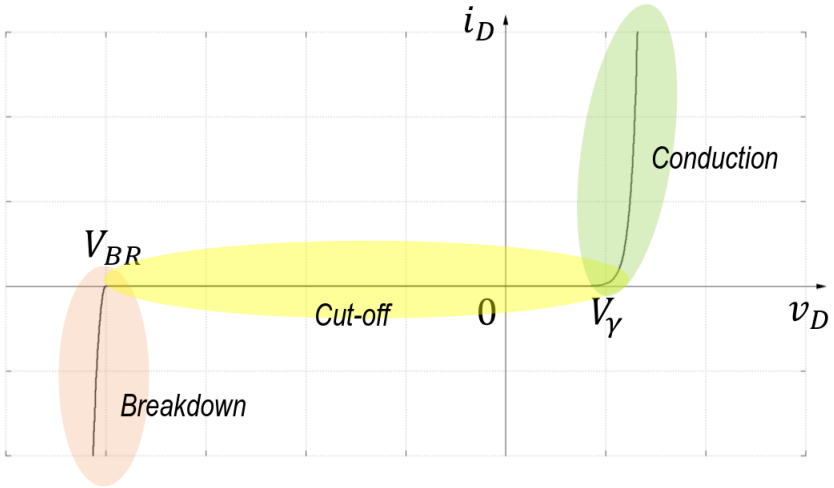
The current-voltage characteristic of a diode is:

$$i_D(t) = I_S \left(e^{\frac{v_D(t)}{\eta V_T}} - 1 \right)$$

Where:

- I_S is the *saturation current*, interpreted as the maximum reverse current we can force through the diode;
- η is the *ideality factor*, between 1 and 3 (usually simplified to 1);
- V_T is the *voltage equivalent of temperature*, around 26 millivolts.

Of course, the breakdown effect experienced by the diode for $v_D(t) << 0$ isn't taken into account in Schottky's law. We usually take approximated models where the diode starts acting like a voltage source if the revers current conduction constraint is satisfied.



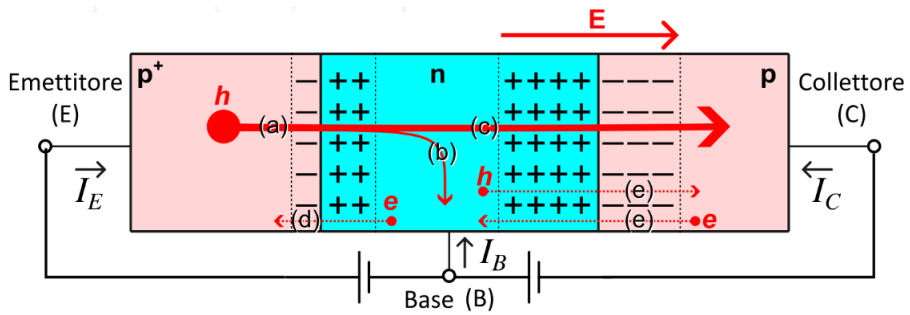
Diodes can be *modeled* in a variety of simplified ways:

- **Ideal switch**: when voltage drop is negative, current is 0; when current is positive, the voltage drop is 0;
- **Ideal switch** with *offset*: when voltage drop is less than V_γ, the current is 0; when current is positive, the voltage drop is V_γ;

- **Ideal switch** with *dynamic resistance*: when voltage drop is less than V_γ, the current is 0; whn voltage drop is more than V_γ, the current follows a resistive law. This is equivalent to applying a small series resistance when the ideal diode is conducting.

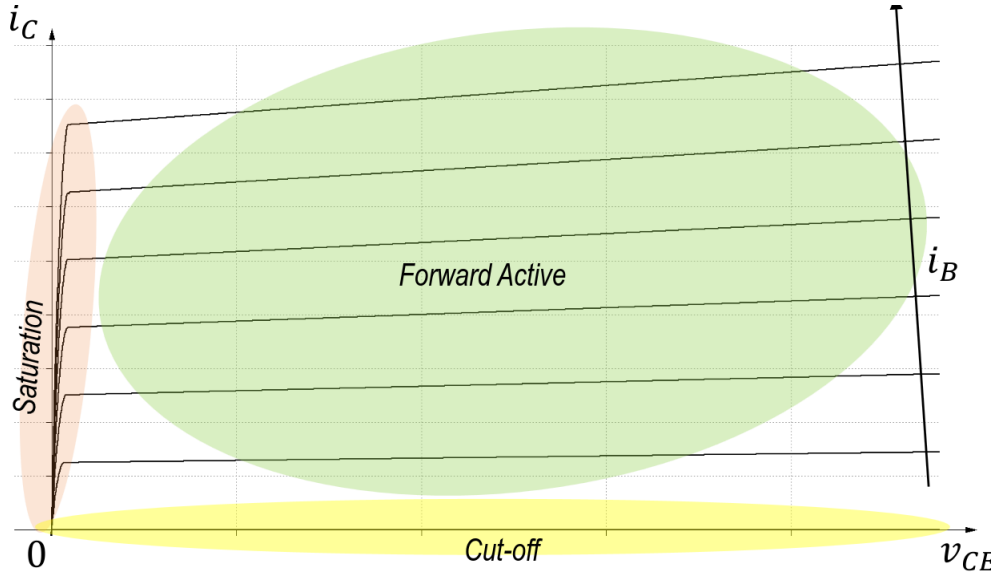
The way circuits are solved by using approximate models is by first making an *assumption* about the state of a diode (conducting or not), and then checking that assumption. Since the models have 2 states, for a diode we can assume that if it's not one state, it's the other. For n diodes we have to make n choices, and thus explore a 2^n assumption space.

BJT transistors are active semiconductor components, commonly used as amplifiers or current controlled switches. They also are a non-linear component, built from 2 PN junctions put head-to-head (which we know on their own made diodes). Two types of BJT transistors exist: PNP and NPN. NPNs are more common.



The BJT transistor is *tripole* made up of:

- **Emitter**: emits charge carriers scaled by the base current;
- **Base**, used to control the device via current;
- **Collector**, collects charge carriers emitted by the emitter.



BJTs work in 3 main regions:

- **Forward active** region, where the collector current is βi_B, so a direct scaling of the base current by a gain factor β. The voltage drop of the base on the emitter is taken with the usual diode law as $V_\gamma \approx 0.7V$. A *reverse active* region also exists but is usually undesirable;

- **Cut-off** region, where the collector current is 0 and the BJT acts as an open switch;
- **Saturation** region, where the v_{CE} between collector and emitter is approximately 0 (around $0.1 \sim 0.2V$). In this region the collector cannot give more current and the BJT acts as a closed switch.

MOS transistors are also active semiconductor components, like BJTs, commonly used as amplifiers or current controlled switches. They use a different technology to obtain basically the same characteristics: historically they were proposed around the time BJTs started becoming popoular, but weren't implemented until much later.

The main difference is that MOS transistors act as *voltage controlled*, instead of *current controlled* devices.

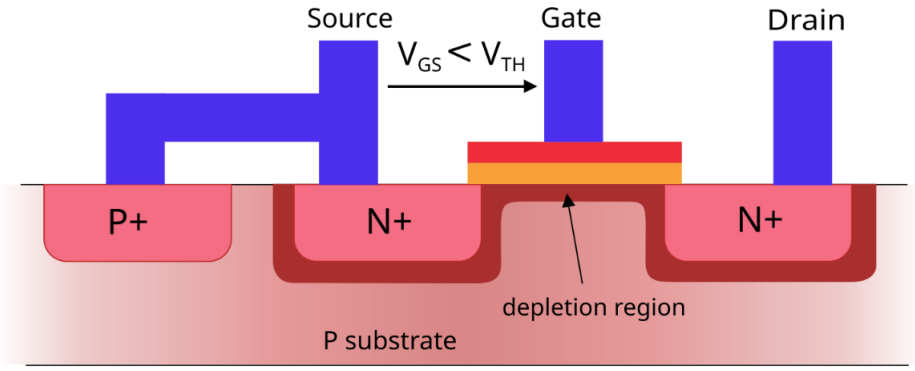
MOSes consist, briefly, of three superimposed layers:

- A base layer of doped semiconductor, which is usually silicon (Si) doped P;
- An intermediate oxide layer, which historically was silicon oxide (SiO_2) and today is mainly polysilicate. In general, we assume that the oxide is a perfect insulator (in reality, it never is, but this approximation is useful to us);
- An upper metal layer, which historically was aluminum (Al).

There are then two additional electrodes, connected to N-doped regions in the P-type substrate. We call these electrodes **source** and **drain**. At this point, the bottom electrode G is called **gate**.

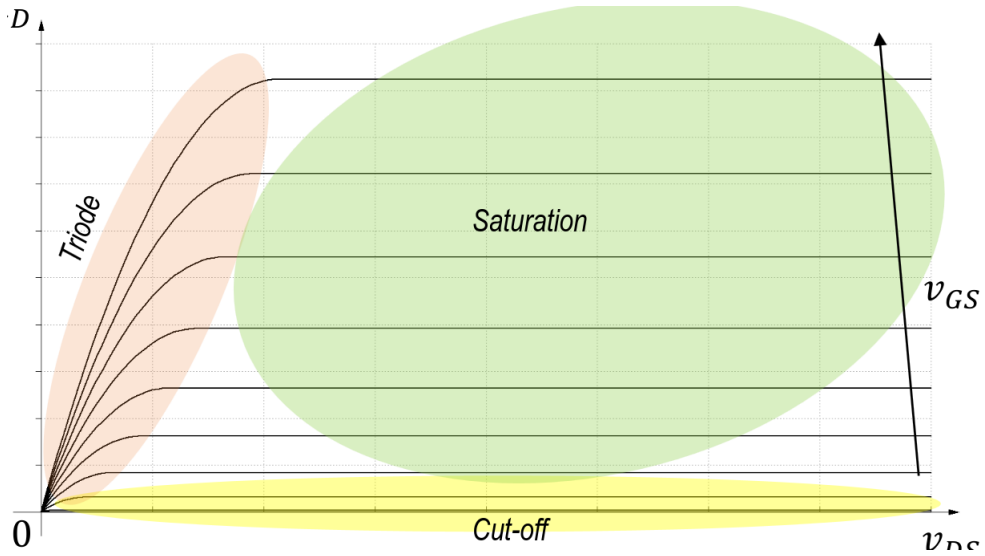
We can see that between the P-type substrate and the two N-type electrodes (source and drain), *diodes* are essentially formed, preventing current from flowing from the source and drain into the substrate.

For the MOSFET to operate, therefore, the N-doped regions need to extend beneath (or at least slightly overlap) the oxide. We call the distance L between the two N-doped regions the *gate length*.



So, we can say that a MOS transistor is another *tripole* made up of:

- **Source**: emits charge carriers scaled by the base voltage;
- **Gate**, used to control the device via voltage, it is always isolated from drain and source;
- **Drain**, collects charge carriers emitted by the emitter.



MOSes work in 3 main regions:

- **Cut-off** region: when the v_{GS} voltage between gate and source is lower than a certain v_{th} threshold voltage, the drain current is zero and the MOS behaves as an open circuit;
- **Triode** or *linear* region: when the v_{GS} voltage is higher than v_{th}, but v_{DS} is less than $v_{GS} - v_{th}$, that is:

  $$v_{GS} > v_{th}, \quad v_{DS} < v_{GS} - v_{th}$$

  then we are in a linear region where current scales mostly directly with voltage applied to v_{DS}. The exact law is:

  $$i_D = \mu C_{ox} \frac{W}{L} \left((v_{GS} - v_{th}) v_{DS} - \frac{v_{DS}^2}{2} \right)$$

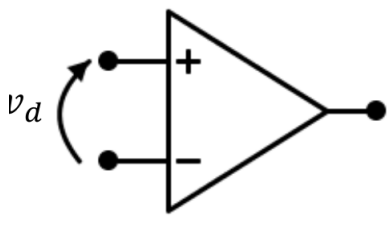
  where C_{ox} is the oxidation coefficient of the oxide (usually a polysilicate), and W and L are respectively the width and the length of the MOS gate channel;
- **Saturation** region: when the v_{GS} voltage is higher than v_{th}, but v_{DS} is higher than $v_{GS} - v_{th}$, that is:

  $$v_{GS} > v_{th}, \quad v_{DS} > v_{GS} - v_{th}$$

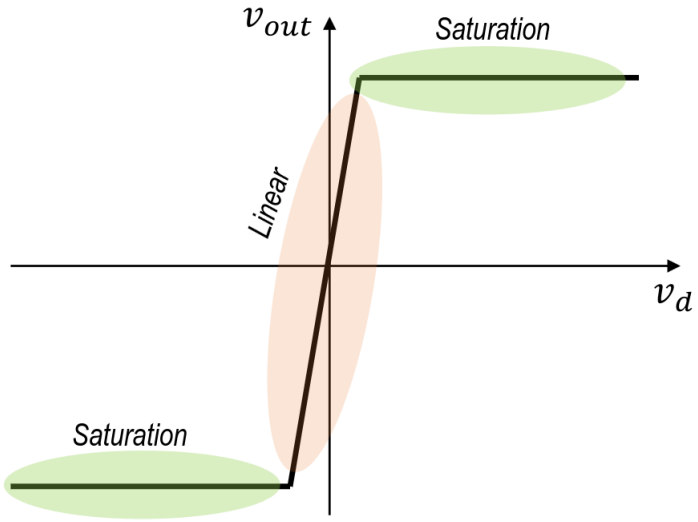
  then we are in saturation, where the current scales directly with v_{GS} indipendently of v_{DS}. The exact law is:

  $$i_D = \frac{1}{2} \mu C_{ox} \frac{W}{L} (v_{GS} - v_{th})^2$$

Operational amplifiers are complex components comprised of DC-coupled differential amplifiers with very high gain values.



What OP amps basically do is take the difference in voltage between two terminals (usually labeled $+$ and $-$) and amply that difference with an extremely high open loop gain (G_{OL}).



This means the output characteristic is:

$$v_{out} = G_{OL}(v^+ - v^-) = G_{OL}v_D$$

Let us note that, even if G_{OL} is in the order of several millions, an OP amp is a powered device which requires power and ground rails, and that cannot return voltages lower than the ground rail or higher than the power rail (a phonmenon known as *saturation*).

They are most commonly used as a **linear amplifier**:

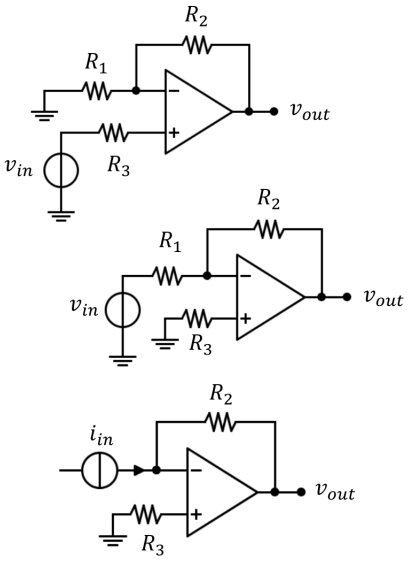
- With a negative feedback network;
- Forced to work in linear region;
- Lower final gain, larger bandwidth, stability.

Or used as a **comparator**:

- With a positive feedback network (or no feedback);
- Forced to work in saturation regions.

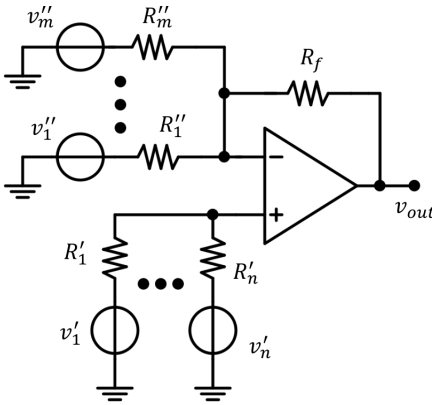
They are widely used in many embedded systems.
Below, a few common circuits with OP amps are noted.

- Non inverting amplifier
  - $v_{out} = v_{in}\left(1 + \frac{R_2}{R_1}\right)$, $R_{in} = \infty$, $R_{out} = 0$
  - R_3 used to reduce unwanted output offsets
- Inverting amplifier
  - $v_{out} = v_{in}\left(-\frac{R_2}{R_1}\right)$, $R_{in} = R_1$, $R_{out} = 0$
  - Note the non ideal (low) input resistance
- Transresistance amplifier
  - $v_{out} = -R_2 i_{in}$, $R_{in} = 0$, $R_{out} = 0$



We note that transresistance amplifiers will be useful to us when we need to implement analog converters.

Generalized analog adders / subtractors can be designed via OP amps by connecting inputs we want to add to the non inverting input and inputs we want to subtract to the inverting input.



Let's make sure to clearly define what we're after before delving into the implementation details. We ideally want a network based on a negative feedback OP amp amplifier, with a single R_f resistance as the feedback network.

- On the non-inverting input, we'll connect in parallel n voltage generators v'_1, v'_2, ..., v'_n, with series resistances of R'_1, R'_2, ..., R'_n;
- On the inverting input, we'll connect in parallel a m voltage generators v''_1, v''_2, ..., v''_n, with series resistances of R''_1, R''_2, ..., R''_n.

We want the output of this network to perform the following operation on the voltage sources, taken as inputs:

$$V_{out} = \sum_{i=1}^{n} A'_i v'_i - \sum_{i=1}^{m} A''_i v''_i$$

- Let's recall that the inverting input gain can be taken as:

$$V_{out_j}'' = -\frac{R_f}{R_j''} v_j''$$

- Whereas the non-inverting input gain can be taken as:

$$V_{out_j}' = \left(1 + \frac{R_f}{\|_{i=1}^{m} R_i''}\right) V_j'$$

The idea is to choose the resistances R_i' and R_i'' such that the expression of V_{out} above holds.

- Choosing the resistances on the inverting side is really easy as we can simply say:

$$R_i'' = \frac{R_f}{A_i''}$$

  and in the inverting input gain equation we get the correct gain;
- For the inverting side, we first compute the voltage on node V^+ through Millman's theorem (theorem 1.2, of which we gave a proof in section 1.2.5):

$$V^+ = \frac{\sum_{i=1}^{n} \frac{v_i'}{R_i'}}{\sum_{i=1}^{n} \frac{1}{R_i'}}$$

  From this equation we see that a good starting point is setting:

$$R_i' = \frac{R_f}{A_i''}$$

  so that the gains at the numerator are correct:

$$V^+ = \frac{\sum_{i=1}^{n} A_i' v_i'}{\sum_{i=1}^{n} A_i'}$$

  Now we use the non-inverting input gain to compute the output of the non-inverting side:

$$V_{out}' = V^+ \left(1 + \frac{R_f}{\|_{i=1}^{m} R_i''}\right) = \frac{\sum_{i=1}^{n} A_i' v_i'}{\sum_{i=1}^{n} A_i'} \left(1 + \frac{R_f}{\frac{1}{\sum_{i=1}^{m} \frac{1}{R_i''}}}\right)$$

$$= \frac{\sum_{i=1}^{n} A_i' v_i'}{\sum_{i=1}^{n} A_i'} \left(1 + R_f \sum_{i=1}^{m} \frac{1}{R_i''}\right) = \frac{\sum_{i=1}^{n} A_i' v_i'}{\sum_{i=1}^{n} A_i'} \left(1 + \sum_{i=1}^{m} A_i''\right)$$

  At this point we impose the output be equal to the positive side of the original output equation:

$$\frac{\sum_{i=1}^{n} A_i' v_i'}{\sum_{i=1}^{n} A_i'} \left(1 + \sum_{i=1}^{m} A_i''\right) = \sum_{i=1}^{n} A_i' v_i' \implies \frac{1 + \sum_{i=1}^{m} A_i''}{\sum_{i=1}^{n} A_i'} = 1$$

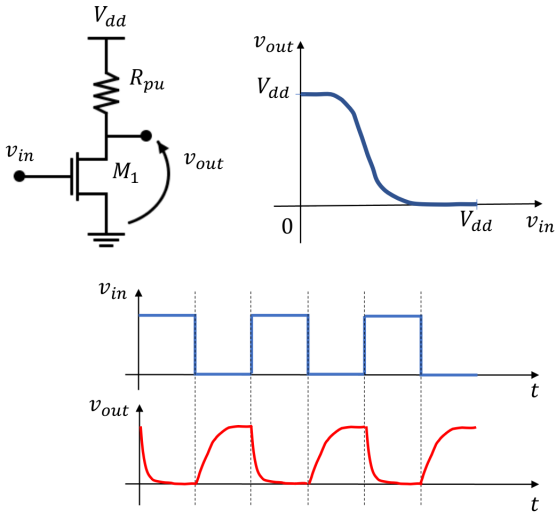
This last equation we obtained is what we have to satisfy in order to get the correct resistor values. It's pretty simple to make the result right by just adding a resistance on the inverting or non-inverting path based on the value of the left hand side, which we'll call α:

$$\alpha = \frac{1 + \sum_{i=1}^{m} A_i''}{\sum_{i=1}^{n} A_i'}, \quad \begin{cases} A_{n+1}' = 1 - \sum_{i=1}^{n} A_i' + \sum_{i=1}^{m} A_i'' \\ A_{m+1}'' = -1 + \sum_{i=1}^{n} A_i' - \sum_{i=1}^{m} A_i'' \end{cases}$$

2.1 Logic gates

We can move on to digital electronics concept, by introducing **logic gates**. A logic gate is an electronic component which performs some sort of operation on digital signals.

A simple example of a logic gate is an **inverter**. The simplest inverter is given by a *resistive load* inverter.



What happens in the circuit is that if we raise v_{in}, and thus v_{GS}, above the threshold voltage v_{th}, then we get conduction betweeen the drain and source of the MOS transistors, and the circuit acts as an open switch. If we then sample v_{out}, we see the voltage after the fall on R_{pu} (which will be close to zero).

In the other two graphs we see two non idealities of this circuit:

- **Asymmetric** response: the falling edge is much sharper than rising edge, as the R_{DS} of the NMOS is much smaller than R_{pu};
- We have **static** power consumption in the on state (when output is at logic 0), as current flows from drain to source:

$$P = \frac{v_{DD}^2}{R_{DS} + R_{pu}}$$

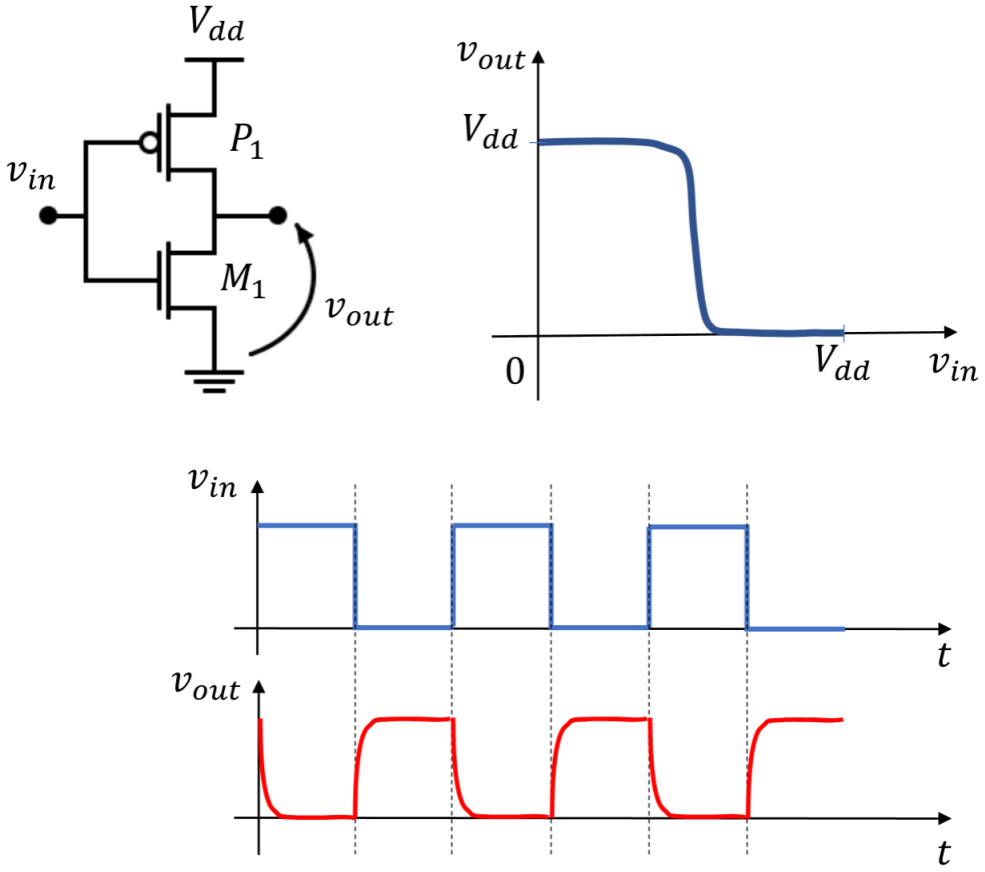
- We also have a certain amount of **dynamic** power consumption, given by the transients happening at state changes. These rely on equivalent capacitances of wires and MOS transistors, and will be studied in more detail later.

We can then see that we need a better design, at least in term of response alignment, and absolutely without static power consumption.

3 Lesson 25-09-26

In the last lesson we introduced the resistive load inverter. We said that the main problem of such a design is the asymmetric response, and static power consumption (as in the output off state we have a closed path from power source to ground, draining power on a resistor). Today we try to solve those problems.

CMOS inverter: a better inverter can be designed by using **Complementary** *MOS* (**CMOS**) designs.



We see that in such a design:

- The response is **symmetric** (or mostly symmetric, we can act on the gate sizes to account for mobility effects);
- There is no **static** power consumption (in the ideal case, some leakage does happen);
- **Dynamic** power is present and measurable based on equivalent capacitance of the wire and the MOS transistors:

$$P \approx C_{e1} f V^2$$

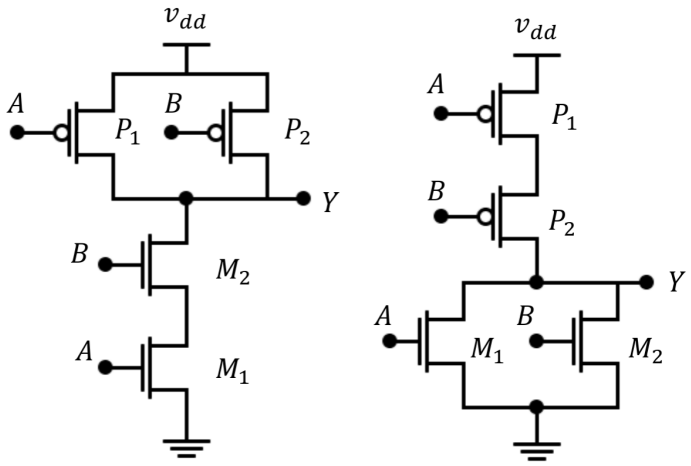
This design is very common in digital circuits nowadays, as it performs much better than the resistive load inverter.

CMOS logic: other logic functions are needed to implement a logic family. For CMOS technolog, we can use combinations of transistors in series or in parallel to implement any kind of logic.

In particular:

- In the pull-down branch:
  - Series NMOS implements a NAND
  - Parallel NMOS implements a NOR
- In the pull-up branch:
  - Series PMOS implements a NOR
  - Parallel PMOS implements a NAND

We show a figure of the synthesis of a NAND and NOR circuit in CMOS technology:

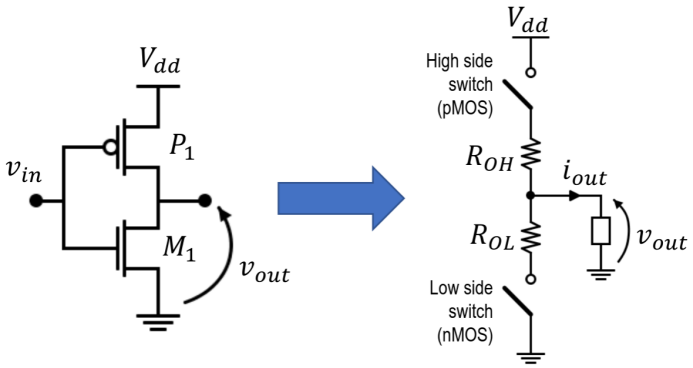


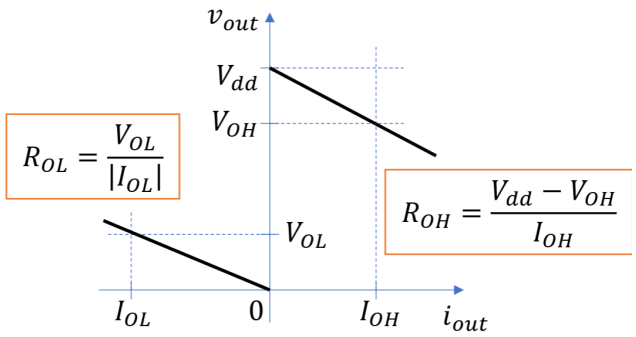
$$Y = \overline{A \cdot B} \qquad Y = \overline{A + B}$$

Logic gate parameters: we can define input and output parameters for logic gates, regarding both *current* and *voltage*.

With regards to **current**, we can model a CMOS inverter converting the MOS transistors to voltage controlled switches (ideal) with series resistances modeling leakage. This way we see we can define the parameters:

- V_{OH}, which is the lowest output voltage to get a logic high output;
- V_{OL}, which is the highest output voltage to get a logic low output;
- I_{OH}, which is the maximum output current such that the output voltage at logic high is greater than V_{OH};
- I_{OL}, which is the maximum output current such that the output voltage at logic low is less than V_{OL}.

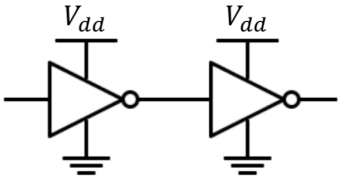


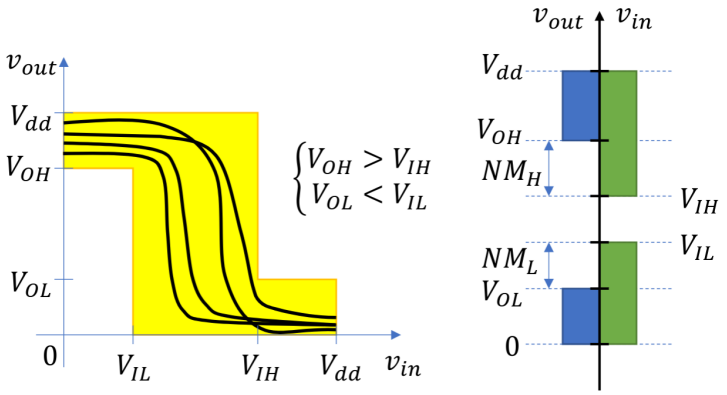


From these parameters (given by constructors) we can compute the value of the modeling resistances:

$$R_{OL} = \frac{V_{OL}}{|I_{OL}|}, \quad R_{OH} = \frac{V_{dd} - V_{OH}}{I_{OH}}$$

Now, with regards to input and output **voltages** in stacked gate setups, we can consider the following:





Here we have a pair of connected inverters. We are interested on whether ouput voltages of the first inverter will be correctly interpreted at the input of the second inverter, so we define the following parameters:

- V_{IH}, which is the lowest input voltage recognized as logic high;
- V_{IL}, which is the highest input voltage recognized as logic low;

From this we can derive stability *conditions* for the whole logic family:

$$V_{OH} > V_{IH}, \quad V_{OL} < V_{IL}$$

where we define the following margins:

$$V_{OH} - V_{IH} = NM_H \quad \text{(noise margin high)}$$

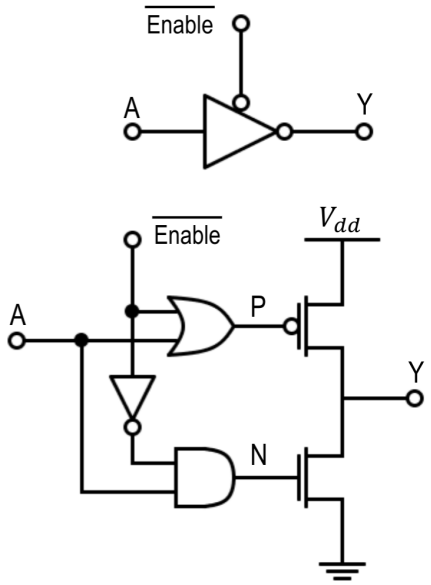
$$V_{IL} - V_{OL} = NM_L \quad \text{(noise margin low)}$$

Tri-state outputs are needed to avoid logic gates shorting when connected to the same outputs.

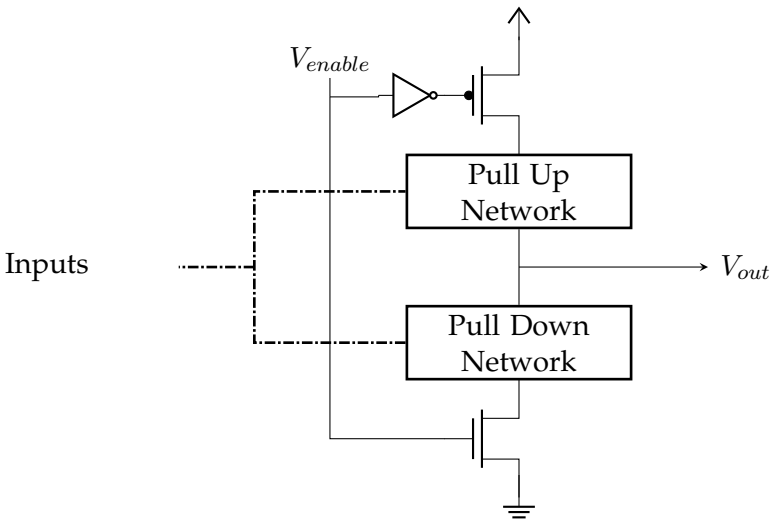
In these designs we have an additional Enable input, which can be *active high* or *active low*:

- When Enable is active, the output is driven normally;
- When Enable is NOT active, both the pull-up (high side) and pull-down (low side) transistors are switched OFF, and the output line is left floating. At this point the voltage level (and consequently the logic level) is set by some other circuit, or remains undetermined.

Below, we show the symbol of a tri-state inverter and a possible CMOS implementation.

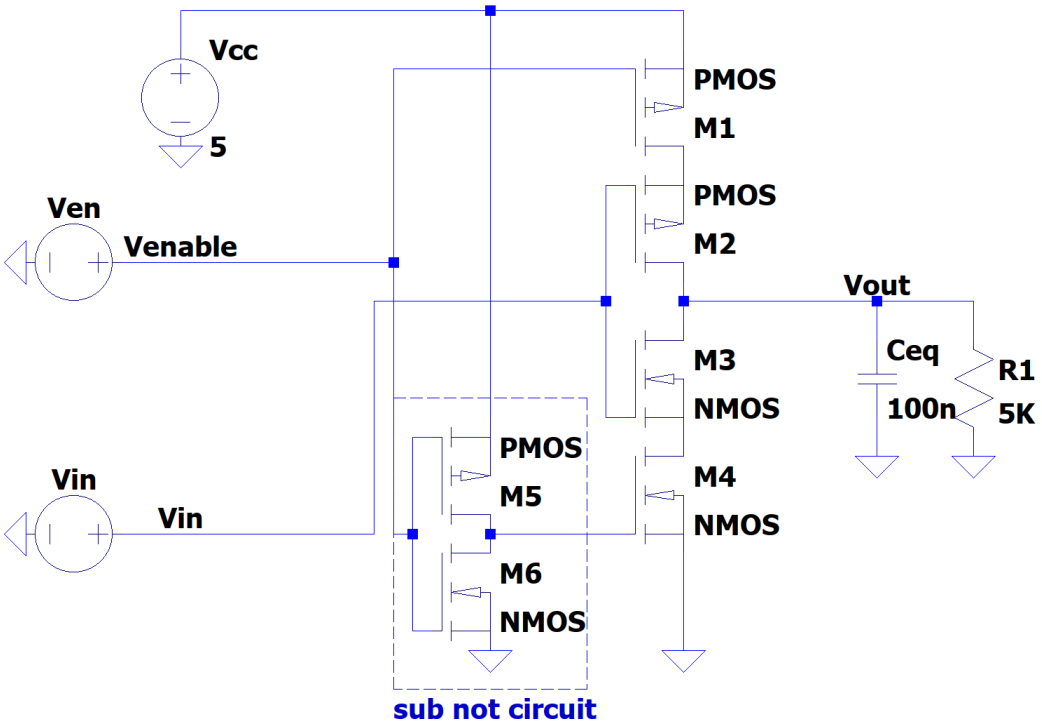


A better design (or which at least uses less components in CMOS logic) for enabled inverters can be obtained using a general method for tri-stating CMOS network outputs. Because the outputs of CMOS logic networks are only given by shortcircuiting to V_{cc} (via the **PUN**, *Pull Up Network*) or to ground (via the **PDN**, *Pull Down Network*), we can fully isolate the output by separating the PUN and PDN's connection to V_{cc} and ground respectively. This is better explained by the following circuit:



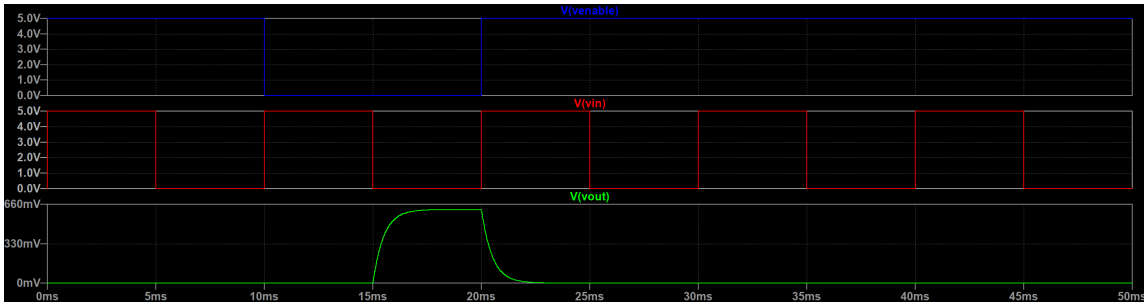
We note that this implementation is active *high*, as when V_{enable} is high the connection to ground of the PDN is broken, and after the inverter (which pulls the enable signal to low) the connection to V_{cc} of the PUN is broken. We can easily turn this circuit into an active *low* implementation by just swapping the positino of the NOT gate.

Lastly, we show how this technique can be used to design a CMOS inverter with enable signal, in a simulator such as LTSPICE:



In the circuit, we implement the active *low* version, and implement the NOT circuit manually via 2 other MOS transistors. We also note the RC network at the end of the circuit: this is useful in testing, as it simulates an actual driven line which might have capacitance and resistance (exagerated here for the example).

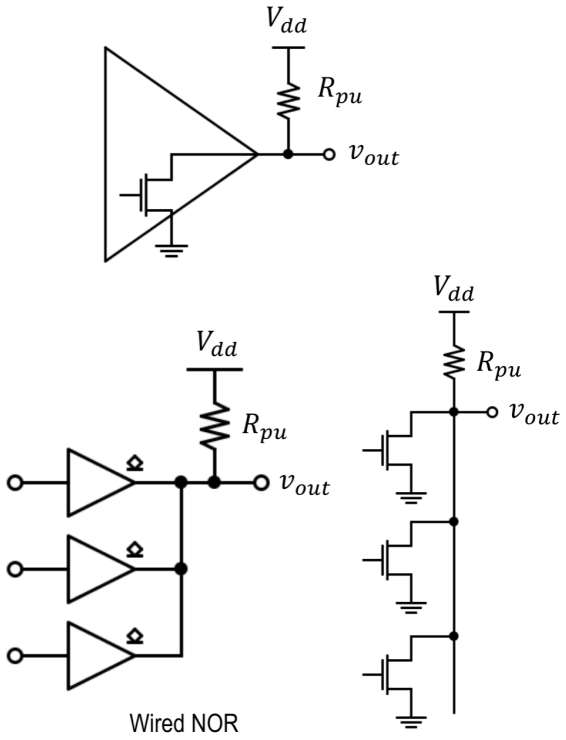
We see how such a network behaves by driving it. In the figure below, after a delay of $10\,\mathrm{ms}$ the inverter is enabled, we do a low to high transition, and then it is disabled again. We see how we can temporarily drive the output line, which then goes back to $0\,\mathrm{V}$ (by draining thorugh the RC network) after disabling the inverter.



As we said, tri-state outputs are useful when multiple logic gates drive the same line, for instance in a shared bus. All drivers must be in high impedance except one: it is vital to guarantee the above condition to avoid short circuits.

Open collector is another technology which allows to drive the same data line, even without knowing the voltage supply value.

In this setup, there is no pull-up transistor at the output: we can only set the output to logic 0. To set a logic 1, we switch off all pull-down transistors, and use an external pull-up resistor.



This is useful to drive the same line with multiple logic gates:

- No possible short circuit: when all logic gates set a logic 1 the output is at logic 1;
- When at least one logic gate sets a logic 0, the output goes at logic 0. This makes open collector a NOR logic function.

This is slighly easier to implement than the earlier tri-state solution, as we don't have to deal with high impedance or short circuit issues.

Of course, we can imagine an *open drain* setup which works the same, only leaving the pull-down transistor outside of the design and pulling *up* thorugh an external resistor.

3.1 Combinatorial circuits

Combinatorial circuits are the simplest we can see with digital logic circuits. In a combinatorial logic circuit, the logic output depends on the current value of the inputs only (no state).

A combinatorial circuit can be described through a **truth table**, which is a list all possible input combinations, and corresponding output values.

Many ways exist to synthesize a combinatorial circuit:

- The main idea is generally to generate the circuit as a **SoP** (*Sum of Products*), minimizing the number of products, and minimizing the number of literals in each product (which corresponds to minimizing components).
- A possible tool is given by *Karnaugh maps*;
- Another possibility is the *Quine-McCluskey algorithm* for more inputs;
- Often, we just stick to logic synthesis *software tools*, which implement these ore other advanced algorithms.

Some other help about this topic can be found in the author's notes at `https://seggiani-luca.github.io/appunti/triennale/2/reti-logiche.html`.

Multiplexers / **demultiplexers** are circuits which allow to select an input and output just one of them, between different inputs. A one bit multiplexer with 2 inputs has this boolean law:

$$y = a \cdot s + b \cdot \overline{s}$$

where a and b are the inputs, and s is the selection signal.

For multiplexers on multiple bits, we repeat the same structure multiple times; For multiplexers with more than two inputs, we need more than one selection signal (usually built hierarchically with $\log$ number of inputs smaller multiplexers). Multiplexers with more than two inputs on multiple bits are built as a combination of the above two cases.

We can summarize this via the following distinction:

- We can expand multiplexers **vertically**, that is add more multiplexers vertically with the same control variables, to extend the width (in bits) of the inputs;
- We can expand multiplexers **horizontally**, that is build hierarchical structures with more control signals, to extend the number of inputs available.

One technique commonly used to build multiplers with MOS transistors is *pass-transistor* logic. Pass-transistor logic involves using an NMOS and a PMOS transistor in parallel to build a (basically) ideal voltage controlled switch. This can implement the logic function of a 2 to 1 multiplexer (above) with less transistors than the equivalent CMOS solution. At this point, more 2 to 1 multiplexers can be expanded as we saw above to have wider or more inputs.

We also note that the inverse of a multiplexer is called a **demultiplexer**, that is a component with multiple outputs, to which we can route a single input based on control signals.

Decoders / **encoders**: **decoders** are components which decode the binary representation of a number, on n bits, to the same number in one-hot representation, on 2^n bits. **Encoders**, on the other hand, do the reverse job of converting a one-hot representation on 2^n bits in a single n bit encoded value.

More complex components like *priority encoders*, ... can be built, but usually an encoder is in an inconsistent state when multiple input lines are high.

Decoders and encoders are still utility logic circuits, but as opposed to multiplexers and demultiplexers they are more easily implemented with standard CMOS logic, or by cascading CMOS logic devices (such as NOT, NAND and NOR gates).

Digital adders / **subtractors** are built in combinatorial logic too.

We start from *half adders*: these operate on 1-bit operands, generating a carrying output bit. For *multibit* adders we combine multiple half adders.

Different area and performance can be achieved based on carry computation. For instance, we remember:

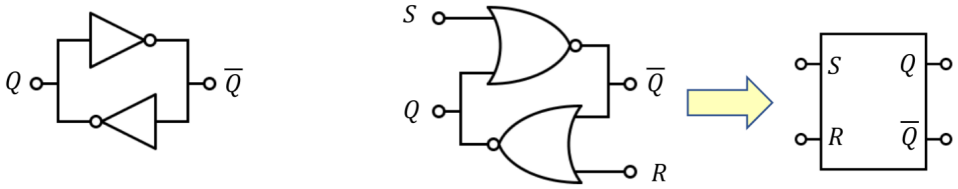
- **Ripple Carry**: this is the traditional method of stacking full adders, where we connect a C_{in} (carry in) and C_{out} (carry out) pin for each adder pair;
- **Carry Look Ahead**: with carry look-ahead we compute the carries in place via a CLA network, and perform the actual sums in parallel. This adds logic but saves time;
- **Carry Select**: with carry select we compute both pairs (without carry and with carry) for each adder, and only at the end of the calculation we choose the correct carry.

Subtractors are just implemented using an adder (using 2-complement notation).

3.2 Sequential circuits

Sequential circuits need a *memory element* to exist somewhere in a circuit. The most popoular memory element is given by a pair of active elements in a loop.

- A pair of inverters in a loop provides a memory element, that is an element that is *bistable*: either of the inverters can start out with an high outputs, and the state is converted. The problem of this design is that we can't control it:
- A pair of NANDS in a loop can be used in the same fashion. The input bits that are left out can be used as SET and RESET parameters for the bistable circuit. We call this a **SR latch**.

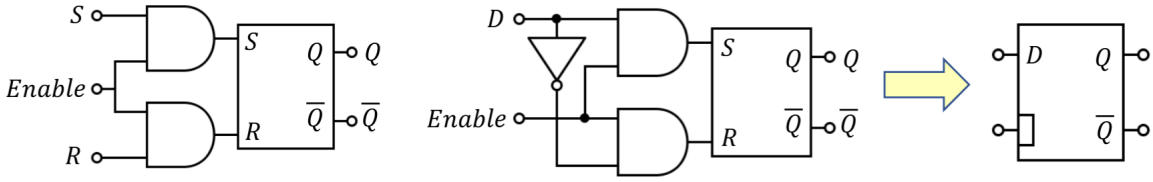


4 Lesson 30-09-26

Last lesson we finished up on sequential circuits, introducing the *bistable circuit* and making it controllable via a pair of NOR ports (implementing an *SR latch*). Let's start again from there.

SR latch w/ enable: let's add enable circuitry to the SR latch. Note that *enable* here doesn't mean setting the output to high impedance (as in tristates), but making the network unresponsive to changes in the inputs.

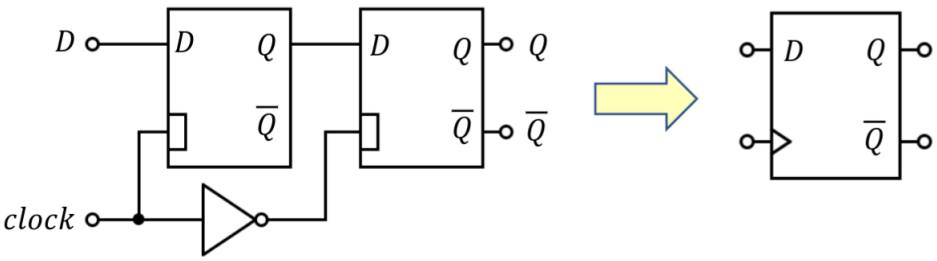
The circuits we could get are the following:



D latch: let's add enable circuitry to the SR latch.

Note that we decided to simplify the SR latch circuitry in the middle circuit, by setting S to a data value D, and R to its negation. This allows us to build what's essentialy a **D latch** (last circuit in the row).

D flip flop: this is one last memory circuit we can build, which is even more convenient than the past two.



In the figure we note the master/slave implementation, but there are many others that can be built. The idea of the D flip flop, no matter the implementation, is that the output value Q follows the input D *only* on the rising edges of the clock signal. This makes D flip flops non *transparent*: a change in the input isn't trasmitted directly to the output (but only transiently during clock rising edges): in other words, if the circuit oscillates, it oscillates synchronized to the clock. This makes this circuit extremely useful in sequential networks as a memory element (for instance in *registers*).

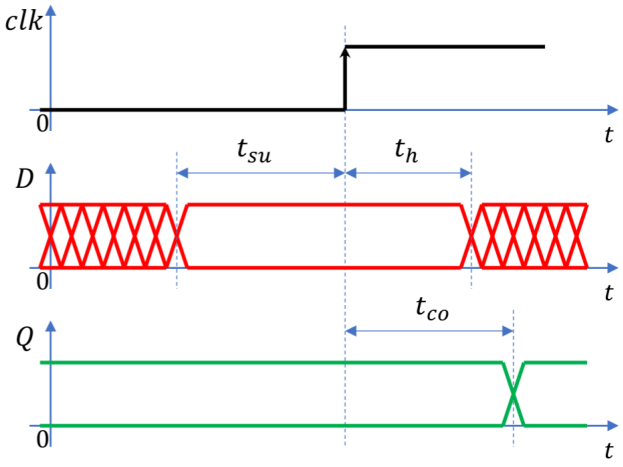
Other characteristics: let's note other characteristics of sequential networks.

Firstly, when talking about flip-flops and latches, we note that there are usually other inputs provided:

- **Asynchronous clear** (or **reset**): sets the output to 0, without waiting the active clock edge;
- **Asynchronous set**: sets the output to 1, without waiting the active clock edge;
- **Clock enable**: masks the clock input.

These signals are often active low.

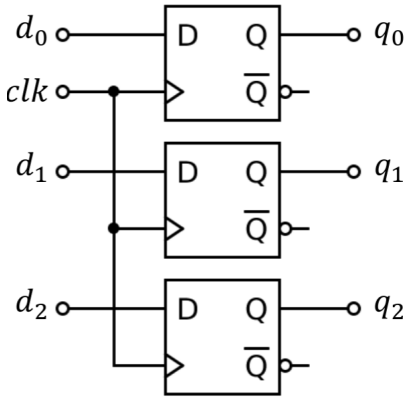
Other important characteristics are the **timing specifications** of flip-flops:



- **Setup** time: the time the data input has to be stable *before* the rising edge, for the flip-flop to correctly recognize its value;
- **Hold** time: the time the data input has to be stable *after* the rising edge, for the flip-flop to correctly recognize its value;
- **Clock to output** time: the time the flip-flop takes to propagate its input to its output after a clock rising edge.

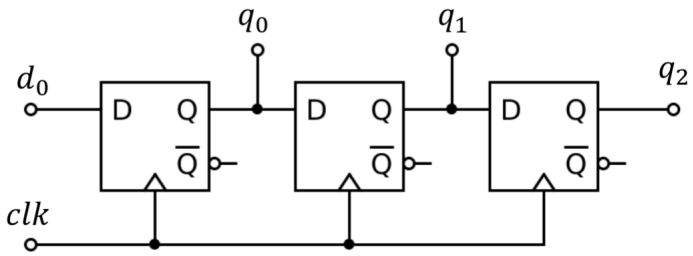
These values are often in the order of microseconds, and directly determine the maximum clock speeds we can achieve.

Registers: we introduced them, now let's look at them. A **register** in the electronics sense is nothing more than a combination of multiple flip-flops with a common clock and different data inputs. This allows registers to store whole *words* of data, of arbitrary size. Let's look at a 3 bit register as an example.



3-bit Register

Shift register: a **shift** *register*, often called a *delay line*, is like a register which takes in a single bit of data and propagates it through a set of flip-flops.



3-bit SIPO Shift Register

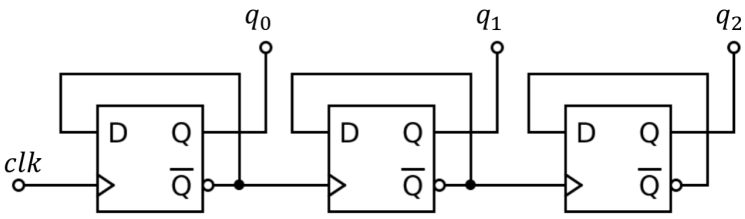
There are different kinds:

- Serial In, Parallel Out (**SIPO**);
- Parallel In, Serial Out (**PISO**): these need extra control inputs such as:
  - Parallel Load (PL)
  - Load Enable (LE)
- Serial In, Serial Out (**SISO**): these often come with parallel out, as well.

All shift registers may have a shift enable signal.

Counter: there are 2 kinds of counters:

- **Asynchronous counter**: the clock propagates from one FF to the next. This means that the most significant bits are updated later than least significant bits. This is undesirable as it entails *data races* along the FFs. Still, it is the one we show an example of as it's the simplest;
- **Synchronous counter**: the same clock is provided to all FFs. Internal logic used to compute the input to each FF: this means all output bits are updated at the same time.



3-bit Asynchronous Counter

This being said, there are other characteristics which determine the behavior of a counter:

- Up or Up/Down counters;
- Binary or Decade counters;
- Parallel Load to initialize counter value;
- Carry input and output to cascade counters;
- Asynchronous reset.

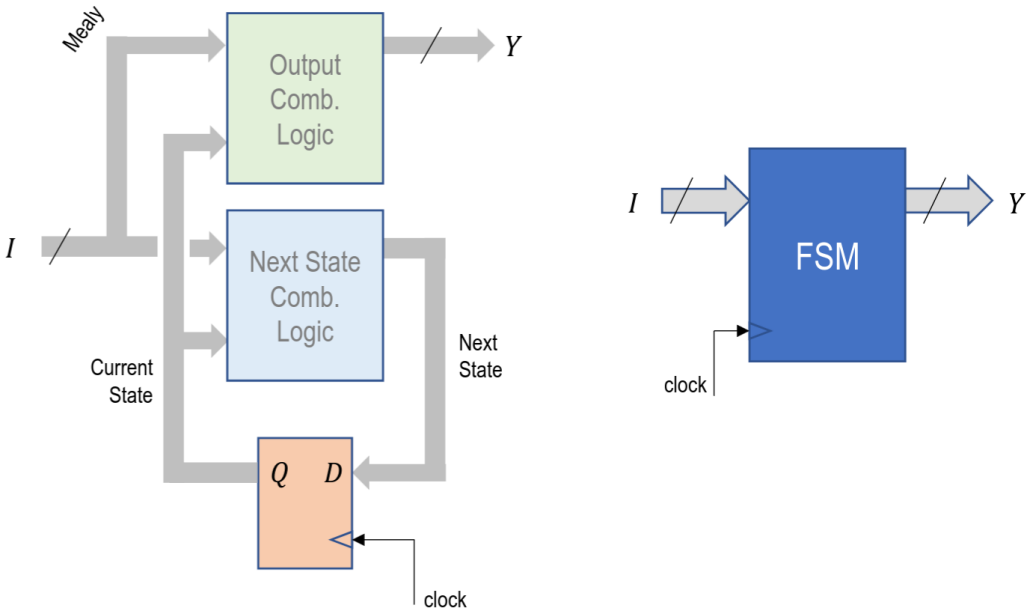
FSM: an **FSM** (*Finite State Machine*) is a graph structure consisting of *nodes*, corresponding to **states**, and *edges*, corresponding to **state transistions**. This way, a state machine defines all possible states of a digital systems and all the ways states can transition one to another.

A FSM description can also entail a network's outputs:

- Edges may have output values (*Mealy* model);
- States may have output values (*Moore* model);
- Transitions are synchronized with an external clock signals (*retarded* or *synchronized* Mealy and Moore models).

State machines can formally be described via two truth tables:

- One for *state transitions* (next state), with respect to inputs and current state
- One for *outputs*, with respect to current state (Moore) and inputs (Mealy).



To implement a state machine as a circuit we first need the *specification* of that state machine, which we said is either given by truth tables or state transition graphs. Then, we can encode the internal state in a special register (the **STAR**, or *STAtus Register*). The STAR register has to be as big as the base-2 logarithm of the state space (in *binary* encoding, as big as the state space for *one-hot* encoding).

At this point the two truth tables become two combinational logic circuits:

- One that takes in the current state (Moore) and/or the current inputs (Mealy) and returns the output of the network;
- One that takes the current state and the current inputs to determine the next state (which gets fed back to the STAR register).

4.1 VHDL

VHDL (*VHSIC Hardware Description Language*) is a hardware description language that can model the behavior and structure of digital systems at multiple levels of abstraction, ranging from the system level down to that of logic gates.

- It is used to *describe* the structure and the behavior of a **digital system**;
- The description can be used to *simulate* and *synthesize* a **digital circuit**.

The nice part of VHDL is that the syntax similar to a programming language.

The basic design element of a VHDL description is an **entity**, which is a design unit comprised of:

- The **entity** specification, which acts as a black-box interface for the component:
  - **Name** of entity;
  - **Inputs**;
  - **Outputs**;
  - **In/out** ports;
- The **architecture** specification, which describes the entity's behavior and/or internal structure. VHDL allows for a number of specification styles:
  - **RTL** (*Register Transfer Level*): uses *concurrent signal assignments*.
    These are *boolean expressions* assigned to a signal, executed in *parallel*, so without any implied order. Not that lists of concurrent assignments are *not* sequential in execution. High level operators such as arithmetic, logic and conditionals can be used;
  - **Structural**: similar to a schematic diagram, but using a textual programming language.
    We first declare *components*, then *signals* to interconnect those components, and lastly *map* signal and input/output ports to components. This allows for a hierarchical description of a circuit (Top/Down or Bottom/Up);
  - **Behavioral**: uses a process, which is a sort of an endless loop, specified in a programming language-like way.
    In most cases it uses an implicit *wait statement* on a *sensitivity list*:
    * The process suspends when it reaches the end
    * Resumes when any signal on the sensitivity list changes

Instructions in the process are, this time, in fact *sequential*, but may still be implemented in parallel to increase performance if the compiler finds it is feasible. Still, we note that signal assignments happen only *at the end* of a certain process. Higher level constructs such as if or case conditional expressions are available, variables can be defined and assignments to variables are immediate.

4.2 Introduction to embedded systems

This course and the entire degree program are focused on **embedded systems**, so it might be natural to ask *what* an embedded system even is. An embedded system is an electronic system, that is a system made up of:

- Hardware components;
- Software components;
- Communication systems.

designed to be part of a larger system, not necessarily electronic:

- Vehicles (cars, planes, ...);
- Industrial systems;
- Home appliances.

This means that an embedded system:

- Is, in fact, **embedded**, i.e. it lives in a larger environment;
- It should **react**, typically in real-time, to external stimuli;
- It is dedicated to a particular **application** (single task).

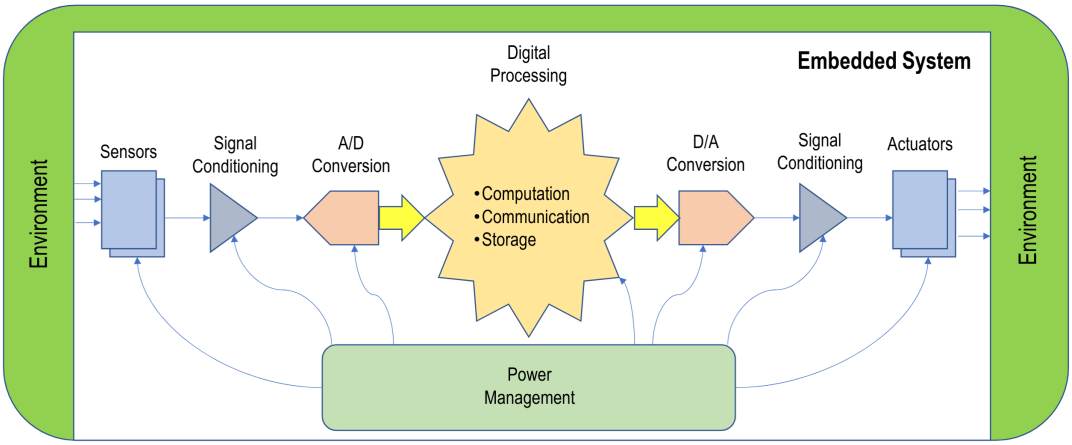
This gives designers the opportunity to highly optimize and adapt to different situations, in highly constrained environments.

Designing embedded systems, in this regard, is difficult because:

- The design space is extremely large;
- It requires skills from many very different domains;
- It requires to simultaneously optimize numerous design metrics.

4.2.1 Architecture of an embedded system

Let us look at the typical components of an embedded system:



- We already know that an embedded system lives inside an **environment**, from which it *senses* data and which it modifies by *acting* on it;
- Data will be obtained from **sensors** that return electrical values proportional to physical quantities in the environment;
- Data will be **conditioned**, which means *amplified*, *filtered*, or anyways manipulated in order to be more useful to the embedded system;
- Data will be converted from an analog signal to a digital value ready for computation by an **A/D converter**;
- The central part of the embedded system performs **digital processing** of the recovered data from the sensors, signal conditioning and A/D conversion parts. What comes out of this phase is digital signals which will be used to control the *actuators*;
- Data will be converted from the digital processing stage back into the analog domain by a **D/A converter**;
- Again, we will have some amount of **conditioning** on the output signal;
- The conditioned signal will drive **actuators**, which can actually affect the environment;
- All of these components need **power management** to be correctly powered by some power supply (often batteries, but also wall power, ...).

4.2.2 Design metrics for embedded systems

A **design metric** is a measurable feature of a system implementation. Common metrics are:

- **Unit cost** U: the monetary cost of manufacturing each copy of the system;
- **NRE cost** N: one-time monetary cost of developing (designing) the system;
- **Total cost** T: total cost of producing N_{unit} tests. Calculated as $T = N + U \times N_{unit}$;
- **Per-product cost** p: cost of a singular product, with amortized NRE costs. Calculated as $p = \frac{T}{N_{unit}} = \frac{N}{N_{unit}} + U$. Keeping the per-product cost low entails spending as little as possible on NRE (as these costs get amortized over units produced) or producing as many units as possible. Of course the unit cost U plays a significant role: lowering is sometimes useful at the cost of increasing N;
- **Time-to-market**: it is the time required to develop a product to the point it can be sold to customers. Smaller time-to-market means less losses due to delayed market entry (for instance, when competing with other products);
- **Performance**: it can be measured in many ways. For instance:
  - **Latency**, or *response time*, the time between task start and task end;
  - **Throughput**, taskes per second, usually $\frac{1}{\text{Latency}}$.

  Parallelism is a way to increase performance. Usually, N parallel blocks allow to increase throughput to $N \times \text{Throughput}_{\text{Serial}}$ (unless Ahmdal's law comes into play);
- **Technologies**, that is technical processes, methods and knowledge, are in some way a design metric.

4.2.3 Technologies

Let's look at a few **technologies** which we have available for the design of embedded systems. We noted the design space is very large, so it is useful to know about them.

Processor technology

Let's look at **processor** thecnologies.

- The *General Purpose* (**GP**) processor, as found in desktop PCs, is the classic programmable micro-processor we are used to.
  - They are not optimized for any specific application;
  - They are often cheaper than general purpose technology: billions of units are sold every year;
  - Flexible, low time-to-market: software is easy to update and modify;
  - No high performance compared to a dedicated design;
  - Not power efficient
- *Embedded* micro-processors, though, are different from PC micro-processors. For instance, they are usually within a *System-On-Chip* (**SoC**), which includes needed components and a number of peripherals.
- A method of implementing embedded processors is *Application Specific Instruction Processor* (**ASIP**). This entails:
  - A general purpose processor with specific hardware for a certain application domain, for instance **DSP** (*Digital Signal Processing*);
  - Dedicated to signal processing application;
  - Possibly dedicated hardware to drive and read external pins;
  - Cheap;
  - Flexible, low time-to-market;
  - Higher performance and better power than a GP processor.
- Another method is *Single Purpose* (custom) *Processor*. This entails:
  - Digital hardware designed specifically for a certain application;
  - Best performance;
  - Best power consumption;
  - Small size;
  - Long time-to-market;
  - Very high NRE costs;
  - Low flexibility;
  - Good for large volume productions and for extremely high performance designs.

IC technology

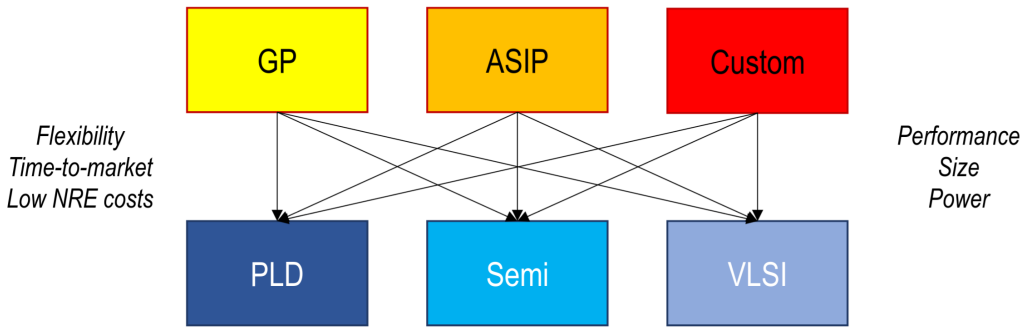
ICs (*Integrated Circuits*) also have different technologies with different use cases.

- *Full Custom* (**VLSI**, *Very Large Scale Integration*):
  - The integrated circuit is entirely designed and optimized for a particular application;
  - Transistor sizing, placing and routing;
  - Best for performance, size and power consumption;
  - High NRE costs;
  - All masks in the production process are dedicated to a single design;
  - Long time-to-market;

  - Currently used to design micro-processors (GPs and ASIPs), memories and off-the-shelf Programmable Logic.
- *Semi-Custom* (Gate Array / Standard cells) technology:
  - The lower layers are fully or partially built: design entails placing of large blocks, routing;
  - Good performance, size and power consumption;
  - Lower NRE compared to a full custom implementation;
  - Some masks are shared among more than one design;
  - Still, high NRE costs (tough lower than VLSI);
  - Long time-to-market.
- *Programmable Logic Device* (**PLD**):
  - The integrated circuit is fully built by the manufacturer: the embedded system designer buys the IC. The PLD can be customized after fabrication for a specific application;
  - Logic functions and interconnections are programmable;
  - Low NRE costs;
  - Fairly good performance and power consumption, but worse than full custom and semi-custom;
  - High unit cost;
  - Large size.

Orthogonality

Processor and IC technologies are **orthogonal**: the design space is very large, and takes very long to explore. We show a figure showing the different processor and IC technologies we considered, and their characteristic (low N vs high U, or low U vs high N).



Early design decisions are difficult, and it is useful to make use of modern design technologies to evaluate different solutions.

It can also be of use to note that the design phase of a product is also a field where more than one methodology (i.e. a technology and its associated techniques) can be used. This is particularly important in companies, which might have long historical experience with one methodology instead of another. Using well-trusted methodologies or using new ones is also an important factor to consider.

4.3 Memories

Memories are the first component of embedded systems we will encounter. They exist in the digital domain, and more specifically in the storage component of the digital processing part of an embedded system.

In general, there are different kinds of memory, which differ in types, operations available, interfaces, timings and composition of different memories to make bigger ones.

We know the usual types of memory:

- **ROM** (*Read Only Memory*), that is EPROMs, ...
- **RAM** (*Random Access Memory*), that is SRAM, DRAM, ...

We will also note the use of memories as *cache memories* and *memory controllers*.

5 Lesson 01-10-26

Last lesson we started talking about *memories*.

5.0.1 Digital memories

The kinds of memory we mainly care about are **digital** memories. These are **ordered** collections of data, where not all data is necessarily available or accessed concurrently.

Data, in our sense, is made out of n-bit *words*:

- A memory contains N_{word} words of identical size;
- Each word as a width of n_{bits};
- Each bit is a binary storage element (such as a flip-flop, even though flip-flops are usually not used in digital memories).

Words are contiguous in memory, and each word is identified by a unique index we call **address**. Addresses usually start from 0 up to $A_{\text{last}} = N_{\text{words}} - 1$.

Sizing

Since we usually address with binary counters, the number of words is always a power of 2: addresses can be expressed on:

$$k = \log_2(N_{\text{words}}) \text{ bits}$$

The number of bits in a word, on the other hand, can be any number (though usually it also is a power of 2, allowing memories to store their own addresses). Sometimes it is useful to include extra bits after 2^n bits, for parity checking or other **EDC** (*Error Detection Code*) techniques.

We can call the **size** of the whole memory the product of words per bits:

$$\text{size} = N_{\text{words}} \times n_{\text{bits}}$$

This can be interpreted as the total number of bits in the memory.

Two memories can have the same size but different organization: for instance, the same total number of bits, but different word size and number of words. The organization affects the memory interface (number and kind of access pins).

We shall also note that *external* access also affects the memory interface:

- Parallel, Serial;
- Random, Sequential;
- Special access modes.

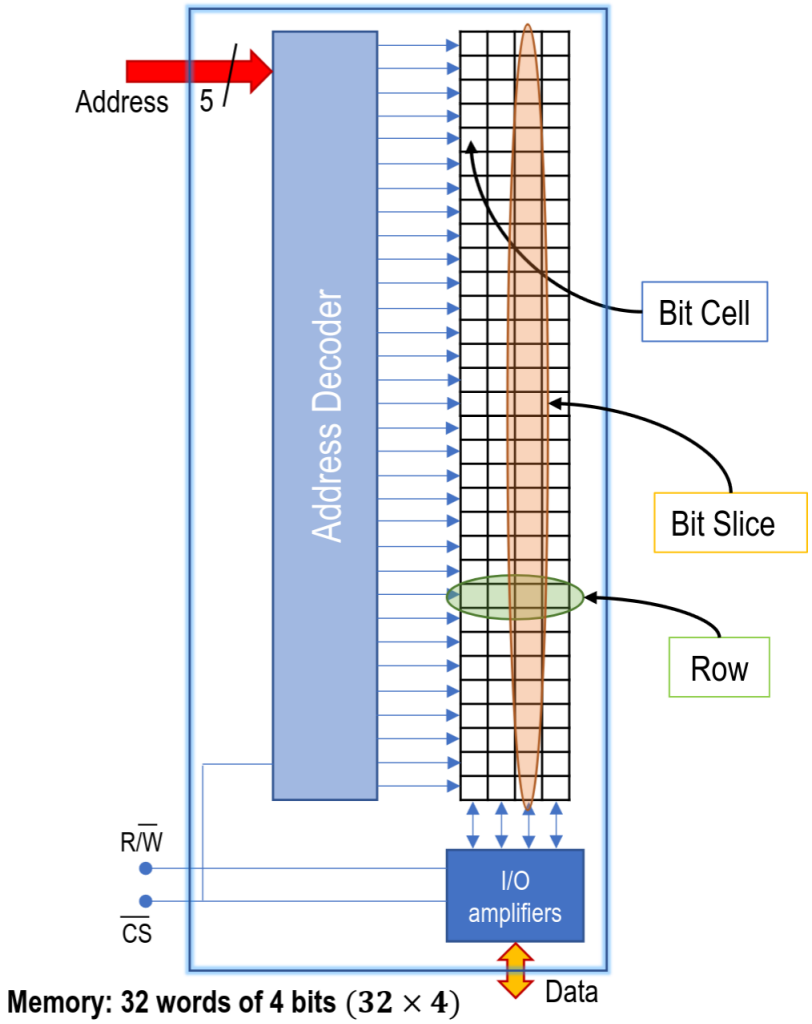
5.0.2 Sense amplifiers

We also quickly explain what *sense amplifiers* are: the signals stored within memories are usually to weak to interface to the external word. For this reason we use amplifiers. These are used to compare a *bit line* (more later) to a *reference line*, and get a biased signal. Note that the reference line can be another bit line!

5.0.3 Logical structure of digital memories

The **logical** structure of a memory, as we explained it, is an **array** of bit cells:

- One row for each word;
- One bit slice for each bit in the word.



Some *address decoder* is used to select a single row of the memory: from k address lines to 2^k word select lines. At the word interface, a sense amplifier and I/O circuit are used to interface to the external world.

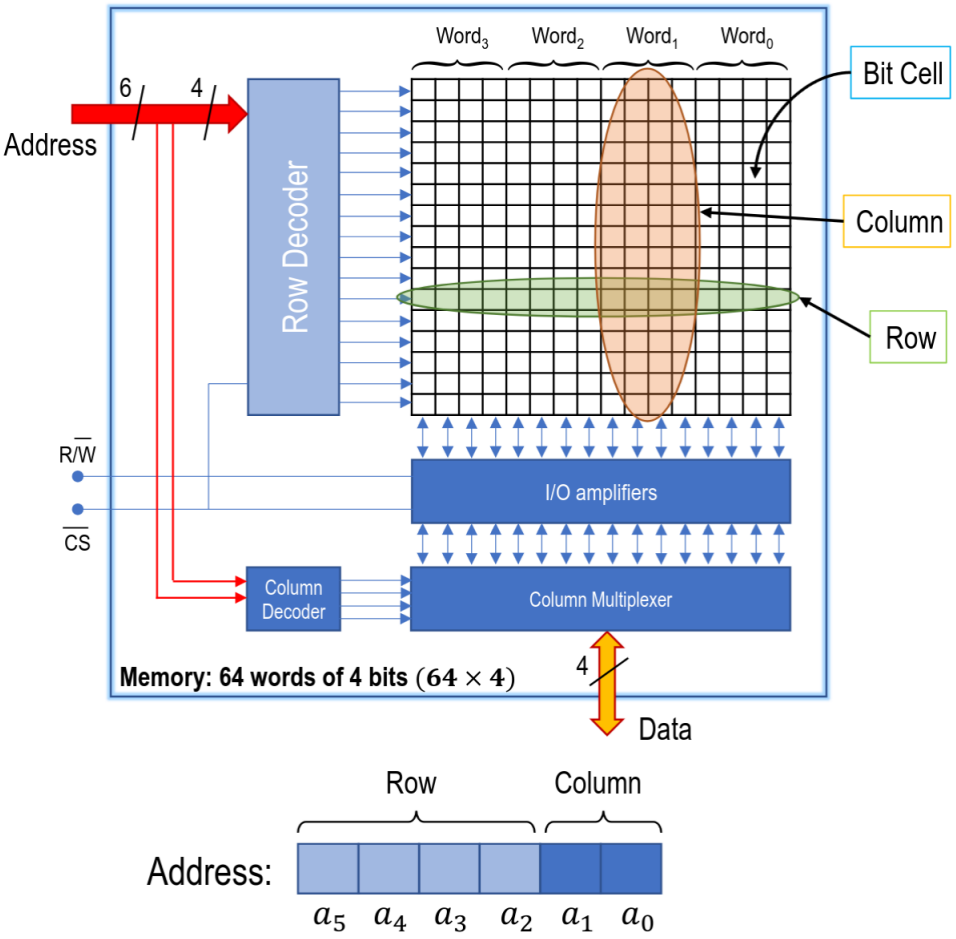
This is not the ideal approach though, as:

- The aspect ratio is unusual and difficult to work with;
- The size of address decoder is exponential on the number of address bits;

- The memory is *slow*: large parasitic capacitance on bit lines increases propagation delays.

5.0.4 Physical structure of digital memories

We then move on to defining the **physical** structure of a real memory. The idea is to not have a *linear*, but a **square** array (or matrix) of word cells.



So we find that the real structure is still an array of bit cells, organized in a square array (optimal aspect ratio). This means that we can have more than one word for each row: one bit slice for each bit in the word, and one column for each word.

Our address decoder has to become a *row decoder*, and we need an additional *column multiplexer* after the I/O amplifiers to select the correct word from a row. As we can see, this is achieved by splitting the address:

- The *upper* side of the address is used to select a row;
- Within a row, the *lower* side of the address is used to select a column.

The sense amplifier and I/O circuit are still used to interface to the external world.

From the user interface, the interface is the same, but the internal organization is different (and way easier to implement physically).

5.1 Memory interfaces

We can now talk about the **interfaces** of memories. Usually these are device pins used to access the memory content, write/read it, ... They are grouped based on their function:

- **Address** bus: identifies the memory location (the index of the word) where an operation is performed. A straightforward address bus is comprised of $k = \log_2(N_{\text{words}})$ lines;
- **Data** bus: provides the data to/from the memory word. Can be bidirectional, or split in input/output monodirectional buses. A straightforward data bus is comprised of n_{bits} lines;
- **Control** bus: this contains the control lines used to perform operations on the data stored in memory. A lot of control lines can be defined, for instance:
  - Clock;
  - Reset;
  - Chip select;
  - Output enable;
  - Write enable;
  - Read/write select;
  - ...

Memory interfaces (as the name suggests) are designed for interfacing a memory to a digital circuit. The digital circuit shall connect to all pins of the memory, drive the inputs according to the specifications in the datasheet, and read the outputs. All of this must be done while complying with the timing diagrams.

- In a *custom* digital circuit, the interface is usually designed specifically for the given memory;
- In a *standard* digital circuit, there is a generic memory interface: it may require external components to correctly drive a certain memory.

5.1.1 Memory operations

This can be a good moment to note that the memory **operations** available are:

- Read: given an address, returns the word contained in memory at that address;
- Write: Given a word and an address, stores the word in memory at that address;
- Match: Given a word, searches the memory for that particular word content and returns the address. This is specific of Content Addressable Memories (**CAM**, *Content Addressable Memory*, used in caches, for instance);
- Erase: initializes the memory to a pre-defined content. Typical of some memories;
- Other operations are sometimes available, such as: Burst operation, Partial operation, Precharge, Refresh, ...

5.1.2 Memory protocol

Data access via the address, data and control lines is performed by an external component on the memory via a predetermined **protocol**. he protocol allows organizing the memory to have better performance and area size:

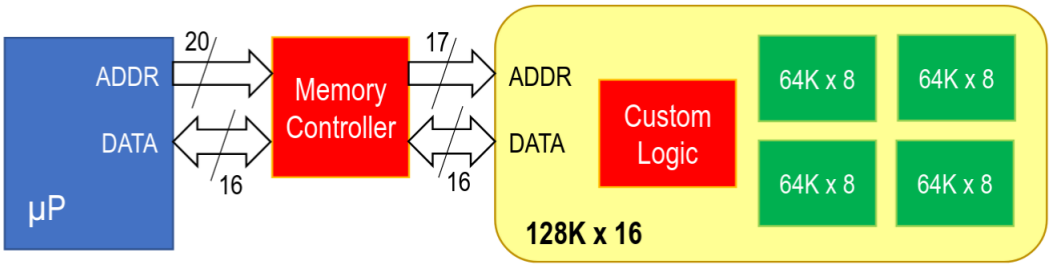
- It defines the sequences of signal activations to correctly perform different operations;

- It defines the timing constraints.

  A memory protocol is often described as a timing diagram in the device datasheet.

5.1.3 Memory composition

One of the most important concepts regarding memories at the logic level is memory **composition**. This entails combining multiple memory chips of a given size s_i to obtain a single addressable memory of size $\sum_i s_i$.



We have 3 main cases:

1. Let's assume the case where the required memory needs more words than those available in a single memory chip. For compatibility, we also assume that word width is the same.

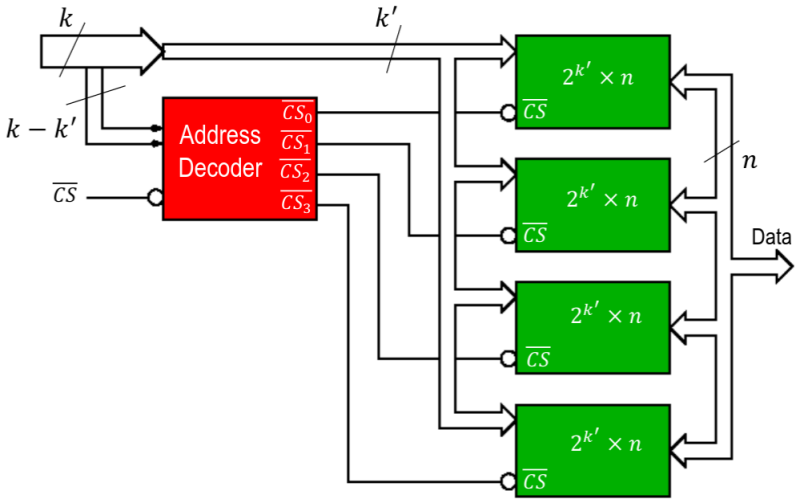
   This means that addresses are size k, with each memory chip having addresses of size k' and:

   $$k > k'$$

   The number of memory chips needed is given by:

   $$n_{\text{chips}} = \frac{2^k}{2^{k'}} = 2^{k-k'}$$

   We want only one of these chips to be active (to *respond*) for a given address range: this means implementing custom logc to drive the $\overline{CS}$ (*active low Chip Select*) signal of the memory chips. This custom logic is basically a decoder, controlled by the $k - k'$ additional address lines (over k') and enabling $2^{k-k'}$ (which have to alternatively drive one singular data output bus, a.k.a. tristate outputs).



2. Now there's the case where the required memory needs words with more bits than those available on a single memory chip. For compatibility, we also assume that N_{word} is the same.

   This means that words are to be size n, with each memory chip having words of size n' and:

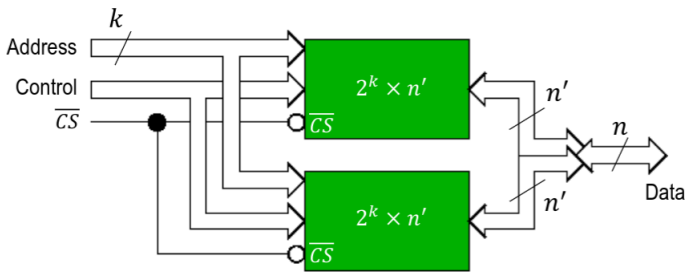
   $$n > n'$$

   The number of memory chips needed is given by:

   $$n_{\text{chips}} = \text{ceil}\left(\frac{n}{n'}\right)$$

   At this point the composition is trivial:

   - All memory chips share the same address,;
   - All memory chips share the same $\overline{CS}$ chip select signal;
   - Their data busses are combined (concatenated) to form a larger data bus.



3. The last case is the combination of the first two: required memory needs more words than those available in a single memory chip and needs more bits in a word than those available in a single memory chip.

   This means that there are: • k required address lines, but k address lines in a memory chip, with $k > k'$; • n required data lines, but n' data lines in a memory chip, with $n > n'$.

   We need more than one memory chip (of course), and exactly:

   $$n_{\text{chips}} = \text{ceil}\left(\frac{n}{n'}\right) \cdot 2^{k-k'}$$

5.1.4 Memory controllers

Lastly, let's spend a note on **memory controllers**. These can be useful, for instance, when general purpose processors are interfaced with off-the-shelf memory chips: the amount of address lines might not match!

Now in this situation, without a memory controller, the microprocessor accesses the same location in memory with multiple addresses. Also, some pins are left floating.

A *memory controller* is used to translate addresses and activate the memory. The memory is mapped to a certain address range in the microprocessor's addressable space: the range does not necessarily start at address 0: the most significant bits can be used by the memory controller to select the appropriate memory. Though, it's good to say that the address range is usually aligned to a power of 2 to simplify the logic.

We know this far too well from our experience with *compiler* tooling: the compiler for the microprocessor should know the chosen address range, so that it generates code that correctly accesses the memory.

Lastly, we note that the memory controller may be integrated inside the microprocessor.

6 Lesson 02-10-26

Last lesson we looked into *memories* and *memory interfacing*.

6.0.1 Addenda on memory interfacing

We introduced **masking** for memories that live within a processor's addressing space, but don't occupy it fully: these just need to consider the top k bits and output an active low enable signal.

An extra case we didn't show is that of memory combinations with different word lengths. Let's say our processor takes 16 bit words, that is it has a 16 bit data bus, and we try to populate it with a single 8 bit wordlength memory.

This means we need a more complex memory controller:

- The Data Bus is not compatible;
- We need multiple transactions.

6.0.2 Addenda on memory parameters

Let's name a few more memory parameters.

- We talked about **size**: this is expressed in bits, bytes or multiples of these. Requirements of this are based on the application specifications:
  - Strong impact on costs;
  - We try to reduce memory requirements;
  - Small Embedded Systems use only some Kbytes of memory.
- **Speed** can be defined in several different ways. For instance:
  - **Asynchronous reads**: from when the address given is stable to when the provided value on the data bus (by the memory chip) is stable;
  - **Asynchronous writes**: from when the address given is stable to when the provided value on the data bus (by the CPU) is guaranteed to be stored.

  Throughput is the opposite measure, defined as the maximum number of operations per second.

  For **synchronous** memories we count the time measured in clock cycles given the clock frequency.
- **Area** is a good example of a physically interesting metric: the actual size of the bit cell.

  The total size is given by the total number of bits multiplied by the size of a single bit cell. Over that, we need to account for the overhead of decoders, multiplexers and I/O circuits.

  Note that different kind of memories have different bit cells:
  - Different bit cells provide different features;

  - Very strong impact on the total area;
  - Very strong impact on the total cost.

  In fact, memories (especially DRAMs) are at the cutting edge of technology: they are the first components to adopt smaller gate lengths;
- **Cost** is related to the size of the bit cell, the size of the memory (in number of bits), and external factors like production volume.
- Another important metric is **power consumption**: this depends on the bit cell structure and technology, and is very important for portable systems;
- **Write ability**: this refers to the manner and speed a memory can be written.
  - *High end* writeability: processor writes to memory simply and quickly. In-system, unlimited cycles;
  - *Middle range* writeability: processor writes to memory, but slower than the high end. In-system, thousands or tens of thousands of cycles;
  - *Lower range* writeability: special equipment to be used to program the memory (external programmer), thousands of cycles permitted;
  - *Low end* writeability: once at fabrication.
- **Storage permanence**, also known as its opposite, *volatility*: it's the ability of the memory to hold stored bits after they are written.
  - **High End** storage permanence: bits are never lost, even without power. Lifetime is that of the device;
  - **Middle range** storage permanence: bits are held for years, even without power. Sometimes more than the lifetime of the embedded system;
  - **Lower range** storage permanence: as long as power is supplied to the memory;
  - **Low End** storage permanence: memory content is lost after a certain amount of time, even if power is supplied. Often after a few tens of millisecond.

6.1 Read Only Memories

We finally come around to talking about **ROM**s (*Read Only Memories*).

Let's first consider *Mask Programmable Read Only Memories*: these are written at fabrication time, so they have:

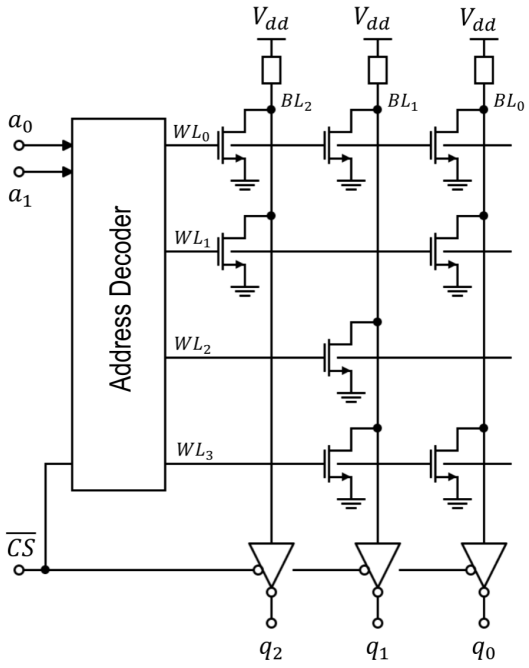
- Lowest write ability (only once)
- Highest storage permanence (for ever)

To summarize, they can only be read and not written to.

The *costs* are high NRE (we need to develop specific masks for memory contents, though the same masks may be shared for memories of the same size and organization), low per-unit, so good for very large productions.

6.1.1 NOR memory

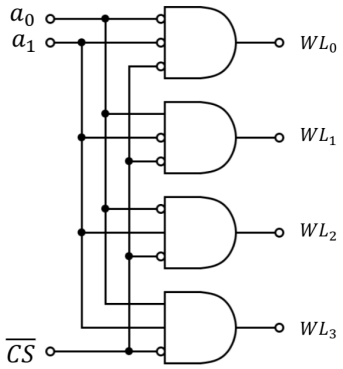
A common implementation of ROM memory is the **NOR** structure.



All bit lines are normally pulled-up to V_{dd}: the output is logic 0 because of the inverter. A single word line is active (high) at any given time, and transistors in the corresponding word are conducting. Transistors in all other words are in cut-off.

A conducting transistor in the active word lowers a bit line to 0, so the presence of a transistor at a given bit cell becomes our "one", the absence our "zero". Transistors of course are defined at the mask level: *Diffusions, Thin Oxide, Polysilicon, Metal*.

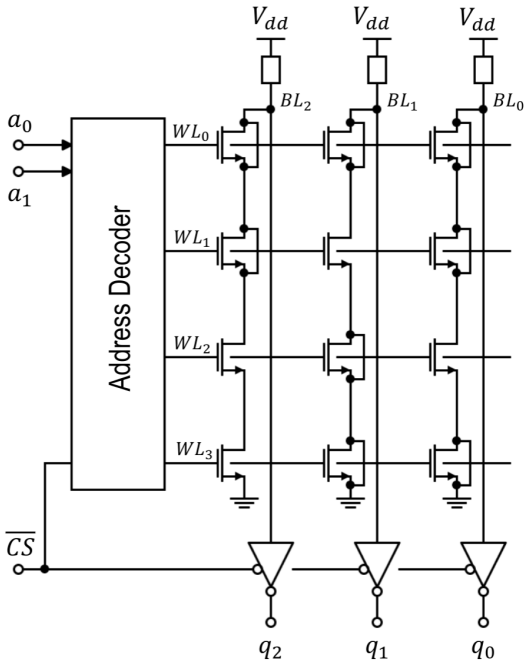
Lastly, let's look at the (simple) topology of the address decoder.



7 Lesson 07-10-26

7.0.1 NAND memory

In the last lesson we saw *NOR memories*. Now we look at the **NAND** structure.



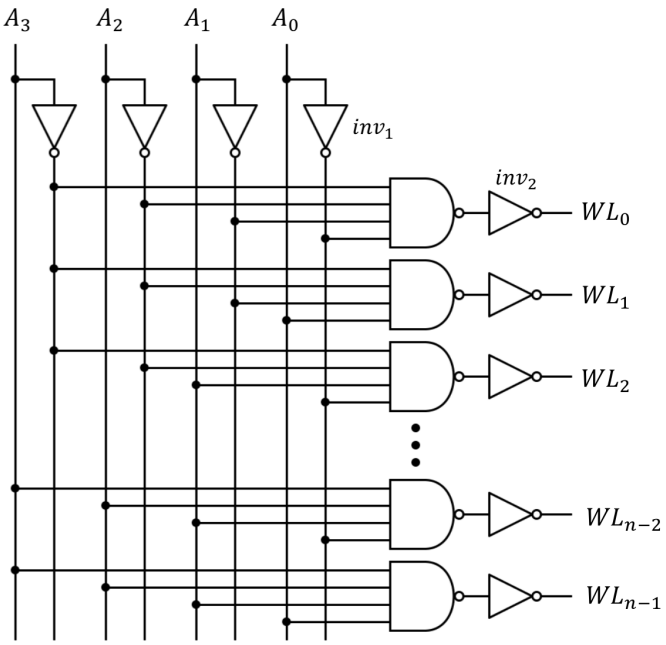
All bit lines are normally pulled-up to V_{dd}: the output is still logic 0 because of the inverter.

Now, we knot that if all transistors are at gate low, we don't connect them to ground: it follows that the bit line stays high and the output low (logic 0 at default).

The control signals for the word lines (WL_i) are active low. This means that every word line is at 1 but the selected one (which is at 0).

Now:

- If there is no short circuit between the drain and source of the selected transistor, we don't get a short to ground, and the line stays high: so the output stays low;
- If there is a short circuit, on the other hand, we short to ground, meaning the output stays high: we encode a logic 1 by shorting a transistor.



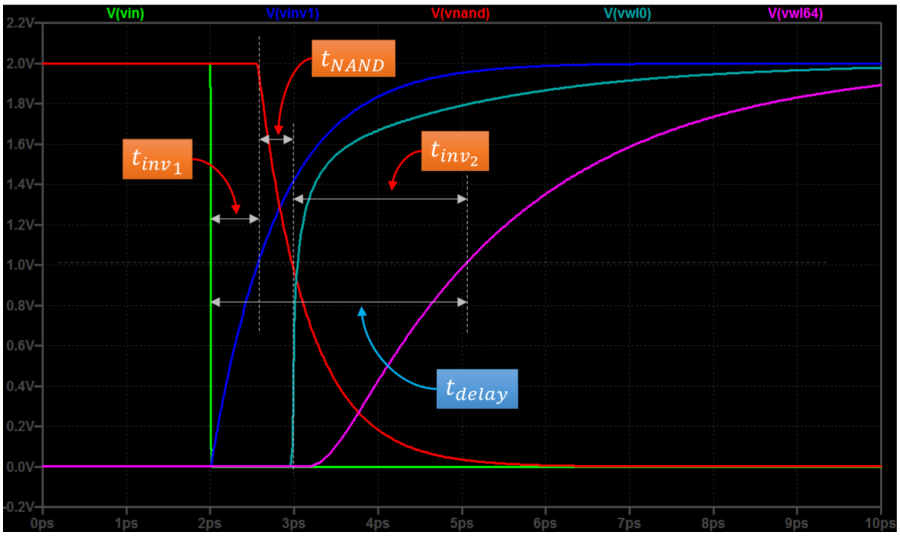
Clearly, the mask topology of the NAND topology is more complicated. We have the delay of the address decoder, given by:

1. Delay of the address lines inverters , which are driving half of the NAND gates, and have large parasitic capacitance.
2. Delay of the NAND gates, with large parasitic resistance when all pull-down transistors are ON;
3. Lastly there's the delay of the Word Line inverters, driving all transistors in the bit cells in a row, with large parasitic capacitance and resistance as the word Line is long.

This tradeoff comes at the gain of higher *densities* overall, meaning NAND memories are usually less performant but smaller.

7.0.2 Memory timings

Let's look at the simulated delay, specifically of the address decoder.



We see the 3 delay source cited above (t_{inv_1}, t_{NAND}, and t_{inv_2}). The total delay is:

$$t_{delay} = t_{inv_1} + t_{NAND} + t_{inv_2}$$

Note that this is the example with 64 bit cells in a row: the delay for the first bit cell is much lower than the delay of the last bit cell We must always consider the worst case!

We must also look at the performance of NOR and NAND technology on bit line discharging. The bit line has a certain parasitic capacitance C_L. When the memory is idle, the bit line is pulled-up to V_{dd}, and C_L charges up to it. When a bit is read, we have to discharge C_L down to 0.

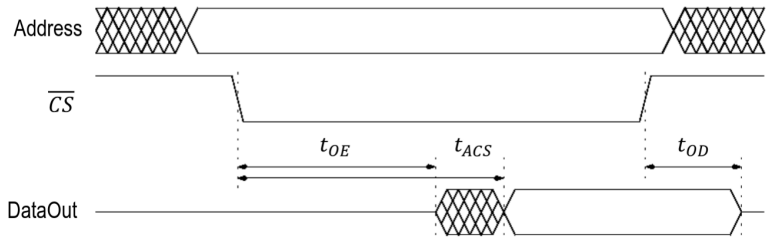
- In a **NOR** structure, the parasitic capacitance dominates. The resistance of the bit line is small (metal), and all capacitances are in parallel. This means that:

  $$\tau = R_{DSon} \cdot C_L = R_{DSon} \cdot C_{drain} \cdot n_{tr}$$

  where n_{tr} is the number of transistors;
- In NAND topology the parasitic resistance dominates: it is the channel resistance of all transistors in the series. The structure can be studied using the *Elmore* delay, and we get that:

  $$\tau = \frac{RC}{2}, \quad R = n_{tr} \cdot R_{DSon}, \quad C = n_{tr} \cdot C_{drain}$$

Latly, we show a picture of the general timing parameters for a memory access (a read, as we're talking EPROM memories) operation.

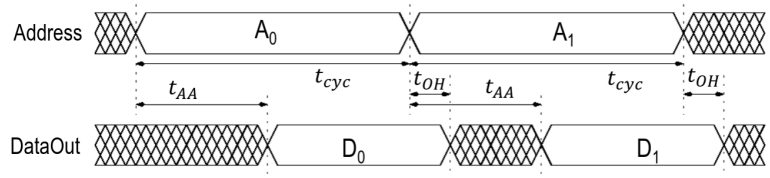


From when address is stable and $\overline{CS}$ (active low) is low, we note the following delays:

- t_{OE} is the **minimum Output Enable** time, which is the time before the output takes some value, but is not yet stable. After t_{OE} the value on DataOut might be wrong, but DataOut is not in high impedance anymore;
- t_{ACS} is the **maximum Chip Select Access** time, after which the DataOut will be stable;
- t_{OD} is the **maximum Output Disable** time, after which the output returns to high impedance.

We then see that the time we have to wait before having a result from memory is in between t_{OE} and t_{ACS}.

Let's see what happens when changing the address.



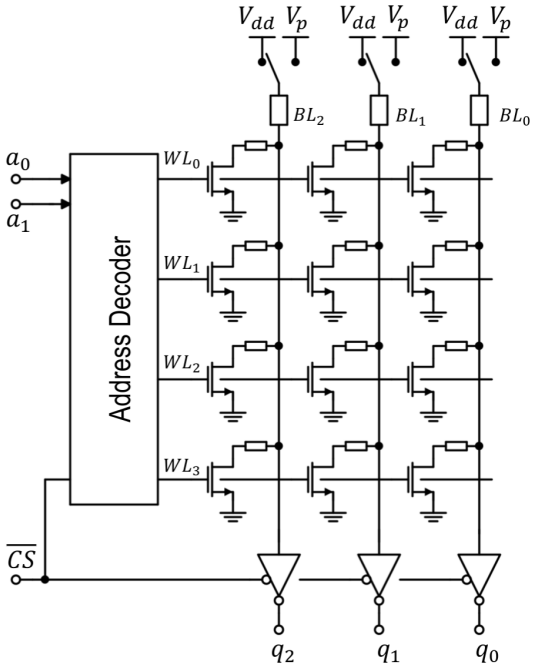
From when the address is changed, we note the following delays:

- t_{AA} is the **maximum Address Access** time, after which DataOut takes the value pointed to by that address;
- t_{OH} is the **minimum Output Hold** time, after which the DataOut goes back to being undetermined after address changes;
- t_{cyc} is the **cycle** time, for which addresses should at least be stable.

We then see that, as long as we keep addresses stable for at least t_{cyc}, the data will be available at most after t_{AA}, and at least for t_{OH} after the address change.

7.1 One Time Programmable Read Only Memories

We now look at **OTP-ROM** (*One Time Programmable Read Only Memories*). These are based on **fuses** or *anti-fuses* (which blow to become shorts), which get blown in order to encode data. Of course this process is irreversible and means writing operations are permanent.



As in the standard NOR structure, all bit cells have a transistor, with the drain of each transistor is connected to the bit line through a fuse.

All fuses are short circuit after fabrication, meaning all transistors are connected to the bit lines: the initial content of the memory is all 1s.

Blowing a fuse turns a logic 1 into a logic 0, with he corresponding transistor being disconnected from the bit line.

As we said, the fuse cannot be restored, the operation is one time and we cannot turn a logic 0 back to a logic 1.

To correctly program an OTPROM the procedure is to:

- Enable the word line of the bit we want to program;
- Select the bit line of the bit we want to progam;
- Drive the bit line at $V_p >> V_{dd}$, high enough to blow the fuse.

We call the mode in which V_p is feed to the memory *programming mode*. In programming mode the data bus is an input which selects the bit lines to be connected to V_p. V_p itself is provided through a dedicated memory pin.

7.1.1 Performance of OTPROMs

First off, **writing** an OTPROM takes considerably (orders of magnitude) higher time than reading it. This is fine, as writing an OTPROM is as we said an one-time only operation.

As for other characteristics, we see that a **read** operation is mostly identical in performance to Mask Programmable ROMs. The disadvantages are higher resistances of the fuse compared to a short circuit, which get a bit slower compared to a Mask Programmable ROM.

OTPROMs are also *untestable*: they need to be produced in small size to have a good yield.

The number of defective ICs in a wafer does not depend on the IC size, but smaller ICs of course have less chance to be detective. This means that usually there are more small ICs in a wafer than large ICs. Note that the yield y is roughly given by:

$$y = \frac{n_{ICok}}{n_{ICtot}} = 1 - \frac{n_{ICdefective}}{n_{ICtot}}$$

7.2 Programmable Read Only Memories

OTPROMs are suboptimal as they can be written *only once*. Because of this, more advanced solutions which allow memory to be rewritten (that is, *reprogrammed*), a number of times. We call these **PROM**s (*Programmable Read Only Memories*).

7.2.1 Floating-gate transistors

The core element of a PROM is a *floating-gate* transistor, that is a MOS transistor which can have its gate biased by applying a high voltage to it. One example of such a transistor is the **FAMOS** (*Floating gate Avalanche MOS*), which is actually not famous at all.

They are similar to a MOS transistor, with an dditional polysilicon layer between two thin $SiO2$ layers below the gate. This means the gate is *floating*. Charges trapped in the floating gate change the threshold voltage V_T, where

- Negative charges increase the threshold;
- Positive charges decrease the threshold.

Programming

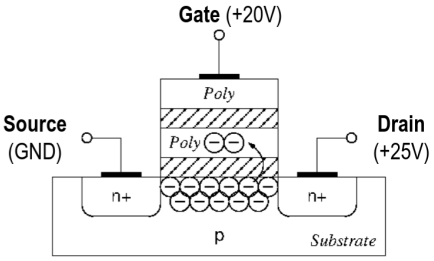
The floating gate is completely isolated, and to program it we use *hot electron programming*:

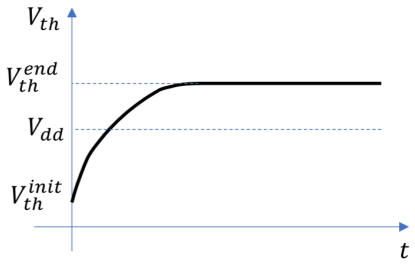
- By applying a high programming voltage V_p to the gate, a rich channel forms (as the gate is pulled high). Note that this alone is not enough to lower V_T!
- By applying a high programming voltage V_p' to the drain-source channel, electrons in the channel are subject to strong electrical fields. High speed electrons travel through the channel, which might interact with the silicon substrate. Due to strong vertical electrical field from the gate, the high energy electrons may overcome the SiO_2 barrier, reaching the floating gate. The result is that the gate is pulled low (from the added electrons) and V_T increases.

Performance

When programming a FAMOS transistor, as the threshold voltage V_T increases, the channel is less rich and the vertical electrical field is less strong. This clearly means that the number of electrons with enough energy to jump in the floating gate decreases (less electrical field), and process is *self regulating* (!) The nice property of this system is that the final threshold voltage is quite consistent among all transistors, regardless of the length of the process.

We then have that V_T grows above V_{dd} by a certain margin. By playing with doping levels, densities, geometries, we can change that margin.





The *disadvantages* of FAMOS transistors are:

- The process takes time and requires high voltages
- How to generate those voltages in the first place: provided externally using pins? Generated internally from the standard voltage supply?
- Damages to the lattice: we cannot apply programming too many times. 1000 – 10000 program ing cycles seems to be the sweet spot in most commercial applications;
- Leakage current removes electrons from the floating gate: the information is lost over time, on a timescale of sveral tens of years.

Erasing

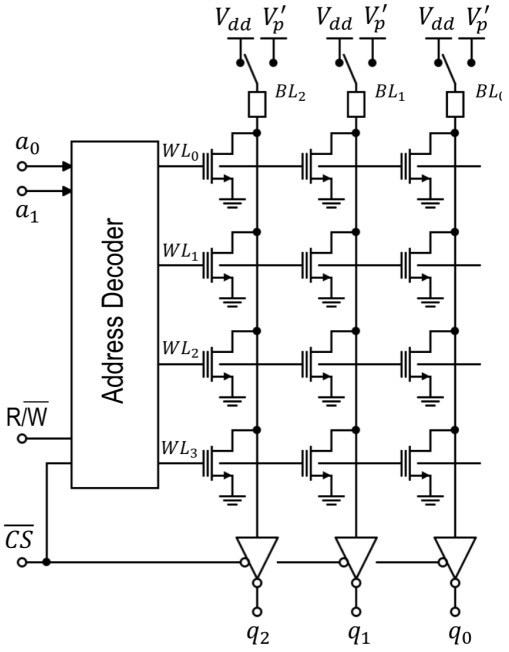
Erasing FAMOS transistors is the most interesting part. Electrons cannot be removed electrically from the floating gate: energy has to provided to the electrons via an *UV lamp*.

As high energy photons hit electrons, the electrons jump back to the substrate and diffuse. After a sufficiently long amount of time (minutes) all electrons are removed, and the floating gate is left unbiased.

For this reason we need a window on the integrated circuit (which we'll tape off when the circuit is at work to avoid accidental exposure to UV light). We also cannot target a *single* transistor: all transistors are erased at the same time. This makes PROM erasure an usually out of the circuit operation, as with programming, done with external hardware in a lab.

7.2.2 Programmable ROM

Let's look at a NOR structure **PROM**.



The funky transistor symbol with the triple gate line represents a FAMOS transistor (with the middle line being the floating gate).

All bit cells have a FAMOS transistor. The drain of each transistor is connected to the bit line. All FAMOS transistors are not programmed after fabrication, so they behave normally. The initial content of the memory, as a result, is all 1s, and programming a transistor (raising the V_T higher, disconnecting it) turns a logic 1 into a logic 0.

As we said, the memory can be reset to the initial state by shining UV rays through a window. All programmed transistors are erased, and all non programmed transistors are not affected: the memory contents return to all 1s.

To correctly program a PROM we proceed similarly to OTPROMs:

- Enable the word line of the bit we want to program;
- Select the bit line of the bit we want to program;
- The selected word line is driven to the voltage $V_p > V_{dd}$;
- The selected bit lines (possibly more than one for the same word) are driven to the voltage $V_p' > V_{dd}$.

To avoid impacting other components, other rows are at ground, and bits not to be programmed on this row have their drains at V_{dd}.

7.2.3 Performance of PROMs

For the read operation, PROMs are identical to Mask Programmable ROM and OTPROMs.

For the *write* operation, that is *programming*, moving negative charges to the floating gate makes the transistor a permanent open circuit:

- Turns a 1 into a 0 (if the bit line is inverted);

- Turns a 0 into a 1 (if the bit line is affirmed).

It works at the word level too.

For erasing, we emove the negative charges from the floating gate. This restores the normal transistor behavior, but applies to the entire memory at once.

This means PROMs have almost completely superseded OTPROMs:

- Much more flexible;
- Cheaper during prototyping;
- Can be reused;
- Still, not suitable for data that is frequently changed.

7.2.4 FLOTOX transistors

Another variant of floating-gate transistor is the **FLOTOX** (*FLOating gate Tunneling OXide transistor*). Similarly to FAMOS, it has a floating gate.

The FLOTOX differs from the FAMOS in its erasure properties: it is *electrically erasable*.

This is achieved through the *tunneling effect*, specifically **Fowler–Nordheim tunneling**. This tunneling effect is a quantum mechanical phenomenon, where a particle (electron) can pass a potential barrier even if it has insufficient energy, if subject to a strong enough electric field. This is the case of electrons to and from the gate, as they have to cross the oxide (SiO_2) layer.

In a FLOTOX thinner oxide layer on the drain (source) side makes tunneling easier. Electrons in a FLOTOX NMOS (PMOS) come from the drain (source) region rather than from the substrate or channel. An higher electrical field than normal on the gate is necessary for this.

Programming

The floating gate is completely isolated, and to program it we use *Fowler–Nordheim tunneling*:

- By applying a high programming voltage V_p to the control gate, a strong electric field is established across the thin tunnel oxide;
- The strong electric field allows electrons to tunnel through the thin SiO_2 barrier from the substrate onto the floating gate;
- The floating gate accumulates negative charge. As a result, the threshold voltage increases:

$$Q_{FG} < 0 \quad \Rightarrow \quad V_T \uparrow .$$

To erase the cell, the polarity of the applied voltage is reversed:

- A high voltage of opposite polarity is applied to the control gate, producing a strong electric field across the tunnel oxide.
- Electrons tunnel from the floating gate through the thin SiO_2 layer into the substrate.
- The floating gate loses its negative charge, causing the threshold voltage to decrease:

$$Q_{FG} \to 0 \quad \Rightarrow \quad V_T \downarrow .$$

In both of these operations the source is left floating or grounded at $0\,\mathrm{V}$. This means that the only dipole involved is the gate-drain one, and to summarize:

- $V_{GD} >> 0$: electrons jump from drain to gate, *increasing* V_{TH};

- $V_{GD} << 0$: electrons jump from gate to drain, *decreasing* V_{TH}.

Performance

Let's make a comparison of programming mechanisms:

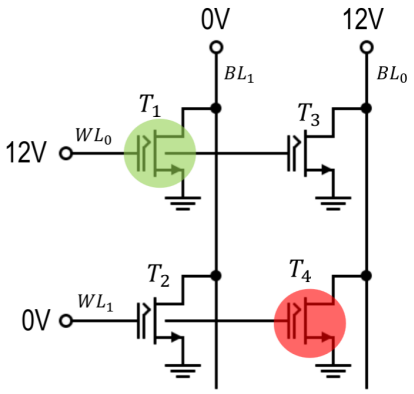
- Hot Electron Programming is faster than Tunneling, which may increase performance, but performance may be limited by other issues.
- Tunneling causes less damages to the lattice than Hot Electron Programming, leading to higher life-time of the device, and many more programming cycles;
- Hot Electron Programming is Self Regulated, Tunneling is not;
- Tunneling requires careful timing to achieve good consistency: the resulting distribution of V_T after programming has a larger standard deviation, and requires larger margins during operations;
- FLOTOX transistors are larger and harder to fabricate: lower yield, higher costs.

Addenda on programming

There is a famous issue with FLOTOX programming. The straightforward implementation (but it doesn't work!) is:

- The drain of each transistor is connected to the bit line;
- The source of each transistor is grounded.

This has a problem: programming a transistor erases another one.



- To program T_1:

$$\text{Gate: } WL_0 = 12\,V, \qquad \text{Drain: } BL_1 = 0\,V$$

- To avoid programming T_2:

$$WL_1 = 0\,V$$

- To avoid programming T_3:

$$BL_0 = 12\,V, \qquad V_{GD} = 0$$

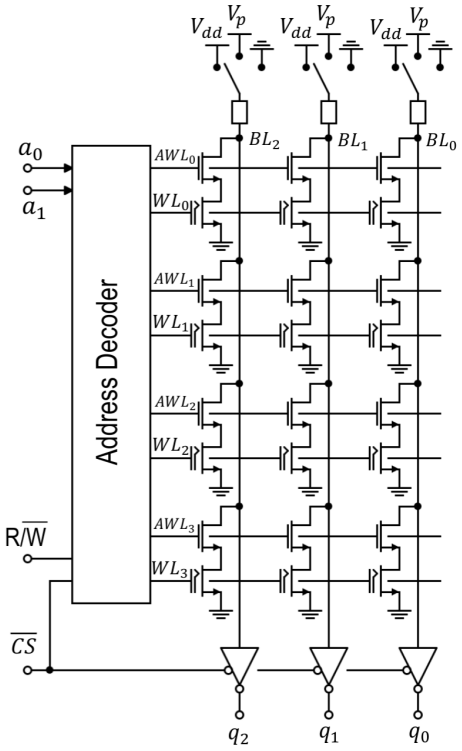
- However, T_4 is erased:

$$WL_1 = 0\,V, \qquad BL_0 = 12\,V$$

The solution is to use an access transistor for each bit cell.

7.2.5 Electrically Erasable Programmable ROM

We use FLOTOX transistors to realize **EEPROM**s (*Electrically Erasable Programmable ROMs*).



All bit cells have a FLOTOX transistor, with thin oxide on the drain side.

All bit cells also have an access transistor; drain connected to the bit bine, source connected to the FLOTOX, gate driven by the address decoder (via dedicated AWL_i lines).

Access transistors are activated when the word is selected.

Out of the factory, all transistors behave normally, and the initial content of the memory is all 1s.

- Programming a transistor turns a logic 1 into a logic 0;
- Erasing a transistor turns a logic 0 back into a logic 1.

This happens *electrically* and *in circuit*.

Let's look at how the main operations are carried out:

- For **programming**, we select the word by raising the word line, and the data in the word by raising the D line (which as always acts as an input for programming).

  We then enter the *programming* mode by enabling AWL on the word line, that is enabling the access transistors. We keep these OFF on other lines to avoid affecting other FLOTOX transistors.

  At this point, to program we raise the word line (that is the gate) to V_p, and the selected bit lines (that is the drains) to $0\,\mathrm{V}$. The rest of the lines are kept at V_p;
- For **erasing**, we select the word by raising the word line, and the data (to erase) in the word by raising the D line.

We then enter the *erasing* mode by enabling AWL on the word line, that is enabling the access transistors. We keep these OFF on other lines to avoid affecting other FLOTOX transistors.

At this point, to erase we lower the word line (that is the gate) to $0\,\mathrm{V}$, and the selected bit lines (that is the drains) to V_p. The rest of the lines are kept at $0\,\mathrm{V}$.

7.2.6 Performance of EEPROMs

The read operation is identical to Mask Programmable ROM, OTPROM and EPROMs (which we called PROMs so far). The eternal interface is to:

- Provide programming voltages;
- Signal to select programming;
- Signal to select erasing.

External interface often simplified by providing internal programming and erasing mechanisms.

They are widely used, and much more flexible than EPROMs, as they can be erased by a micro-processor in circuit. On the other hand, they are expensive, and the bit cell is larger than in an EPROM.

Writeability is very good as it is:

- In circut;
- Repeatable, thousands of times.

We also have *erasure* in circuit, with single bit access both in writing and erasing.

The storage permanence is very high, and the NREs are kept low (though they are higher than EPROMs).

8 Lesson 12-10-26

In the last lesson we introduced programmable memories, covering:

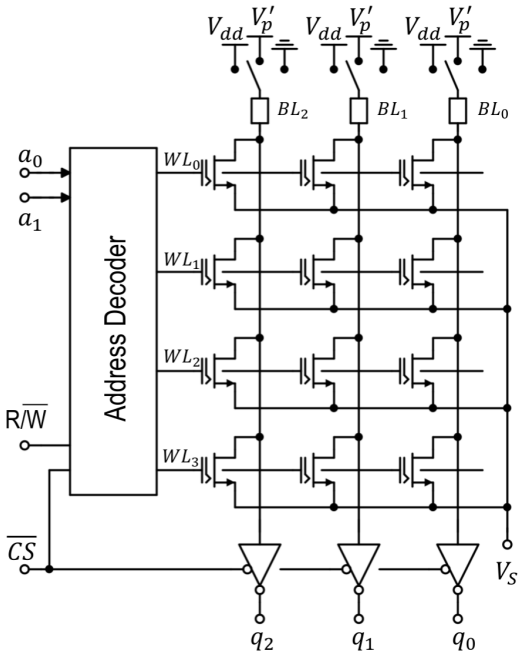
- **MPROM**s (*Mask Programmable ROMs*), programmable only once by altering the design at production time;
- **OTP-ROM**s (*One Time Programmable ROMs*), based on fixed MOSFET paths programmed only once by *blowing fuses*;
- **EPROM**s (*Erasable Programmable ROMs*), programmable electrically, and erasable only via photoelectric effects (shining a light on them). These were based on **FAMOS** (*Floating gate Avalanche MOS*) transistors;
- **EEPROM**s (*Electrically Erasable Programmable ROMs*), programmable and erasable electrically. These were based on **FLOTOX** (*FLoating gate Tunneling OXide MOS*) transistors.

8.1 Flash memory

We now cover **flash** memory storage, which seeks to lower the cost and latency problem of common EEPROM memory, while still providing a completely electric read/write interface.

8.1.1 NOR Flash

NOR Flash memory is based on FLOTOX transistors with thin layers on the source side of the channel.



All Sources are tied together and not connected to ground. All FLOTOX transistors are not programmed after fabrication, so that they behave normally and the initial content of the memory is all 1s.

NOR Flash transistors are programmed and erased in different ways:

- **Hot Electron Programming** is used for *programming*, to avoid affecting other transistors;
- **Tunneling effect** is used for erasing.

Let's see in detail:

- To **program** the memory, the word is selected via its address and data is provided on the data bus.

  The select word line (targeting the gates) is driven at programming voltage V_p, and all other word lines are kept at ground.

  The selected bit lines (targeting the drains) are also driven at programming voltage V_p', and all other bit lines are kept at ground.

  Lastly, sources are connected to ground with $V_S = 0\,\mathrm{V}$.

  The high energy electrons which end up traveling through the channel are thus able to be injected in the gate, raising the gate threshold V_{TH} and encoding a logic 0;
- To perform a **full erase** of the memory, all word lines and bit lines are grounded.

  The source is then driven to high programming voltage.

  Since the thin oxide layer is on the source side, the result is that we experience electron tunneling from gate to source, and a logic 1 is encoded in every transistor (so each memory location).

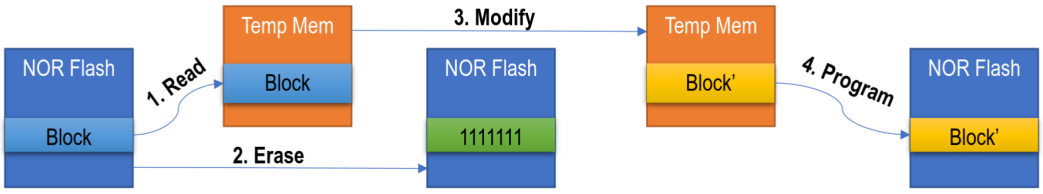
- A **block erase** is also possible by splitting the V_S line among different groups of transistors (at least as big as a single word). Usually block size ranges from a few Kbytes to hundreds of Kbytes.

  Sources of different blocks can be independently driven: the erase is therefore applied to only one block at a time.

8.1.2 NOR Flash peformance

In regards to performance, we have that the **read** operation is identical to Mask Programmable ROM, OTP-ROM, EPROM and EEPROM. The performance in **writing**, other hand, sacrifices single bit access while erasing (present on EEPROMs). When writing, an entire block is erased and rewritten, even when writing a single word.

This calls for more complex memory writing mechanisms, which need temporary buffers to store the blocks being modified. One such mechanism is detailing in the figure.

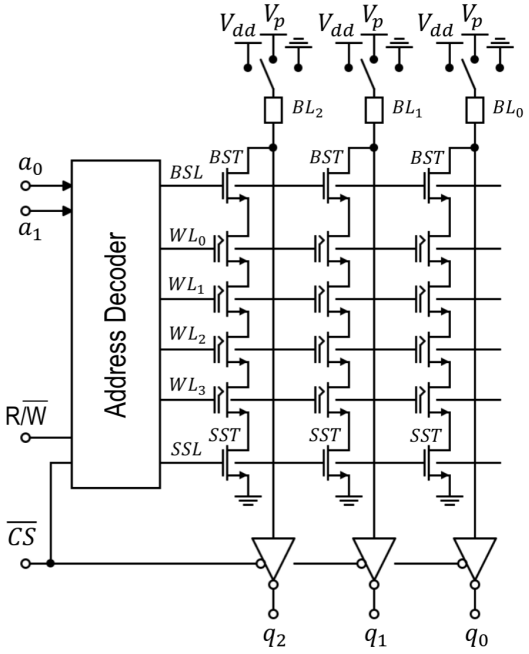


Lastly, we note that for NOR Flash memory a **CFI** (*Common Flash memory Interface*) exists, and is agreed upon and used by constructors.

- Reading is performed as in a ROM;
- Writing is based on commands: a standard set exists, and software drivers are usually used to access these kinds of memories.

8.1.3 NAND Flash

NAND Flash is the NAND version of NOR flash. It still uses FLOTOX transistors, but with thin oxide on the drain side. The difference is that tunneling is used for both *reading* and *writing* cells.



As with MPROMs (see lesson 7), NAND topology is slower (higher parasitic capacitances and resistances), but denser (lower costs and size).

To enable correct usage, access transistors are placed at the top and bottom of a series:

- At the top (drain side) we have a **BST** (*Bit Line Select Transistor*);
- At the bottom (source side) we have a **SST** (*Source Select Transistor*).

All FLOTOX transistors are not programmed after fabrication, so the initial content of the memory are all 1s.

In NAND Flash, all gate voltages are complemented compared to NOR topology. Voltage values used in NAND Flash memories are:

- V_{th} is the nominal threshold voltage of a non programmed transistor;
- V_{thprog} is the threshold voltage of a programmed transistor;
- $V_{pass} > V_{thprog}$ is a gate voltage such that all transistors are conducting, regardless of their programming state;
- V_{read} is a gate voltage such that only non programmed transistors are conducting;
- $V_{off} < V_{th}$ is a gate voltage such that all transistors are not conducting.

We move on to operations.

- First off, **reading** is non trivial. We first set all bit lines to V_{dd} (so that the pull-up transistors actually pull up).

  Then we set the selected word line to V_{read}, and all others to V_{pass}. This means that all transistors, except in the word to be read, are conducting.

  At this point:

  – If the transistor in the word is not programmed, it is conducting: the Bit Line is grounded, and the output is logic 1;
  – If the transistor in the word is programmed, it is not conducting: the series is interrupted, the Bit Line remains high, and the output is logic 0.

Note that reading a NAND flash memory is slow, as the NAND topology has much higher delays than the NOR topology. For this reason the memory is divided in **blocks**: each block is tens or hundreds of Kbytes. Each block is divided in **pages**, where ach page is tens or hundreds of bytes. A page is a row, or a fraction of a row, of the bit cell array.

Most NAND Flash Memories are read *page by page*, storing in a temporary SRAM buffers. Words are read sequentially within a page, and no random access to individual words is permitted. Because of this, NAND Flash Memory is not a drop-in replacement of a ROM: it cannot be used to store and execute code directly.

- **Programming** a transistor iw performed by setting the selected word line to V_p, and all the other word lines to V_{pass}.

  Then, the desired bit line is set to ground, and all the other bit lines to V_p.

  Lasly, the BSTs are enabled and the SSTs disabled, meaning the sources are floating (Which is the programming condition for FLOTOX transistors via electron tunneling).

  With this method we do program the selected transistors, and other transistors aren't affected:

  – Transistors in other words (row) have a small V_{GD};
  – Other transistors in the same word have $V_{GD} = 0\,\text{V}$

- **Erasing** is done in blocks.

  BST and SST access transistors are deactivated, so that all channels are left floating.

  Then all word lines are grounded.

  The erasure tunneling happens directly via the *substrate*, which is set to progamming voltage V_p.

  This means we erase an entire *block*, or the entire memory. For this reason, blocks are in isolated substrate wells. All blocks can be erased concurrently, affecting the entire memory.

8.1.4 NAND Flash performance

We already said that NAND Flash provides very low read efficency, and as such is more suited to whole page caching, with serial access protocols.

The serial interface is similar to **SPI**, with up to 4 parallel bits (non-SPI mode).

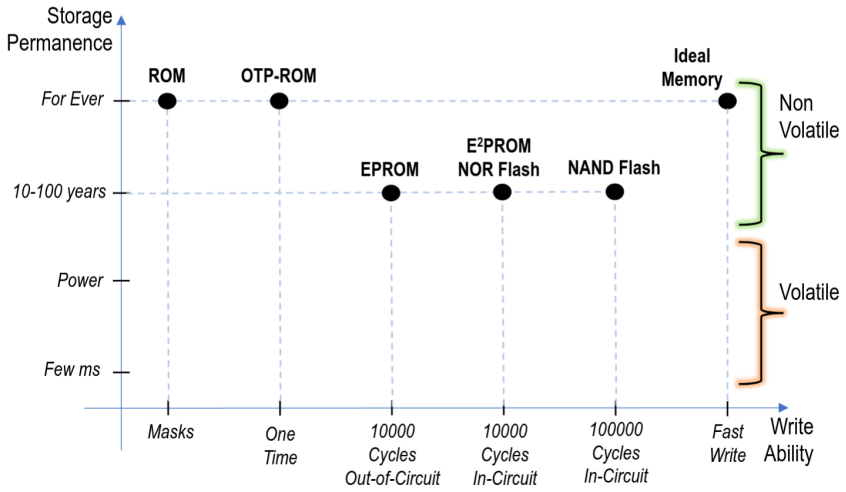
To summarize, to increase performance a page is first transferred to an internal cache (made up of SRAM). Reads and writes always occur from/to the cache, with the internal memory controller governing read/writes to and from the actual NAND memory cells.

As with NOR Flash, the protocol defines a number of commands.

Lastly, we note that we need to read – modify – rewrite, like in the NOR Flash Memory case. It's easier to write the updated block to an empty block, marking the initial one for erasing.

8.2 Summary of ROM memories

We show graph of Write Ability and Storage Permanence of ROM memories (see section 6.0.2).



We see that:

- All ROM memories have a large penalty in writing, and as such are used mostly to read data;
- Flash memories are faster (than for instance EEPROMs), but erasing and writing not at word level;
- All ROM memories are *non volatile*, but this is sometimes not necessary. In these cases we can sacrifice storage permanence by write as fast as reading, using word and bit levels (instead of programming voltages). With this method we can use standard access methods, with no need of programming and erasing.

8.3 Read Only Memories

We now move on to **RAM**s (*Read Only Memories*). RAM denotes memories that are fast to write, and random access. ROMs are also random access, but RAMs are much faster than E2PROMs and Flash memories (several order of magnitude faster especially in writing).

RAMs usually allow single bit and single word access in reading and writing, with no distinction between programming and erasing: write is identical in the two cases ($0 \rightarrow 1$ and $1 \rightarrow 0$).

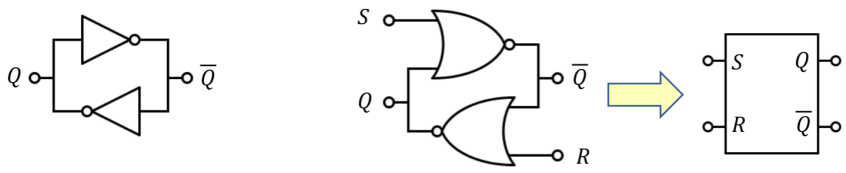
There are two main kind of RAMs:

- *Static* RAM (**SRAM**, **CAM** or *Content Addressable Memory*);
- *Dynamic* RAM (**DRAM**, **SDRAM**, **DDR**).

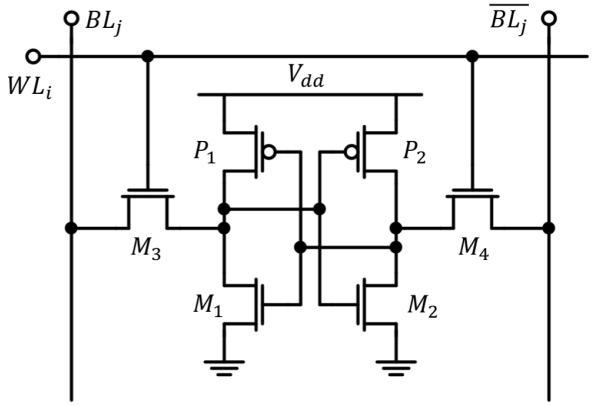
8.3.1 Static RAM

Static RAM relies on *bistable* circuits, that is the usual sequential data latches.

As we saw in 3.2, a NOT loop is a basic example of a bistable circuit, with an unstable center state.



To create an SRAM cell from an SR latch, we need a special topology.

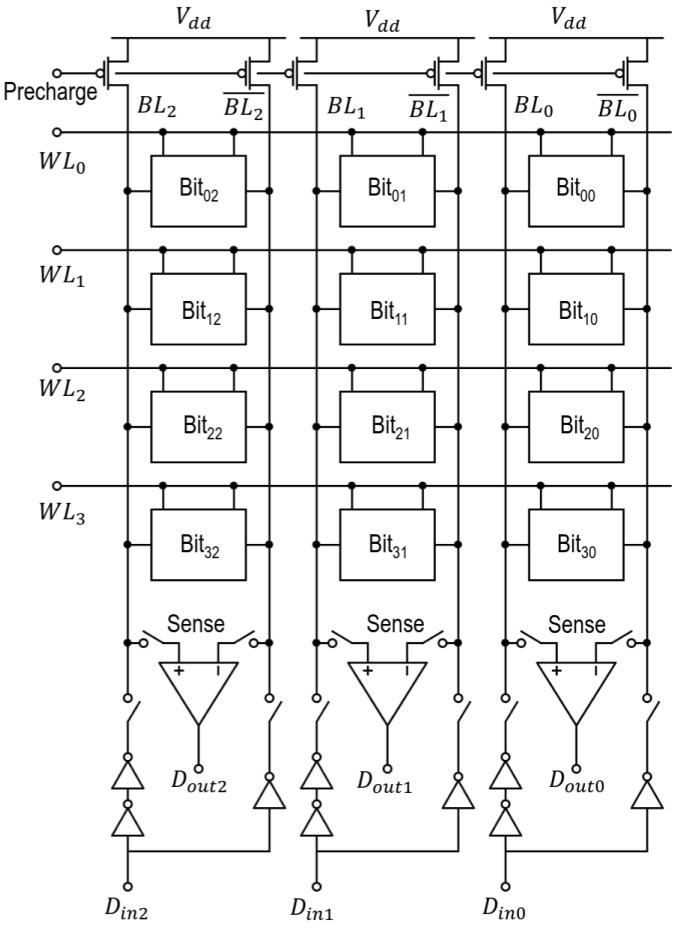


This features:

- The M_1, M_2, P_1 and P_2 transistors to realize the basic bistable cell;
- The *access* transistors M_3 and M_4, configured as pass transistors. These allow programming of the bit.

We note the two Bit Lines for each bit cell: one affirmed and one complemented. A single Word Line, on the other hand, drives the gates of both the access transistors, enabling reads/writes to the bistable cell.

Let's now look at the organization of static RAM cells to form a full SRAM:



We see that a number of components have to be studied:

- The single *bit cell* is the same as we already saw. Note that a single pair of affirmed and complemented bit lines is shared across multiple cells;
- The *precharge* circuit. This is used to precharge the bitlines to a level which won't interact with the values stored in the bit cells on reads;
- The *sense amplifier*: this is used to sense values in the bit lines and amplify them to full logic levels. All of this machinery is really needed because sharing multiple bit lines across bit cells multiplies the resistances and capacitance experienced by these;
- Lastly we need a *column decoder* and *mux* for allow I/O data lines at the bit lines.

Let's now go through *operations*.

- **Reading**. This happens in a few steps.
  1. First we charge both the bitlines to V_{dd}. Then we detach the precharge source. This means that the V_{dd} voltage is mantained through wire capacitance on the bit lines (usually in the order of picofarads);
  2. After the bitlines are left floating, the M_3 and M_4 passgates are opened. This means that one side (the high side) of the bit lines will stay high, and the other will go low. By sensing the voltage difference across them, we can recover the data which was initially stored in the bit line.

  Note that in this operation M_3 starts conducting current, thus raising the voltage at M_2's gate. To avoid bit flips, M_3 should be sized smaller than M_1 (to have a higher R_{DSon} resistance and take more voltage in the voltage divider that is formed).
- **Writing**. This is arguably simpler.
  1. We first load the bit lines (affirmed and complemented) to respectively the affirmation and the complement of the value we want to store. This process is similar to precharging, with the difference that the voltage applied to the bit lines is kept until wire capacitance charge, and throughout the whole writing process (so the bit cells are actively driven);
  2. To actively drive the bit cells we then just raise the word line, so that the bitline values propagate to the bit cell.

  Note that for this operation to work we need to be able to force a bit flip, and thus the pass gate transistors M_3 and M_4 should be larger than the PMOS transistors P_1 and P_2 (so that their side in the voltage divider is larger).

8.3.2 SRAM performance

SRAM performance is great in both reading and writing.

- For the *read operation* we said that the low side nMOS transistors in the bistable circuit should be larger than the corresponding nMOS access transistors. This means that the ON resistance is lower for the low side nMOS transistors.

  We then have that the sense amplifier detects very small changes of the input voltages, with ΔV in the order of a few hundred millivolts.

  As such, the performance of reading is very fast, ranging from a few nanoseconds to tens of nanoseconds;
- For the *write operation* we said that the nMOS access transistors should be larger than the high side pMOS transistors in the bistable circuit. Sometimes the pMOS are im-

plemented using resistances, or pMOS that are always conducting, designed to have a high R_{DSon}. The exact ratio of all transistors depends on technological parameters: transistors are not minimum size.

The performance for writing is very fast, ranging from a few nanoseconds to tens of nanoseconds, which is comparable to the read operation.

There are also *no limits* in the number of write operations (far different from most ROMs we saw), and *no distinction* between writing 0 or 1.

8.3.3 SRAM variants

A few variations on SRAM are available.

- **NVRAM** (*Non Volatile RAM*): a battery keeps power for several years. It is more expensive, larger in physical size, and heavier;
- **Multiport SRAM**: features independent ports with separate busses, independent access transistors and row and column circuits.

  This allows multiple users to read concurrently at multiple location (even the same one). They can also read a location while writing another location.

  Note that, when reading a location while writing the same location, the result of reading is not defined.

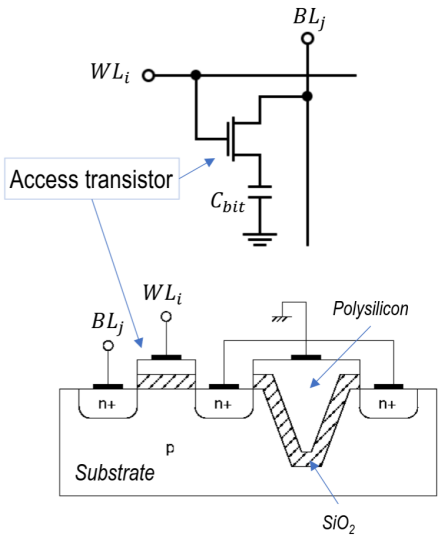
  Users can also write multiple different locations concurrently, but still note that writing the same location from different ports yields an undefined stored value.

8.4 Dynamic RAM

Our goal with dynamic RAM is to design a much smaller bit cell, to make a fast and cheap memory.

The idea is to store information as charge on a capacitor: a capacitor is a form of analog memory where if charge is present, the logic value is 1; if charge is not present, the logic value is 0.

Lastly, note that the capacity used in DRAM is actually the parasitic capacitance C_{bit} of a special transistor. A second *access* transistor allows to charge or observe the storage capacity C_{bit}.



Looking at the layout cross-section, the nMOS access transistor is a standard one. Te capacity transistor, however, is a so-called **trench capacitor**, comprised of an nMOS transistor with Source and Drain connected together. These are in turn onnected to the Source of the access transistor. The polysilicon Gate goes inside the substrate, increasing the area A of the gate. The gate is connected to ground, so that the equivalent capacitance is:

$$C_{bit} = C_{ox} \cdot A$$

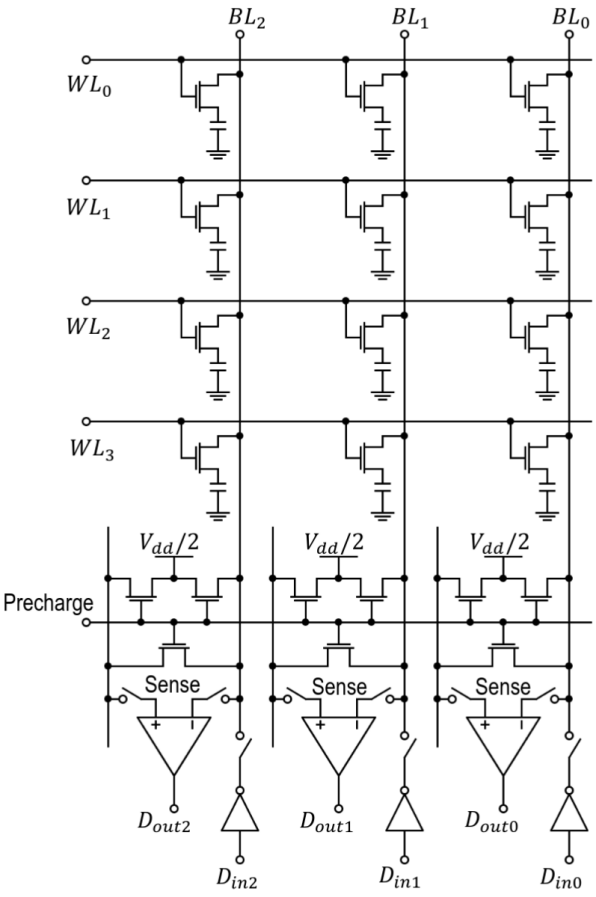
To improve the analog memory, C_{bit} should be the highest possible. Technology is defined to increase the parasitic capacitance: it cannot be embedded on the same integrated circuit as standard logic.

The DRAM bit cell is small, much smaller than SRAM. *Stacked capacitors* then allow for smaller footprint in planar technology. This allows DRAM memory to be far cheaper than SRAM.

Of course, there are problems associated with DRAM:

- Leakage currents destroy the information after some time (milliseconds), so we need to periodically refresh the data stored;
- Reading the charge on the capacitor requires to remove some charges, which need to be restored after reading to not lose data.

Let us now look at the usual DRAM topology.



We see:

- The usual matrix of bit cells, organized in word lines and bit lines. The capacity is easy to connect in this configuration as the word line just drives the access transistor gate, and the capacity drives the bit line (through the access transistor's source);
- We the have the *precharge* circuit to connect bit lines to $\frac{V_{dd}}{2}$. This is a nice unbiased voltage we can use to extract useful ΔV from the capacities;
- We then have the *sense amplifier*. It is a differential amplifier to detect small voltage differences. To have a ΔV difference in the first place, we need an additional comparison line. This is usually the $\frac{V_{dd}}{2}$ line from before;
- As with SRAM, we have the *I/O* section with inputs and outputs, switches driven by column decoder and mux.

Let's see the *operations*:

- For **reading**, we start with all wordlines inactive, and precharge the bitlines to $\frac{V_{dd}}{2}$.
  We then activate the selected wordline. Charge is thus shared between the C_{bit} capacitances and C_L, which is the cunulative capacitance of a bit line.
  When we then activate the sense amplifiers and select the desired word, the sense amplifiers detects the small change, positive or negative. This gets us the final read result.
  Note that the final voltage on the storage capacitor is close to V_{ref} (our $\frac{V_{dd}}{2}$): information is lost (we say that reading is *destructive*). To restore it we need to write after reading. This is performed internally by the memory, so it is not necessary to issue a write command. On the other hand, rewriting is necessary also in other words of the same row (even if not read).
  We can do some calculations to evaluate the charge transfer between C_{bit} and C_L in both the reading 1 and reading 0 state:
  - For the 1 state, we start by computing the total charge on the C_{bit} capacity and the C_L capacity before raising the word line. Assuming that after raising the word lines these are equalized, we have to find the resulting voltage. In symbols:

$$\begin{cases} Q'_{tot} &= C_{bit}(V_{dd} - V_{th}) + C_L V_{ref} \\ Q''_{tot} &= (C_{bit} + C_L)V_X \end{cases} \implies V_X = \frac{C_{bit}(V_{dd} - V_{th}) + C_L V_{ref}}{C_{bit} + C_L}$$

    At this point we compute the final ΔV as the difference between the resulting V_X and the V_{ref} the sensing amplifiers compare against:

$$\Delta V = V_X - V_{ref} = \frac{C_{bit}}{C_{bit} + C_L}(V_{dd} - V_{th} + V_{ref})$$

    Lastly, note that $V_{ref} \approx \frac{V_{dd}}{2}$ as per previous discussions, so we can say;

$$\Delta V = \frac{C_{bit}}{C_{bit} + C_L}\left(\frac{V_{dd}}{2} - V_{th}\right)$$

  - For the 0 state, we do the same thing in reverse. Assume the C_{bit} capacity starts low, with zero charge:

$$\begin{cases} Q'_{tot} &= 0 + C_L V_{ref} \\ Q''_{tot} &= V_X(C_{bit} + C_L) \end{cases} \implies V_X = \frac{C_L}{C_{bit} + C_L} V_{ref}$$

We compute the final ΔV the same as before:

$$\Delta V = -\frac{C_{bit}}{C_{bit} + C_L} V_{ref} = -\frac{C_{bit}}{C_{bit} + C_L} \frac{V_{dd}}{2}$$

using the same value of V_{ref} as before.

What we wanted to show is that $\Delta V_1 \approx -\Delta V_2$, so that the sensing amplifiers see basically the same voltage delta in both reading cases.

At this point we just have to note that at the end of the read operation the voltage on the storage capacitor is V_X, which is very close to V_{ref}. The original logic value is lost, and in fact all bits in the same row are lost, even if they are not part of the word that is being read. Still, the information is known at the output of the sense amplifiers! This is true for all columns, even those that were not selected: the logic value can thus be written back by driving the Bit Line with an appropriate voltage (we'll see write shortly), and the original logic value is restored on C_{bit}. This also applies to all bits in the row, as the value is taken from the sense amplifier of the corresponding column.

- For **writing**, we also start with all wordlines inactive.

  We then apply the desired to the bit lines, and precharge the bit lines of all other words to V_{ref} (as to not interfere with their values once we raise the word line).

  After this setup we raise the word line, and storage capacitors are charged or discharged to the wanted values.

  Note that this seems fast, but has the same destructive effect on bits we are not interested in as reading: we require a read (through the sense amplifiers) and a write back. This means that the performance is the same as reading.

A solution for DRAM memory is the *Comparison Line*. The sense amplifier is placed in the middle of a Bit Line: the other half of the Bit Line used for comparison.

When *precharging*, both halves of the Bit Line are precharged to V_{ref}. On word selection, only a single Word Line is active: either the upper or lower half is affected. The other half keeps the precharged value.

When *sensing*, the sense amplifier compares the two halves and gives the result. Writing back is easy as one side remembers the old value.

Of course this requires that, when *writing*, both halves are driven with the input data signal.

8.4.1 DRAM performance

We said that DRAM is slower than SRAM (by an order of magnitude) both in writing and in reading (in fact, it exhibits the same basic timing characteristics). This is because of the writeback phase required by reading and writing, both destructive operations.

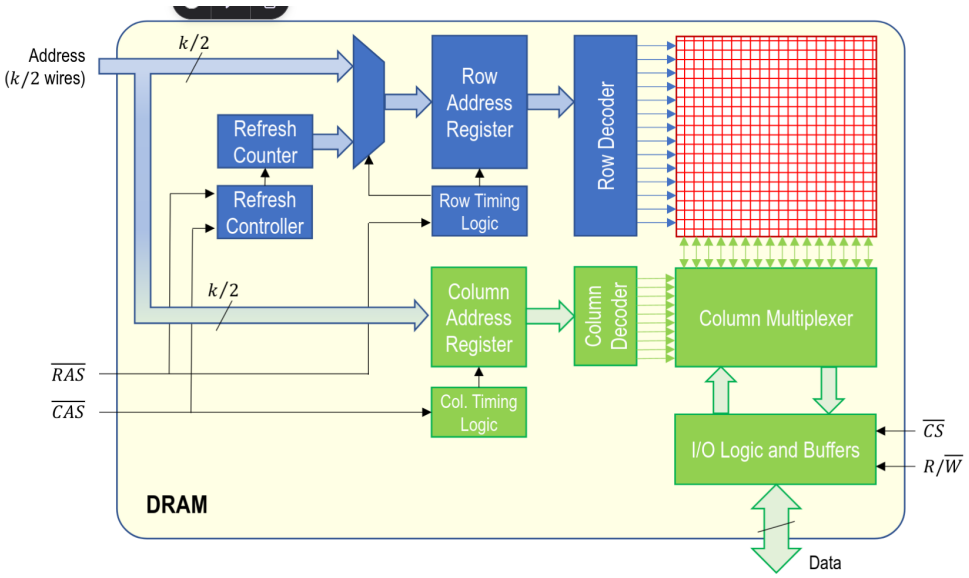
Still, DRAM has excellent write ability and very fast word and bit level writing. As with SRAM, it can be written an infinite number of times.

An important factor is the *very low storage permanence*: data is lost after a few milliseconds, and has to be **refreshed** periodically.

Assuming a DRAM has N rows and the charge on C_{bit} remains valid for at least a time T_{store}, then a new row should be refreshed every $T_{\text{refresh}} = \frac{T_{\text{store}}}{N}$.

8.4.2 DRAM interface

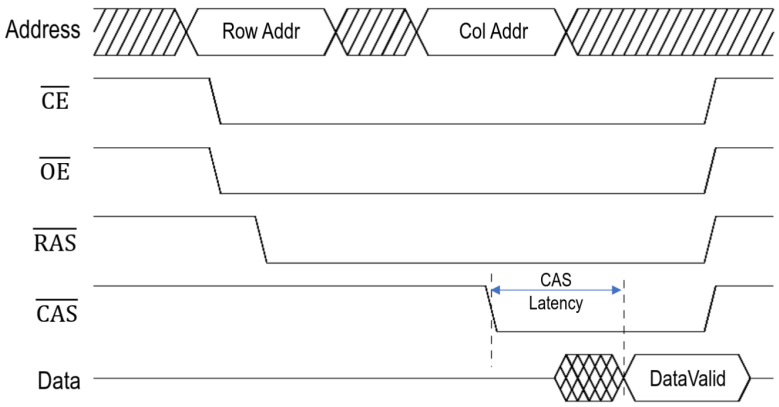
Let's look at the interface we use DRAMs with in a bit more detail.



For the overall structure we see that we multiplex two halves of the address in time, using row and column registers. This requires the additional control signals **RAS** (*Row Address Strobe*) and **CAS** (*Column Address Strobe*), usually active low. *Strobe* signals are signals used to notify the memory when an address is ready (column or row).

As for *refreshes* (see the previous paragraph), we have that when a row is opened, all data is refreshed. Special circuits for automatic refresh are needed if refresh is automatic. Refresh can also issued by a microprocessor (CAS before RAS). Usually, when refreshes are issued by the microprocessor (or other external circuits), at least one read per each wordline has to be performed manually in a certain time period (the $T_{refresh}$ we calculated in the previous section). Of course, as we saw, there is an *overhead* due to refresh.

Let us now look at the timings for a simple read operations, to get an idea of the typical timing of the interface.



- **Read operation** ($\overline{\text{CE}} = 0$, $\overline{\text{WE}} = 1$):
  - Present the Row Address (most significant bits) at the address pins and activate $\overline{\text{RAS}}$ (low) to latch it internally;
  - Present the Column Address (least significant bits) at the address pins and activate $\overline{\text{CAS}}$ (low) to latch it internally;
  - After a minimum delay, the output data is valid and stable.

8.4.3 DRAM variations

DRAM is relatively slow because reading data requires precharging the bit lines, selecting (opening) a row, and waiting for the sense amplifiers to detect the resulting voltage changes. Because of this, some variations exist.

- Since the row address is sufficient to open a row (or page), **Fast Page** Mode exploits the fact that all words in the same row are sensed, even those that are not explicitly read, and must also be rewritten. Once a row is open, other words within the same row can be accessed by specifying only the column address; these words need not be contiguous. To read another word in Fast Page Mode, a normal read operation is performed while keeping RAS active. CAS is then deactivated (set high), a new column address is presented at the address pins, and CAS is reactivated (set low) to latch the new column address internally. After a minimum delay, the new output data becomes valid and stable;
- *Extended Data Out* (**EDO**) Mode is similar to Fast Page Mode, but with an additional register added at the output of the memory. The Column Address can change while keeping the data of the previous operation on the DataOut Bus. This is not possible in Fast Page Mode, and the additional register implements a sort of small pipeline, increasing performance (we can change the column address while reading a word);
- *Synchronous DRAM* (**SDRAM**): all operations synchronized on the active edge of a clock. This simplifies internal structure compared to asynchronous memories. Simple circuits improve performance. These RAMs allow for contiguous addresses, column sent only once. This is useful using caches;
- *Double Data Rate* (**DDR**) is a variation on SDRAM. For contiguous data, the output is updated on both clock edges (rising and falling). For faster operation, we can use multiple clock edges, from multiple clocks out of phase.

  There are different DDR DRAM variations:

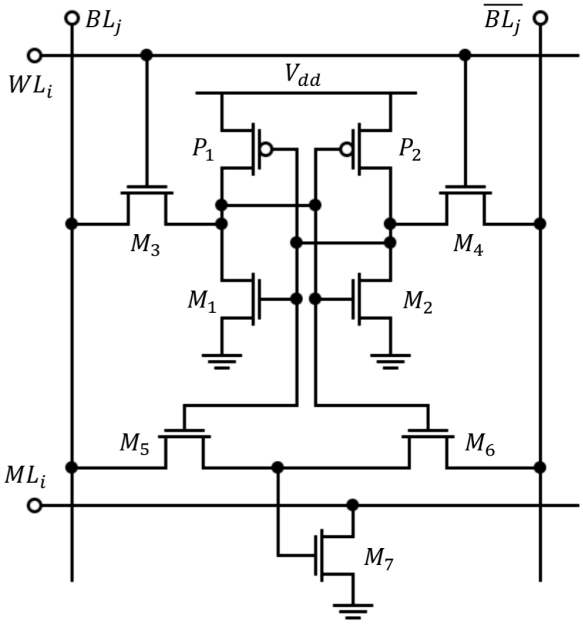
  - Normal applications: DDR, DDR2, DDR3, DDR4, ...
  - Low Power, for mobile applications: LPDDR1, LPDDR2, ...
  - High bandwidth, for graphic applications: GDDR2, GDDR3, GDDR4, GDDR5, ...
  - Special architectures such as RAMBus and eDRAM.

9 Lesson 13-10-26

9.1 Content addressable memories

CAMs (*Content Addressable Memories*), also known as *associative memories*, are special kinds of memory where the main operation is a **match operation**. Instead of providing the address and read the data, want to look for the *address* of some given data. This can be done in software, via indices or simple linear scans, but is far more efficient and faster (if not more expensive) to do in memory.

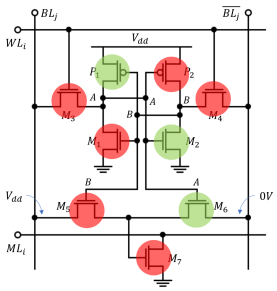
Let's look at the cell.



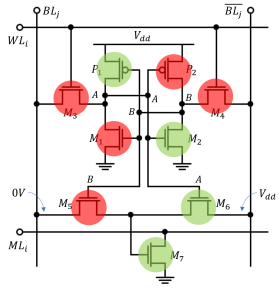
For the match operation, the word Line WL_i remains inactive for the entire operation: we don't want to change the bit cell content. Instead we recharge Match Line ML_i to V_{dd}, then apply the value to BL_i and its complement to $\overline{BL_i}$. If there is no match, ML_i will be discharged.

There are 4 main cases to study here:

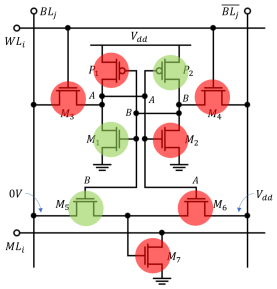
1. Contaning 1, matching 1:



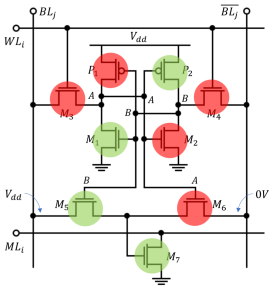
2. Contaning 1, matching 0:



3. Contaning 0, matching 0:

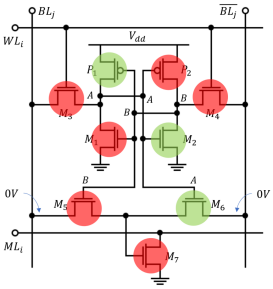


4. Contaning 0, matching 1:

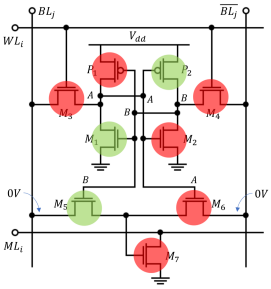


We then have other 2 cases, which relate to the *don't care* search state. We can perform a search by not caring if we set both bit lines to 0.

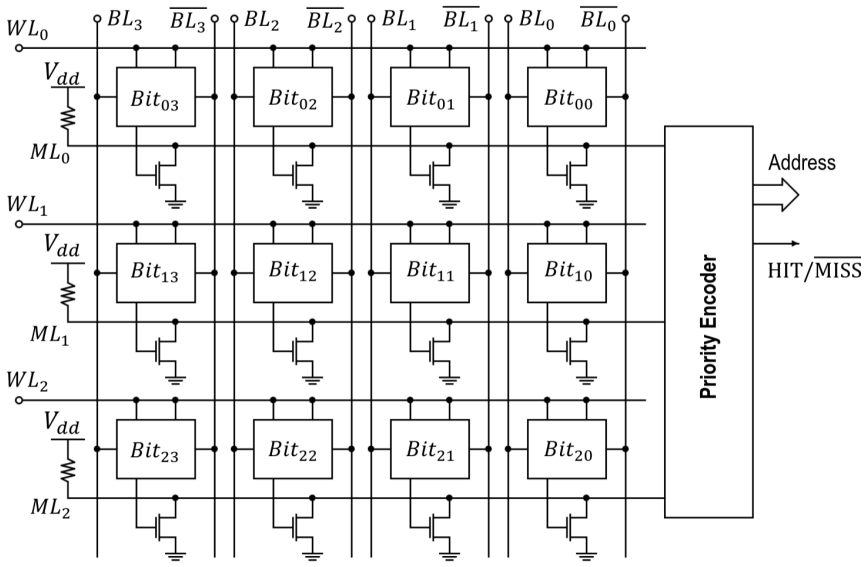
1. Contaning 1, don't care:



2. Contaning 0, don't care:



At this point we have documented the behavoir of the CAM cell enough to look at the actual topology:



Note that:

- Each Match Line spans an entire word, but not an entire row! This means that if all bits in Word i match, the Match Line ML_i is high. If on the other hand at least one bit in Word i does not match, the Match Line ML_i is low;
- Then note the *priority encoder*. If at least one Match Line is high, the priority encoder asserts HIT, and it generates the corresponding address (the highest). On the other hand, if all Match Lines are low, the priority encoder asserts MISS.

9.1.1 Associative SRAM

Let's look into how an actual **associative SRAM** is built from this premise.

In the simplest case an associative SRAM is only a circuit with the topology shown in the last section. When such an Associative SRAM is switched ON the content is random.

Let's assume some data is written, but only partially occupies the memory: we want to match only against the written data.

To do this, we have all words in the associative memory contain an additional bit, called the *valid bit*. At power on also the valid bit is also random, so a special associative memory operation initializes all valid bits to 0. This means marking all words as invalid

When writing a word, the corresponding valid bit is automatically set. At this point, when matching, all valid bits are automatically matched against logic 1, whereas all invalid words do not match.

9.1.2 Associative SRAM performance

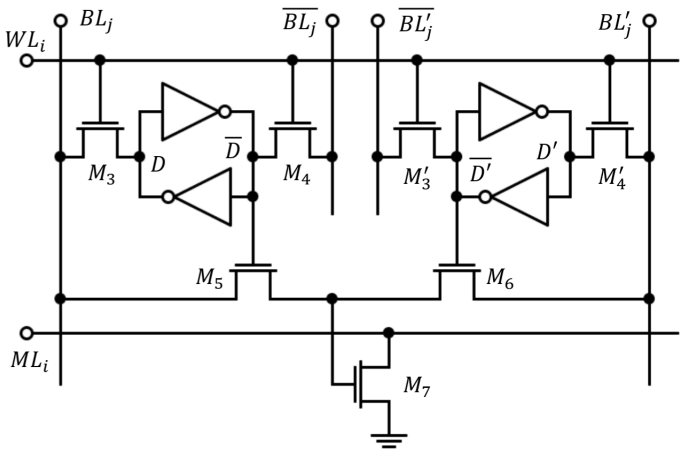
Doing matchin in hardware means all matching performed in parallel, which is much better than scanning the entire memory sequentially. This makes for a very quick result (tens of nanoseconds).

Of course, because of increased parasitic capacitances, the larger the memory, the slower the matching.

It goes without saying that such a ram is more expensive than an SRAM, with 9 transistors compared to 6 transistors for a single cell. The bit cell is also larger, has high power consumption, and especially high power consumption during the match operation (we have to precharge all match lines, only to discharge most (or all) of them).

9.1.3 Ternary CAM

There is another kind of CAM, which allows for *don't care* states to be encoded directly in the memory.



To do this we have two bit cells for each bit in the memory. We map the usual `10` and `01` values to logic 1 and 0 (same as normal SRAM), but also reserve `11` for don't care.

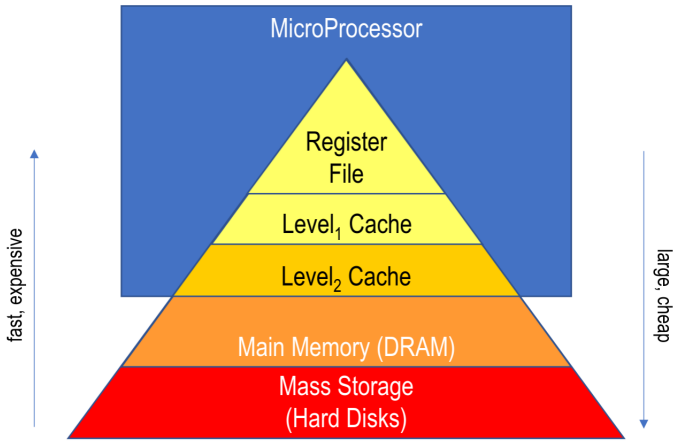
Matching can be done the same, as searching for `00` when the actual value is `11` keeps the ML_i high.

This kind of memory is really useful in *routers*, where routing tables often involve bits which are not specified by the address mask. For *cache* memories, on the other hand (which will be the devices we will study later), we don't care about this functionality, and as such ternary CAMs are *not used*(!).

9.2 Cache memory

Computing systems need large, fast and cheap memories. We saw a few options:

- **DRAM**: large, cheap but not fast;
- **SRAM**: fast, but not large and cheap;
- **Flip-flops**: we can implement *registers* with these which are as fast as we can get.



At this point we note that if SRAM is small, it becomes cheap, so there is some value in building a hierarchy of different memories to get high performance at low cost.

- Expensive (and fast) memories should be used to store little, frequently accessed data. An SRAM is a great fit for this;
- Cheap (and slow) memories should be used to store the bulk of the data. DRAMs is also a great fit for this.

Note that the computing system should not be aware of the differences: it simply sees a large, fast and cheap memory.

A clear problem is that data in the large DRAM does not fit into the small SRAM. As we said, we intend to keep fractions of the DRAM in cache. To maximize the use of the cache, we should ideally predict the data that is required next, and transfer it from DRAM in cache before it is needed.

9.2.1 Hits and misses

We refer by **MISS** to the situation where data is queried but not present in cache, and by **HIT** to the situation where data is queried and indeed present in cache.

- For a MISS we have to check both the cache and the DRAM for actual data, so the total time the read operation takes is:

$$t_{\text{MISS}} = t_{\text{cache}} + t_{\text{DRAM}}$$

- For a HIT we have to check only the cache, so the total time the read operation takes is:

$$t_{\text{HIT}} = t_{\text{cache}}$$

By defining the hit rate H as:

$$H = \frac{n_{\text{HIT}}}{n_{\text{TOT}}}$$

We get that the average time is:

$$t_{\text{avg}} = Ht_{\text{HIT}} + (1-H)t_{\text{MISS}} = Ht_{\text{cache}} + (1-H)(t_{\text{cache}} + t_{\text{DRAM}})$$

Clearly, higher H is better.

9.2.2 Predictions

To (somewhat) accurately predict the data that the microprocessor will need we exploit **locality** of data. There are 2 types of locality:

- **Temporal** locality: the same data is used more often if it was recently referenced;
- **Spatial** locality: data around the data which was just used is more likely to be used next.

Instructions, for instnace, fit exactly in this framework as they are highly local (if we fetch instruction at `0x12` we will likely fetch at `0x16` next, assuming 32 bit instructions).

9.2.3 Cache performance

Let's talk about performance. A large cache potentially makes execution faster, and if more data fits in the cache, the HIT Rate is higher. Once all code and data is in cache we see no improvement in making the cache even larger.

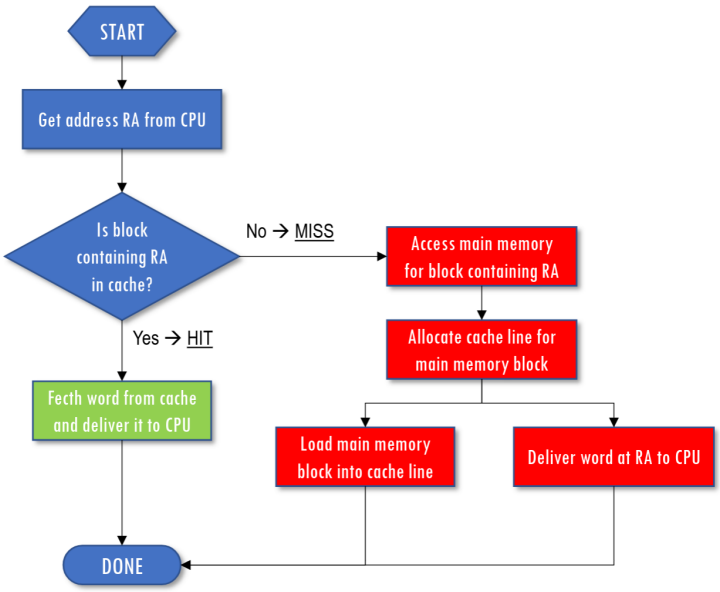
Also, large memories are slower, with higher parasitic capacitances (meaning higher t_{cache}). Checking cache for data takes time, and large memories consume more power.

A large cache is also more expensive, and the cost is additional to the large DRAM.

This makes cache sizing a trade-off between costs and performance, which is often application dependent.

9.2.4 Cache policy

Let's look at how actual cache systems are structured.



First we introduce the idea of a minimum unit of storage in cache, that is the **cache line**. Data in DRAM is divided in **blocks**, with each block the same size of a cache line. Blocks are aligned to the cache line size, and data is always moved to and from the cache in blocks.

At this point the microprocessor generates a main memory address, called **RA** (*Read Address*). Of course, he word at RA belongs to a certain block.

If that word is in cache (that is, in a certain cacheline), we have to note addresses in DRAM and cache are different. This means we need to find a fast way of knowing whether a given main memory block is alread stored in cache or not, and where.

The required data is identified by its address RA in main memory. Since data is transferred between main memory and cache in blocks, we can identify each block using its block address BA, obtained from RA. To determine whether a block is already present in the cache, we can use a *Content-Addressable Memory* (CAM), in addition to the SRAM used to store the actual data. The CAM contains one entry per cache line, storing the corresponding main memory block address BA, or just enough of it to identify the block. When a memory access occurs, the processor computes BA from RA and searches for a matching entry in the CAM. If a match is found, we have a *cache hit*: the CAM identifies the corresponding cache line, allowing the requested word to be retrieved from SRAM. Otherwise, we have a *cache miss*: the block is fetched from DRAM, stored in a cache line, and its address is recorded in the CAM. The requested word can then be returned to the processor.

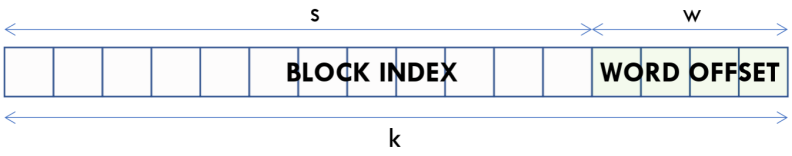
This is usually referred to as *fully associative* caches. We'll see that it's not the only approach.

10 Lesson 14-10-26

In the last lesson we started cache memories.

10.0.1 Cache parameters

Let's first look at some *parameters* regarding cachelines and DRAM blocks.



From the figure we see how a standard address is divided in multiple regions, that is:

- The **block index**, of width s, which identifies the block the address pertains to;
- The **word offset**, of width w, which identifies a single word in the block (shared by multiple words).

The parameters (s and w, as well as the full address size k) are defined below.

- **Mainsize**: the size of DRAM main memory (in words). We have:

$$k = \log_2(\text{Mainsize})$$

- **Cachesize**: the size of SRAM (in words);
- **Linesize**: the number of words in a cache line. We have:

$$w = \log_2(\text{Linesize})$$

- **Nlines**: the number of cache lines. It's defined as:

$$\text{Nlines} = \frac{\text{Cachesize}}{\text{Linesize}}$$

We also define:

$$r = \log_2(\text{Nlines})$$

- **Nblocks**: the number of blocks in DRAM. It's defined as:

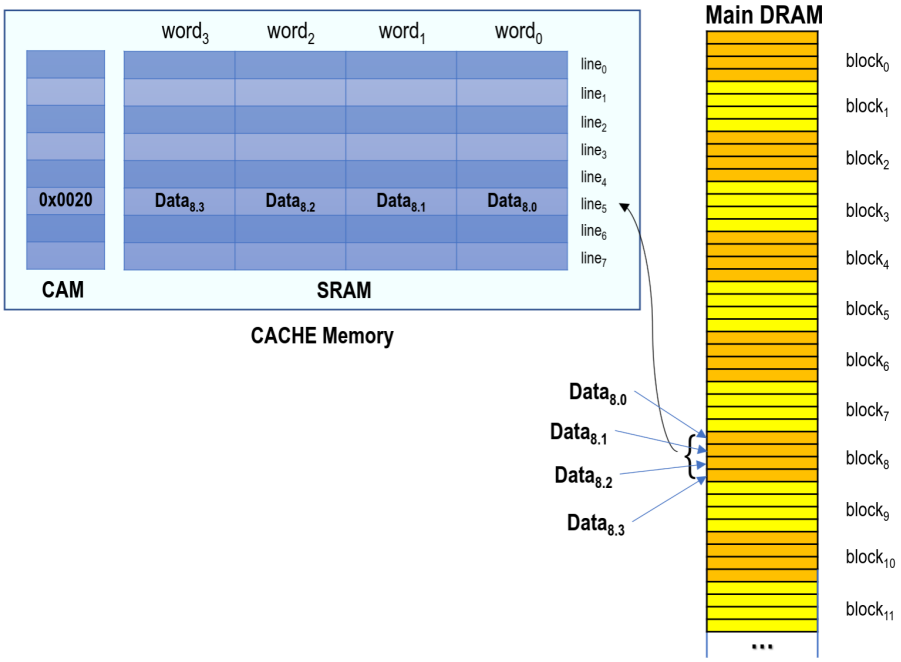
$$\text{Nblocks} = \text{Mainsize}Linesize$$

We also define:

$$s = \log_2(\text{Nblocks})$$

10.0.2 CAM cache

The first approach introduced involved a CAM. Let's give a quick demonstration.



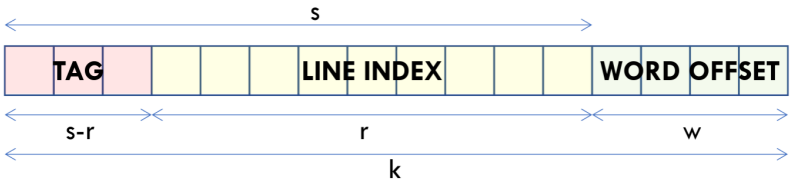
- Say the cache line is 4 words large, with the whole cache initially empty. The processor issues a read for the word at address `0x0022` (the Read Address). This belongs to the ninth block (block8), and the block address is `0x0020` (the Block Address). Say we mean to store block8 in the sixth cache line (line5): to do this we store its address in the sixth location of the CAM. After this is done, we return word2 from the cache line.
- Next, assume the processor issues a read for the word at address `0x0023` (the Read Address). This belongs to the ninth block (block8), as well, so when we match the Block Address `0x0020` to the CAM, we get a HIT, and the CAM returns the line5. At this point we can quickly read word3 from the cache line

There are some clear *policies* to define for this:

- When reading from DRAM and storing in cache, we must decide where to write the data. This is the **mapping** policy;
- If the cache is full and new data has to be written, some previous data must be erased. This is the **replacement** policy;
- When writing some data, cache and main memory should be both updated. this is the **consistency** policy.

10.0.3 Direct mapping cache

An alternative approach is one where the association between a block in main memory and a cache line is unique by design. Such an unique mapping, in hardware, is easily implemented via a modulo hash operation, by taking the least significant r bits of the block index as a line index, and the most significant bits as a unique tag.



In practice, we don't need to compute anything, as the index can be directly obtained from the address word by proper masking and/or shifting:

$$\text{Line} = (\text{Address} >> 2)\%(\text{Nlines} - 1)$$

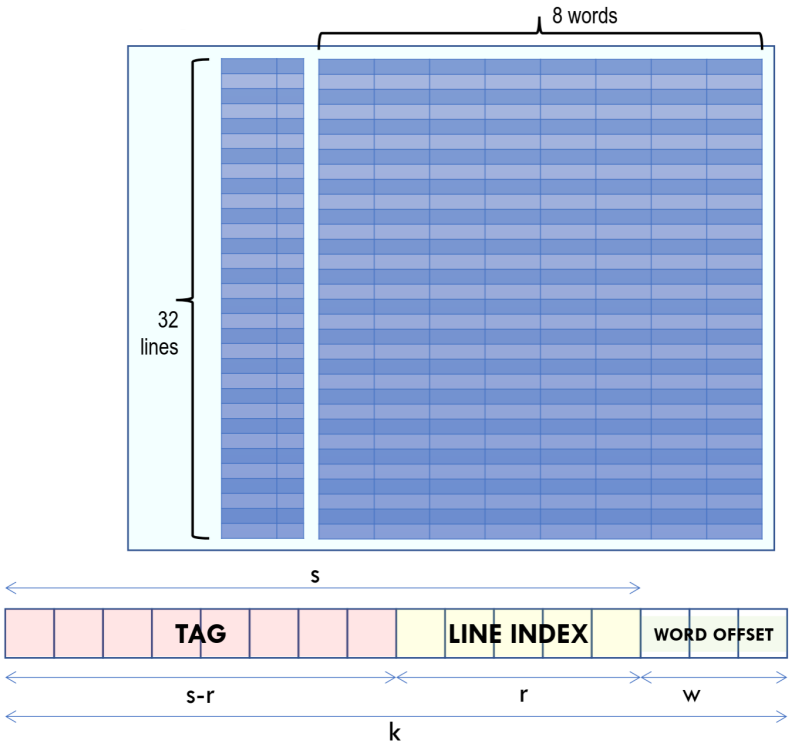
in binary, where $(\text{Address} >> 2)$ is nothing more than the *Block Address*.

Multiple blocks are assigned to the same cache line (as we said, the DRAM is bigger than the whole SRAM), with only $\frac{\text{Nblocks}}{\text{Nlines}}$ blocks mapped to each cache line. Only one of them is loaded in cache at any given time (so we have to *evict* old ones).

Consider blocks in main memory mapped to the same cache line.

- The address of each block always has the w least significant bits at 0, so we do not need to store this information in the CAM.
- The *Line* index is the same for all blocks assigned to the same cache line, as it corresponds to the cache line to which they are mapped, so we also do not need to store this information in the CAM.

Only the *TAG*s differ: the TAG is made of the upper $s - r$ bits of the address. This is the only information that needs to be stored in the CAM: different blocks mapped to the same cache line are guaranteed to have different TAG.



At this point, given the address of some data, what we need to do is:

- Match the TAG with the CAM at location pointed by *Line* index (only if the entry is valid). The CAM is actually made of *Nlines* separate CAMs of a single word;

- On MISS, we read the memory block from DRAM at address *Blockaddress*. The TAG is stored in the CAM, in the location pointed by *Lineindex*, so we can instantly match (and rewrite for misses). The block data is stored in the SRAM, in the line pointed by the *Line* index, possibly replacing other data already present. Then, a validity bit is set for the line, and we can read the word at *Word Offset* in the line pointed by the *Line* index.

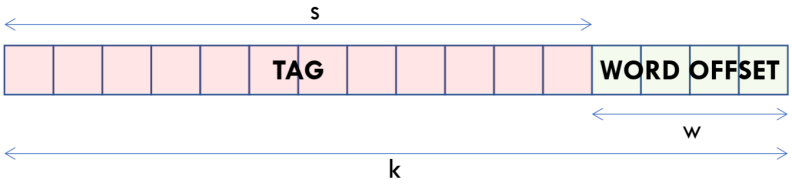
  A word can be spent on the validity bit: this is concatenated to the TAG, and when we match we match TAGs with the validity bit at 1. If the cacheline was never written to then the validity bit is at 0 and we definitely don't match;
- On HIT, we just read the word at *Word Offset* in the line pointed by the *Line* index.

Let's look at the *performance* of this. The advantages are that direct mapping cache is very simple to implement, cheap. We don't even need a CAM, as the *Line* index already points to the only location to match. This makes for low power consumption, and the simplest replacement policy: no policy at all, since it's built into the memory structure.

The disadvantages are that we may replace some data even if the cache is not full (on misses), leading to poorer performance. The HIT Rate is also the poorest out of all cache designs we'll see.

10.0.4 Fully associative mapping cache

This is the one we introduced in 9.2.4. No pre-determined association is defined between a main memory block and a line: when loaded from DRAM, a block is stored in the first available cache line. This means that there is no pre-determined *Line* index.



Basically, in mapping terms, we can say that TAG is the same size as *Block* index, and there is no *Line* index.

At this point, given the address of some data, we match the TAG with the entire CAM (only valid entries).

- We first match the TAG with the CAM;
- On MISS, we read the memory block from DRAM at address *Blockaddress*. Need a replacement policy if cache is full. After that, the block data is stored in the SRAM, in the same line as TAG. A validity bit is set, and we read the word at *Word Offset* of the line;
- On HIT we read the word at *Word Offset* in the line that generated high when matching with the tag.

Let's talk about performance. The advantages are the best use of the cache, and the best hit rate possible. This means we ossibly get the best performance.

The disadvantages, of course, are the complexity to implement, the costs, and the fact that we need a large CAM with higher power dissipation and higher delays.

10.0.5 Set associative mapping cache

Set associative mapping is an in-between approach between direct mapping and fully associative.

Cache lines are grouped into **sets**:

- **Nsets** is the number of sets;
- **Setlines** the number of lines in a set.

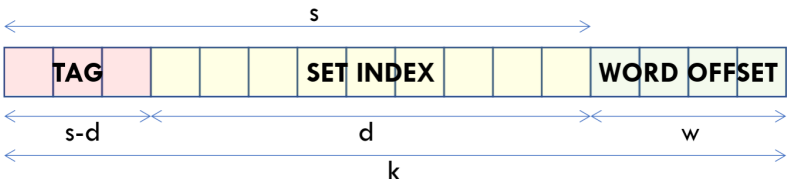
From this we have that:

$$\text{Nsets} = \frac{\text{Nlines}}{\text{Setlines}}$$

And the bits needed to identify a set are:

$$d = \log_2(\text{Nsets})$$

We have *direct association* of a main memory block towards a set, and *fully Associative association* within that set.



As with common direct mapping, the set index is $\text{Set} = \text{Blockindex}\%\text{Nsets}$, and the TAG is $(s - d)$ bits long.

At this point, fiven the address of some data, we:

- Match the TAG with the CAM of the *Set* pointed by the *Set* index (only valid entries). There are *Nsets* separate CAMs, each of *Setlines* entries;
- On MISS, we Read the memory block from DRAM at address *Blockaddress*. The TAG is stored in the CAM of the *Set*, in the next empty line. We of course need a replacement policy if the set is full (**LRU**, *Least Recently Used*, is a good candidate). The block data is then stored in the SRAM, in the same line as TAG. A validity bit is set for the line, and we read the word at *Word Offset* in the line.
- On HIT, we read the word at *Word Offset* in the line that generated HIT when matching.

10.0.6 Cache replacement policies

When a cache is full, a cache line must be replaced to store a new one. That is our *cache replacement policy*. We have a few options:

- **Direct mapping**: we have no choice, may replace a cache line even if the cache is not full;
- **Associative mapping**: set associative mapping may replace a cache line even if the cache is not full (but the target set is full). The algorithms are:
  - **LRU** (*Least Recently Used*): we store timestamps of last usages. The con is the need to sort timestamps (within a set or within the full cache, for Set and Fully associative, respectively). Intel processors used some interesting heuristics to approximate this in the 486 era;

- **LFU** (*Least Frequently Used*) we use counters to know the number of accesses. We need to sort number of accesses (within a set or within the full cache, for Set and Fully associative, respectively);
- **FIFO** (*First In, First Out*): we restart from first line when the bottom (of a set or the full cache) is reached. A simple pointer is required (one for each set, or one for the full cache, for Set and Fully associative, respectively);
- **Random** choice is not that useful, but sometimes involved in lower steps of more complex algorithms.

10.0.7 Cache consistency policies

We then need policies on how we handle *writes* thorugh the cache. That is, when writing, we have decide how to update cache and main memory. This is important especially in multi-core systems and with peripherals accessing the memory, where each core has a dedicated cache.

A few approaches:

- **Write through**: all writes go to main memory as well as cache. Multiple CPUs can monitor main memory traffic to keep local caches up to date When a core makes a write, the other cores invalidate the cache entry relative to the written block. This involves lots of traffic, which slows down write operation. On the other hand it has the best consistency. Plus note that on average, only 15% of memory references are writes.
- **Write fatback** the update is initially made in cache only. An update bit is then set for cache line (usually called the *dirty bit*). If line is to be replaced in cache, we write back to main memory if dirty bit is set. This saves a lot of traffic on the memory bus, and makes for improved performance. The tradeoff is of course the fact that other caches get out of sync with the dirty ones (or in fact they might have different dirty values to write back concurrently).

10.1 Programmable Logic Devices

A **PLD** (*Programmable Logic Device*) is a way to implement custom computation hardware for a given application.

We said (in some course, don't remember which) that there are a few approaches to computation logic design:

- **Full custom IC**s: these obviously give the best performance and power consumption, but with very high NRE costs (masks, design). Designing a custom IC requires a long time-to-market, and is good only for demanding very high volume designs;
- **Gate array**s: these have very good performance and power, with high NRE costs (masks). We only need to build the higher layers masks, not the full IC like in full custom. Still, this has long time-to-market, and is good for high volume designs.

  Note that, altough **FPGA**s (*Field Programmable Gate Array*) have "gate array" in their name, they are usually considered PLDs. The good mental model is that gate arrays require additional IC routing after manufacturing, whereas PLDs are programmable after manufacturing, typically using SRAM configuration bits, flash, or antifuses;
- **Programmable Logic Device**s: good performance and power (compared to software), with low NRE costs (compared to a custom IC). There are no masks involved, and the

user buys an already manufactured integrated circuit. This involves higher per-unit costs, so it's good for low volume designs and prototypes.

To summarize, a PLD is an integrated circuit (hardware device) that contains logic elements of various complexities. The interface allows the designer to configure and connect them in different ways, configurating the device after manufacturing. A PLD implements a hardware digital circuit, where the function of the circuit is configured by the user of the PLD. The masks do not depend on the implemented function, so the same device can be used for different applications. PLDs can be produced in large volumes, with NRE costs (for the PLD manufacturer) spread over many different customers.

PLDs can be configured to implement a logic circuit, either combinational or sequential. The final circuit is hardware, not software, but it is usually described using a high level "programming" language (VHDL, Verilog, SystemC, Chisel, HLS). Designers define internal logic functions, Interconnections and pin, using said language.

Technologies used to configure PLDs are:

- Fuse, anti-fuse (one time programmable, think OTP-ROMs, 7.1);
- Floating gate (reprogrammable, think EPROMs and EEPROMs, 7.2 and 7.2.5);
- SRAM (reprogrammable, reconfigurable at run-time, faster than floating gate technolog.

10.2 ROM based PLDs

We know from Shannon's theorem that memories (LUTs) and multiplexers can implement any combinational logic functions. For instance, a $2^m \times n$ can implement any set of n functions of m shared inputs.

The memory can be OTPROM, EPROM, E2PROM (permanent), SRAM (volatile). Definitely, it can't be ROM, otherwise we would make it full custom. Also not NAND Flash, because of the clunky sequential read access. Lastly, not DRAM, because of the complex access protocol (and active refreshes).

When used as a circuit, the memory is always read: it is written only before its use, to configure the circuit.

11 Lesson 15-10-26

11.0.1 ROM based PLD performance

In the last lesson we began covering PLDs, starting with ROM based PLDs. These are easy to implement, but have memory size requirements growing exponentially with the number of inputs. This makes them slow, with address decoder which become large and very deep in levels of logic.

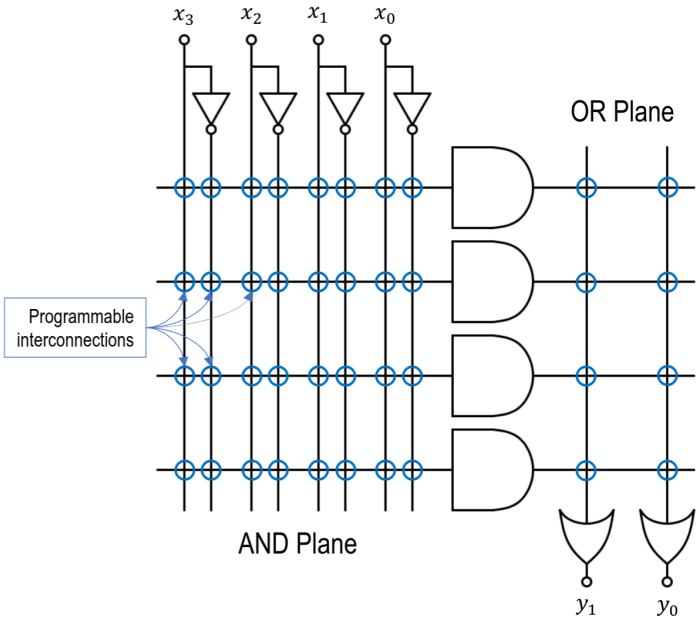
11.1 Programmable Logic Arrays

The alternative approach is given by *Programmable Logic Arrays* (**PLA**s). The basic structure of a ROM based PLD is:

1. The NOT plane, where we get signals and their inverted values;
2. The AND plane, which ands the signals from the NOT plane;
3. The final OR sum on every output.

This is the basic **SoP** (*Sum of Products*) architecture.

A programmable Logic Array is just a circuit made up of programmable AND and OR planes, where the interconnections are designed by the user.



An m-input n-output PLA with k AND gates (each with k inputs), and $k < 2^m$, can implement any set of n combinational logic functions of m inputs, provided that the total number of implicants (cubes) is not larger than k, the number of available AND gates. Note that a ROM can implement any function as it has $k = 2^m$ AND gates.

This is usually not too much of an issue, plus it's really efficient:

- PLA size linearly depends on the number of inputs: the number of AND gates k does not depend on the number of inputs m (it is only restricted by it, should it become a PLD). This means the PLA size linearly depends on the number of AND gates;
- ROM size is exponential on the number of inputs.

The number of AND gates (k) is decided by the manufacturer based on constraints, like performance, costs, etc...

It is interesting to see how the **programmable interconnections** are designed. All intersections are programmable interconnections, and the programming techniques are basically the same as memories:

- For the OR Plane, we usually use wired OR logic similar to memories, programmed via fuses or floating gate transistors;
- For the AND Plane, we similarly use fuses or floating gate transistors.

11.1.1 PLA performance

Let's evaluate the performance of PLAs. Speed is low, as in ROMs (if not slower), due to the several programmable interconnections: we have high parasitic resistances and capacitances.

The problem worsens when implementing sequential functions, where the output is in feedback to the input, *outside* the integrated circuit.

Lastly, by increasing the number of input and output pins, we get gates with many inputs (which are slow).

11.2 Programmable Array Logic

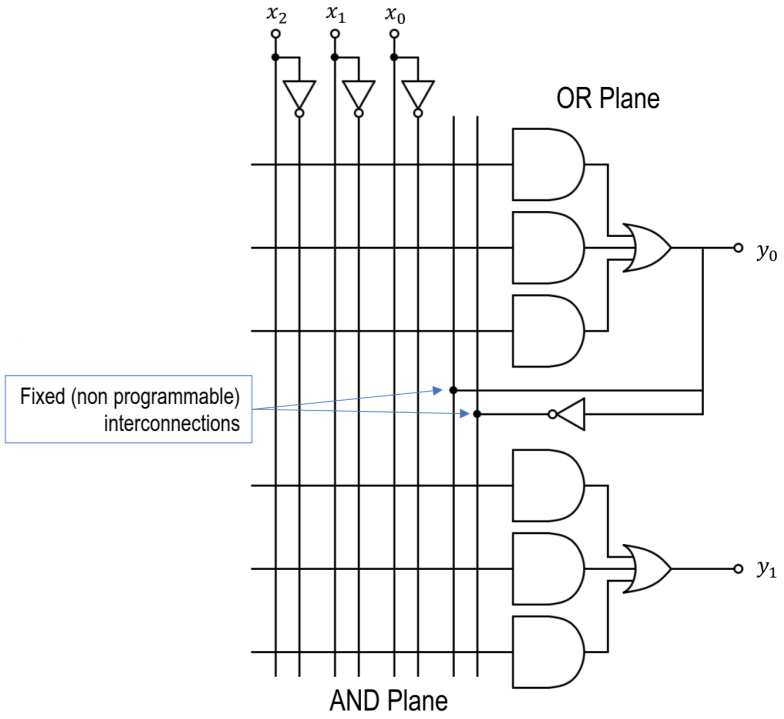
Programmable Array Logic (**PLA**) is yet another approach that aims to increase speed by reducing flexibility.

The idea is to reduce the number of programmable connections, by reducing parasitic resistances and capacitances. This makes devices faster than ROMs and PLAs.

We then settle on *programmable AND, fixed OR*, with the AND plane identical to PLA and a fixed OR plane.

There are limits the number of implicants for each function, regardless of the total number of AND gates in the AND plane, so a restricted set of combinational functions that can be implemented compared to ROM and PLA.

Let's look at the structure:



From the figure we notice:

- The input NOT plane, giving affirmed and complemented inputs;
- Grouped AND gates, with programmable (not shown in figure) interconnects to the inputs;
- Fixed output OR plane connections, *but* with output feedbacks before the AND plane, alongside inputs. This allows to implement functions using multi-level logic.

11.2.1 PAL performance

PAL has a few probelms. First of, fewer functions can be implemented using a PAL with respect to a PLA, and outputs cannot share implicants among functions.

Feedback connections may help, but they use more resources, making for higher costs and delays.

Also, sequential logic cannot be implemented directly: feedbacks can be used in principle to implement sequential logic, but they are too slow to be useful.

11.3 Macrocells and macroblocks

A **macrocell** is a configurable logic circuit made of a PLA or a PAL, with a flip-flop added to each output. Multiplexers are used to select, either via:

- Combinational bypass (direct and inverted);
- Latched output (through the flip-flops, direct and inverted).

Only the selection inputs of the multiplexers can be configured, not the data inputs or outputs, and they are configured using the same technology as the combinational logic.

A **macroblock** is a collection of macrocells, which share all inputs and PAL feedback.

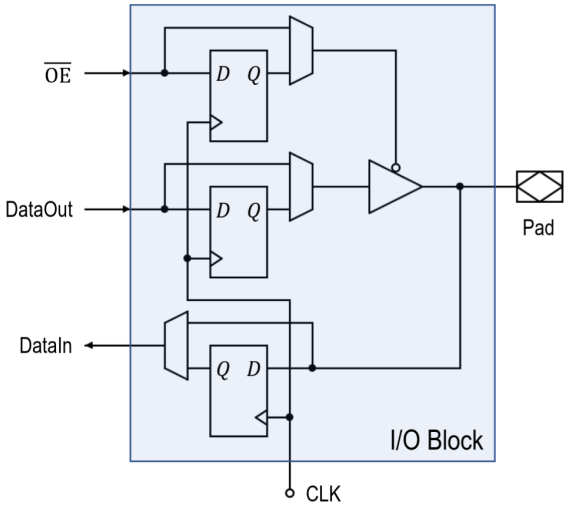
12 Lesson 16-10-26

12.1 Complex Programmable Logic Devices

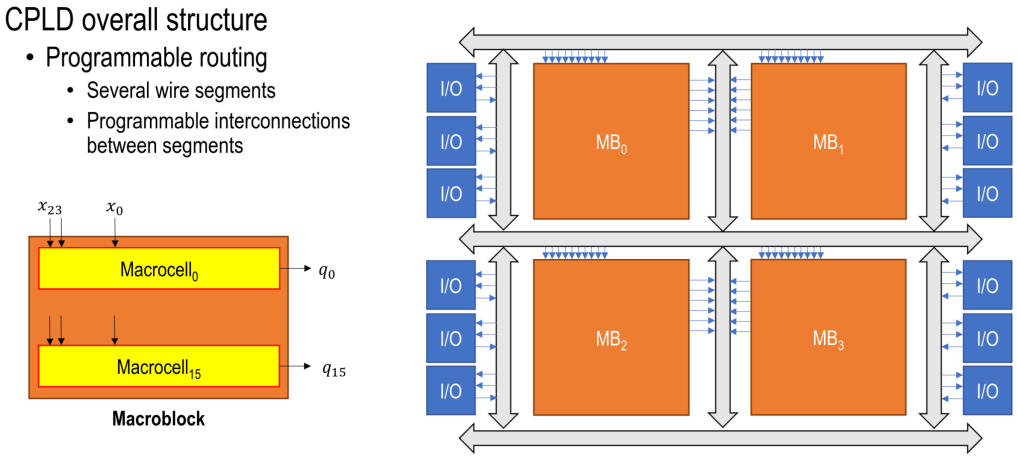
In complex circuits, the number of inputs and outputs increases. Arithmetic operations usually require many signals, such that delays increase as the number of inputs grows. Noting that functions operate on a subset of the total number of inputs, we see that it is not necessary to have a big PAL (or PLA) with all functions implement.

We said that grouping together multiple macrocells formed a logical unit known as a macroblock. Now, by grouping together multiple macroblocks, we get a *Complex Programmable Logic Devices* (**CPLD**). CPLDs need programmable interconnections to route signals from one Macrocell to another, ithin the same logic unit or between different logic units.

First off, we briefly talk about **I/O blocks**. When packaging logic devices into IC packages, for each pin we might like to have some circuitry that governs wether that pin is inputting or outputting data, and also acts as a synchronized buffer for that data. Such circuit is called an I/O block and shown below.



At this point we can package the actual macroblocks (shown left) toghether, interfacing them with the I/O blocks.



12.1.1 CPLD performance

The main problems of CPLDs is the restricted number of flip-flops: we cannot implement complex finite state machines, so we cannot implement complex sequential data paths, and naturally we cannot implement a processor (which requires lots of Flip Flops for the FSM in the control unit, and lots of Flip Flops for registers in the data path). As such, CPLDs used for glue logic, configured at low granularity.

12.2 Field Programmable Gate Arrays

We now get to **FPGA**s (*Field Programmable Gate Arrays*). Many digital circuits need large storage for temporary data: for instance pipeline registers, shift registers, counters, FSM registers, memories, FIFOs, register files, ... CPLDs do not provide enough storage

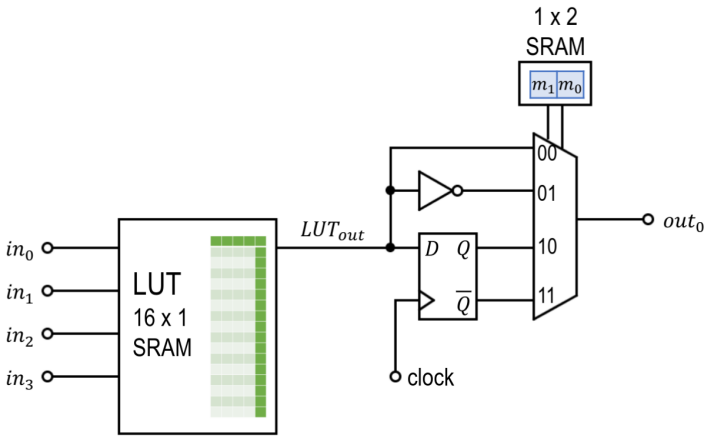
FPGAs are programmable logic devices based on *Look-Up Tables* (**LUT**s). A LUT is a small memory used to implement combinational logic functions, from 3 to 6 inputs and 1 or 2 outputs. The limited number of inputs is to avoid the exponential explosion. They are often implemented via SRAM. Each LUT can also have an associated flip-flop.

As such a field Programmable Gate Array is collection of a large number of **CLB**s (*Configurable Logic Blocks*, in AMD terminology) or **LE**s (*Logic Elements*, in Intel terminology). CLBs and LEs are smaller than a macrocell, organized as a matrix array of rows and columns. Programmable interconnections allow to route signals from one CLB to another, with wires running between rows and columns. Configurable I/O blocks allow to connect to the external world, in a similar fashion to CPLDs.

12.2.1 Logic Elements

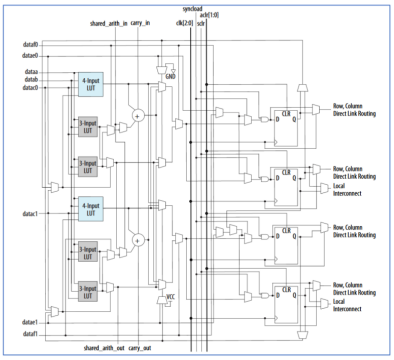
Let's stick to Intel terminology. A **logic element** is a device which can have many different designs, with the figure shown being a simple example.

Usually, an LE comibnes a LUT with output selection circuitry (a multiplexer, which might choose between standard outputs and synchronized ouputs). The multiplexer configuration is usually based on the same technology as LUTs (SRAM in most cases).

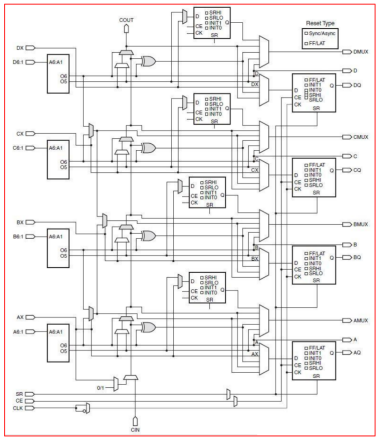


Field Programmable Gate Array (FPGA)

- Logic Element
  - Advanced LEs and CLBs from Intel/Altera and AMD/Xilinx



Intel/Altera ALM



AMD/Xilinx CLB

13 Lesson 21-10-26

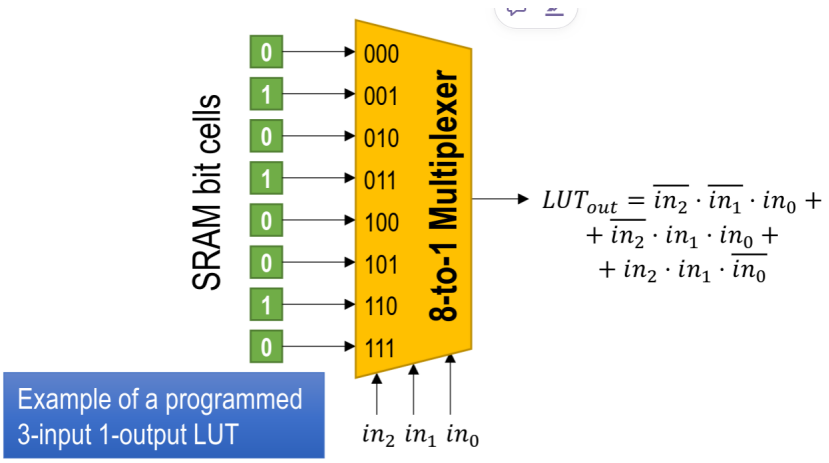
13.0.1 FPGA LUTs

Let's look at FPGA *Look-Up Tables* (**LUT**s) in detail. A LUT basically implements an n-input, 1-output combinational logic circuit, made up of:

- 2^n SRAM bit cells;
- A $2^n \to 1$ multiplexer, with n selection inputs.

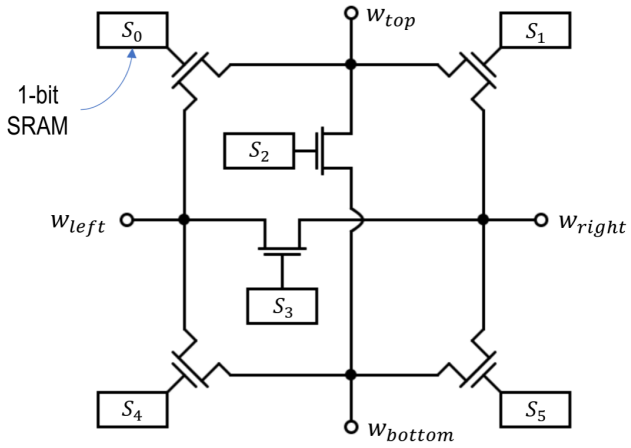
Note that an n-input,-1-output LUT is like a $2^n \times 1$ memory with:

- A single row, so need of a row decoder, and a row that is always selected;
- 2^n columns, where the MUX is the column decoder and column multiplexer of the memory.

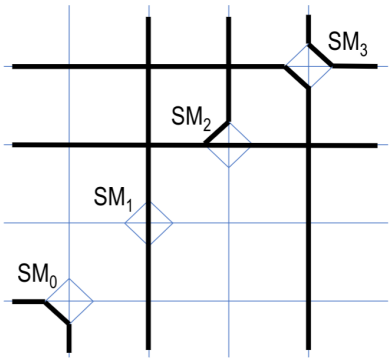


13.0.2 Interconnections

Interconnections in FPGAs are usually implemented via **switch matrices**, usually made up of 6 pass transistors.



It is pretty clear how, by changing the contents of the control SRAM we can change the thresholds of the pass transistors and thus the pairs of nodes across w_{top}, w_{right}, w_{bottom} and w_{left}, creating any kind of interconnection.



	SM_0	SM_1	SM_2	SM_3
S_0	0	0	1	0
S_1	0	0	0/1	1
S_2	0	1	0	0
S_3	0	0	1	0
S_4	1	0	0	1
S_5	0	0	0	0

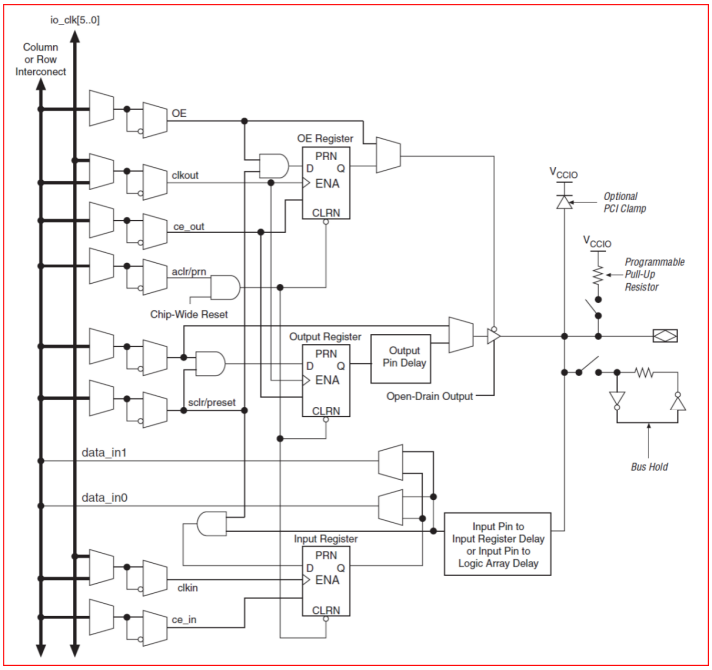
A switch matrix divides wires into independent segments. Segments are connected by properly activating transistors in the switch matriix. If not connected, adjacent segments may belong to different interconnections, with no conflicts because inactive transistors separate them.

Because a switch matrix introduces delays and increases the area, we tend to limit the number of switch matrices to improve performance and decrease costs. On the other hand a switch matrix increases flexibility, and more switch matrices make routing easier.

13.0.3 I/O blocks

FPGAs also feature **I/O blocks** such as the ones saw for CPLDs in 12.1. Because FPGAs are usually more complex, though, the I/O blocks are usually more complex too. In this they have:

- Output buffer with Enable signal;
- Combinational or sequential inputs and outputs;
- Optional Pull-Up, Clamp and Bus Hold circuits, configurable by controlling the selection signals of multiplexers



13.0.4 Additional FPGA blocks

FPGAs, being more complex and versatile than other logic devices, usually feature additional blocks such as:

- **Memories** and **FIFO** (*First-In, First-Out*) queues. These are usually SRAMs, but not control SRAM cells like we saw before: actual SRAM topologies in the order of kilobytes, with address decoders and bit multiplexers. Sometimes interfaces for external dynamic memories are available.
- **Carry chains** between different CLBs or LEs: these allow implementing efficient adders;
- Embedded hardware **multipliers** or **DSP** (*Digital Signal Processing*) blocks;
- Clock management circuitry;
- High speed **I/O**;
- Even **processor cores**.

13.0.5 FPGA performance

Let's do a few performance and design considerations. First off, the kind of technology used to program the FPGA is slighlty varied:

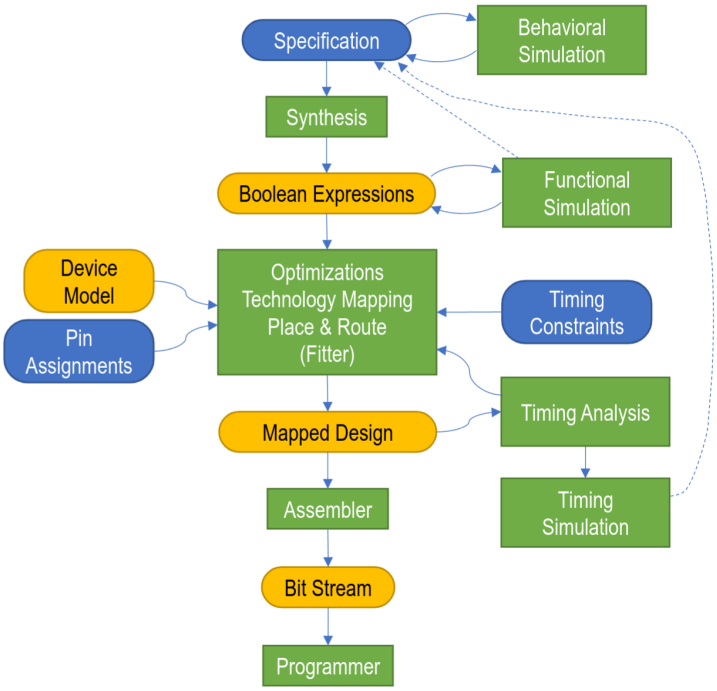
- **SRAM** is dominant, used for LUTs, but also to control MUXes or pass transistors. Controlling standard CMOS technology, it can be reprogrammed on the fly, which makes for a standard technology which allows to easily embed other blocks (which is the reason FPGAs are usually at the cutting-edge of a given technology). The main con is the large size, power consumption, and the fact that SRAM is not permanent;
- **Fuse and Antifuse** is sometimes used as its small and permanent, but it cannot be reprogrammed nor tested;
- **EEPROM** or **Flash** can be reprogrammed, but don't represent a standard proces.

We can then compare to other PLD technologies, namely **CPLD**s.

FPGAs are configured at a finer level than CPLDs, which have few large configurable blocks, while FPGAs have many small configurable blocks. FPGAs are volatile, typically implemented using SRAM technology, and must be reprogrammed at every power-up, whereas CPLDs are non-volatile, typically using EEPROM technology. FPGAs use standard technology, making it easier to include additional resources. They are more expensive, but cheaper per gate, and usually much larger. CPLDs are more predictable, while FPGA delays can vary significantly with the implementation.

13.1 FPGA design flow

We know come to the **design** part of using FPGAs.



This is easier than wrangling LUTs manually, as it usually involves:

1. **Specification** and **Design Entry**, through specialized hardware description language such as VHDL, Verilog, SystemC, Chisel, HLS.
2. **Behavioral Simulation**, that is the execution of the specification, with no timing information, needed to verify the algorithm and quickly explore the design space;
3. **Initial Synthesis**, which generates synthesizable boolean expressions independent of the target implementation;
4. **Functional Simulation**: similar to Behavioral Simulation, also known as **RTL** (*Register Transfer Level*) simulation. This still has no timing information;
5. **Logic Synthesis**. This is the hard part. It involves echnology independent optimizations, technology mapping and Place & Route. At this point we need a device model to be known, provided by the manufacturer, and we need the pin assignments. Logic synth. is driven by timing constraints, as we want to look for best implementation on a given technology. As we said, it's a very complex problem, solved via simulated annealing, genetic algorithms, ecc... which may get different results at any run;

6. **Timing analysis** and **Timing Simulation**: we can finally compare the circuit with timing constraints, also known as gate level simulation. This is far slower than other simulation kinds;
7. **Assembler**: generates the Bit Stream for the target device, and as such all SRAM configuration bits;
8. **Programmer** writes the Bit Stream on the actual device through its JTAG Interface. The SRAM acts like a very long shift register, which is serially programmed, to reduce pin counts (and allow JTAG chaining, more on the good ol'wikipediaz).

14 Lesson 23-10-26

14.1 Interconnections and communation protocols

We are finally getting to connecting our components together. As components are usually quite smart themselves, simple interconnections are not fit for the purpose, and we have to define more complex *communication protocols*.

There are a number of protocols:

- **UART** (*Universal Asynchronous Receiver Transmitter*);
- SPI;
- I2C;
- **CAN** (*Controller Area Network*).

Interconnections can be at several levels:

- *Within* an integrated circuit:
  - Metal layers, parallel bits
  - Lots of wires, high performance
  - Low parasitics, low noise
  - Delays can be accurately estimated (Elmore)
- Between *integrated circuits* on the board
  - Copper lines on a PCB, parallel or serial
  - Higher noise, higher delays than in an integrated circuit
- Between *boards* in the same embedded system:
  - Cables made of wires, parallel or serial
  - Even higher noise and delays
- Between *embedded systems*:
  - Cable made of wires or wireless connections
  - Unpredictable delays and extremely high noise
  - Need to use at least error detection in many cases

The goal of interconnections is to transfer bits as quickly as possible, while not sacrificing metrics:

- Correctness (definitely, as we are transfering digital information);
- Reliability;
- Power consumption;

- Costs;
- ...

Interconnections are defined at different *levels*, as we usually see in the OSI model:

- **Physical**, which refers to the actual electrical wire interconnects.
- **Protocol**, which refers to how the signals these wires transmit are organized;
- **Application**, of interest from the computer engineering point of view, refers to the actual application messages our systems are sending to each other.

Physical and protocol focus on how to transfer bits from a transmitter to a receiver. This requires bit synchronization (especially for serial protocols). In this, we examine the information needed to successfully complete a transfer, with no details on the semantic content of the messages that are transferred.

14.1.1 Interconnection delays

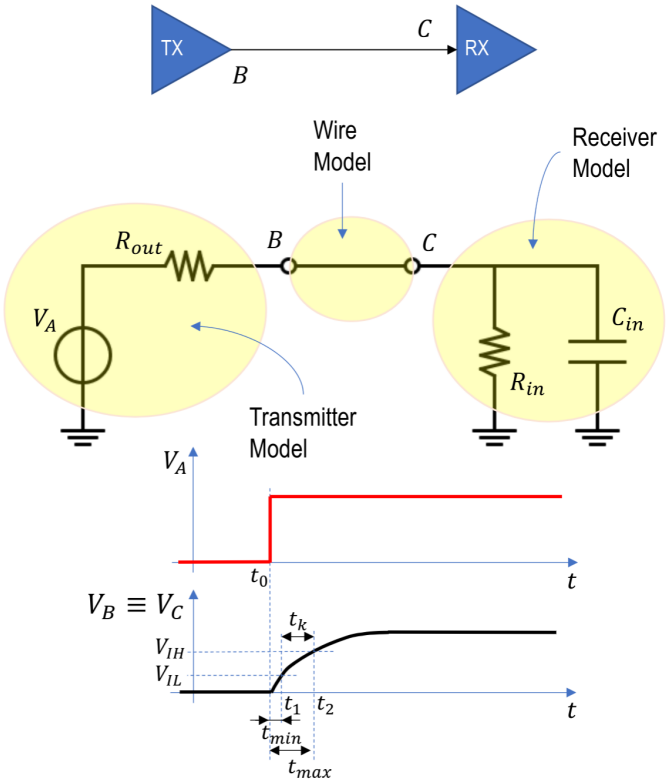
When connecting the output of a logic gate to the input of another logic gate, **delays** affect the exchange of data. We're interested in *measuring* these delays: even if the transmitter generates an ideal step, the waveform at the receiver is generally exponential with a certain time constant. This is, as usual, due to parasitic resistance, capacitance and inductance of transmitter, receiver and wire. The different output response results in a difference in switching thresholds of the receiving logic gates. This causes different delays for nominally identical circuits.

The delay is affected by an *uncertainty*, called **skew**.

Once the *skew* is known, the delay can be expressed as:

$$t_d = t_{\text{prop}} \pm \frac{t_k}{2}$$

Where t_{prop} is an average propagation delay.



The delay depends on the receiving gate threshold, between V_{IL} and V_{IH}.
We see that at the receiver we get an exponential ramp with time constant:

$$\tau = C_{in} \cdot (R_{in} \parallel R_{out})$$

Now, based on the input gate threshold, the delay time sits between t_{min} and t_{max}:

$$t_{\text{min}} = t_1 - t_0$$
$$t_{\text{max}} = t_2 - t_0$$

Where t_1 is where we meet V_{IL} and t_2 is where we meet V_{HL}.
The **skew** t_k is exactly:

$$t_k = t_{\text{max}} - t_{\text{min}} = t_2 - t_1$$

So we know that to correctly characterize the entire interconnection we have to:

- Find the minimum of all minimum delays t_{min};
- Find the maximum of all maximum delays t_{max};
- Compute the average t_{avg} and skew t_k.

$$t_{\text{avg}} = \frac{t_{\text{min}} + t_{\text{max}}}{2} = \frac{t_1 + t_2 - 2t_0}{2}$$
$$t_k = t_{\text{max}} - t_{\text{min}} = t_2 - t_1$$

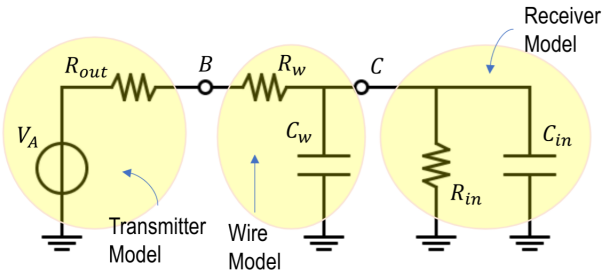
To summarize, we model an interconnection as lumped parasitic resistance R_w and capacitance C_w. The transmission shows a delay due to the input parasitic capacitance of the receiver, and due to the lumped parasitic capacitance of the wire.

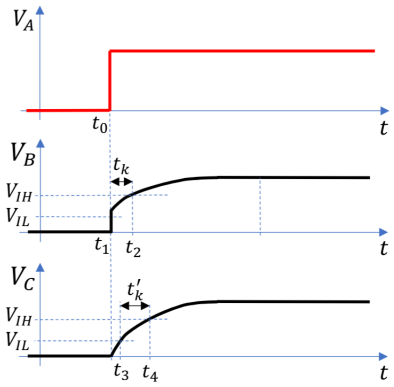
The delay shows an *uncertainty* (skew) due to different thresholds among nominally identical logic gates, and due to possibly different positions of the receiver along the interconnection wire. In short, wire length causes different parasitic resistance R_w and capacitance C_w values, which affects both the delay and its uncertainty (skew).

14.1.2 Models

Let's see some popular models for delays in interconnects:

- **Exponential**
  Let's complete the previous model by putting the *wire* back into the picture. As always, we model it with a parasitic capacitance and resistance:



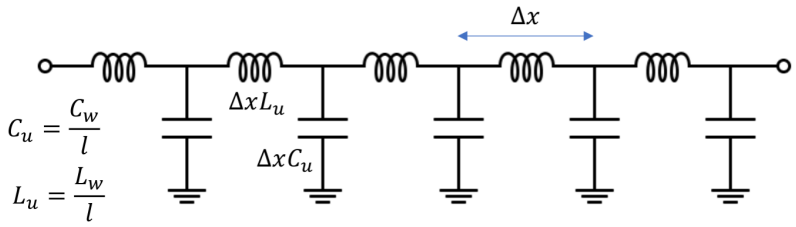


  In this case our time constant changes:

$$\tau = (C_{\text{in}} + C_w) \cdot (R_{\text{in}} \parallel (R_{\text{out}} + R_w))$$

  Note that this situation is made even more complex by the fact that the wire constants C_w and R_w *change* with the length of the wire, meaning that we have an added layer of uncertainty.

- **Transmission lines**
  The idea is to model an interconnection using distributed parasitics: we take a wire and divide its length l into N segments, each lumped. This means each segment has length $\Delta x = \frac{l}{N}$. We are interested in what happens if we take the limit of elements going to infinity ($N \to +\infty$).

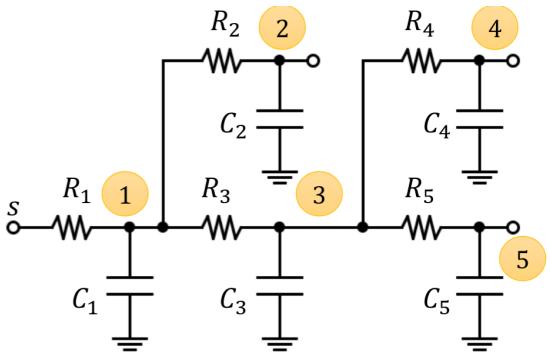


  If considering only inductance and capacitance, this is the **ideal transmission line with no losses**. Signal propagates with a certain speed, with possible reflections, and the delay depends on the position of the receiver along the line. For this we can compute an average and a skew.
  Note that, if the resistances are also included, losses are introduced in the model and we get the **ideal transmission line with losses**.

- **Elmore delay**
  Note that this already appeared in 7.0.2, for NAND topology memory timings. There's a useful note here: `https://raw.githubusercontent.com/seggiani-luca/ees-notes/`

`fcbb33c4bfdef8fc531aa81c04cddb69d309ad2a/notes/nand_topology_delay/main.pdf`.

The idea once again is to model an interconnection using distributed parasitics, by dividing the wire into N segments, each with a lumped model. For complex interconnections, we can determine a tree stemming from a single node s, where each segment is characterized by resistance R_j, and capacitance C_j.



Matematically, the time constant τ_i of the delay at node i is given by the sum of all RC segments up to i:

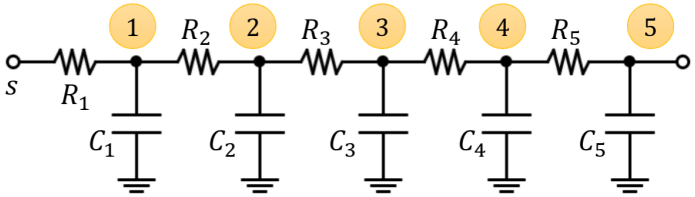
$$\tau_i = \sum_{k=1}^{N} C_j R_{ik}$$

Where R_{ik} is the shared path resistance between node i and k:

$$R_{ik} = \sum R_j \text{ where } (R_j \in \text{path}(s,i) \cap \text{path}(s,k))$$

- **Elmore delay line**

Elmore delay is easily applied to an RC segment line:



The shared path resistance is always the sum of all resistance between root s and the considered node. For the last node i the shared path resistance with node k is always:

$$R_{ik} = R_{kk} = \sum_{j=0}^{k} R_j$$

At this point we have that, for a wire of length l divided into N identical segments:

- All resistances are identical: $R_i = r\frac{l}{N}$ (where r is the resistance per unit of length);
- All capacitances are identical: $C_i = c\frac{l}{N}$ (where c is the capacitance per unit of length).

This means we can apply Gauss's formula:

$$\tau_{\text{tot}} = \tau_N = R_i C_i \frac{N(N+1)}{2} = R_{\text{tot}} C_{\text{tot}} \frac{N+1}{2N}$$

In the limit this is:

$$\tau \to \frac{1}{2} N^2 R_i C_i$$

Where:

$$R_{\text{tot}} = N R_i$$
$$C_{\text{tot}} = N C_i$$

Which should be the case we saw in our notes.

There is as slight notation difference between this in the notes:

– In the notes we took each capacitor at contributing t_i to the total delay time, thus giving:

$$t_{\text{tot}} = \sum_i t_i$$

– Here we took t_i as the delay at node i, already containing the delay from previous nodes (see the formula above for τ_i), thus giving:

$$t_{\text{tot}} = t_N$$

Nevertheless, the result we get in both cases is the same, we just changed the definitions of the derivations.

14.1.3 Parallel vs serial

To transmit a word of n bits, bits are transmitted according to a synchronization signal. The synchronization signal is a clock, where:

- The clock frequency is f_{ck};
- The clock period is $t_{\text{ck}} = \frac{1}{f_{\text{ck}}}$.

Note that the clock shall be transmitted as well as the signal. There are several ways of communicating the clock. We wish to compare these techniques based on throughput:

- The number of bits per second is Thr_{bits};
- The number of bits per second is $\text{Thr}_{\text{words}}$.

Assume the transmitted symbols represent single bits: we can increase performance by using multiple bits per symbol, thus lowering noise margins.

- **Parallel**
  This is the idea of **parallel**, as opposed to **serial**, transmission. We transmit an entire word using n parallel wires, with an additional wire for the clock.
  This increases performance linearly:
  – The number of bits per second becomes $\text{Thr}_{\text{bits}} = n f_{\text{ck}}$;
  – The number of bits per seconds stays $\text{Thr}_{\text{word}} = f_{\text{ck}}$, assuming the n parallel lines form a complete word. Because of this, the number of bits is often a power of 2.

  Parallel is certainly performant, but this comes at a cost we need a lot of wires for large busses, and its not good for long interconnections as its heavy, bulky, and of large size;

- **Serial**
  Serial is parallel's chill cousin, where we usually transmit the word and the clock using a single wire. Bits are multiplexed in time, one per clock cycle, and one after the other, sequentially (LSB or MSB first). This inevitably introduces latency at the word level. A problem is also that the clock must usually be embedded in some way with the data, which needs special encodings for clock recovery.
  The effects on performance are:
  - The number of bits per second becomes $\text{Thr}_{\text{bits}} = f_{\text{ck}}$;
  - The number of bits per seconds becomes $\text{Thr}_{\text{word}} = \frac{f_{\text{ck}}}{n}$, assuming again that n is the number of bits in a complete word.

  With them being mostly opposite, serial is way cheaper and simpler than parallel, if not a bit less performant.

14.1.4 Clock

There are limitations on the clock frequency. The receiving end usually has flip-flops to store the incoming data: setup and hold times t_s and t_h must be satisfied at the receiving end.

Let's consider first a simplified model (no skew t_k), and then a more accurate one.

- **No skew**
  With no transmission delays and perfect synchronization, the maximum f_{ck} depends on t_s and t_h of the receiving flip-flops only:

$$t_{\text{ck}} > t_s + t_h, \quad f_{\text{ck}} < \frac{1}{t_s + t_h}$$

  This is fine as, if clock and data have different (but known) delays, the transmitter properly produces the clock earlier or later so that setup and hold times are satisfied at the receiver.
  To satisfy these contraints we define 4 parameters:
  - t_1^{TX}: time the data is stable at the TX side before the active clock edge;
  - t_1^{RX}: time the data is stable at the RX side before the active clock edge;
  - t_2^{TX}: time the data is stable at the TX side after the active clock edge;
  - t_2^{RX}: time the data is stable at the RX side after the active clock edge.

  It must be that:

$$t_1^{RX} \geq t_s, \quad t_2^{RX} \geq t_h$$

- **Skew**
  Let's reintroduce skew. This means the delay is not the constant t_{prop} (unknown but not significant) anymore, but a varying:

$$t_d = t_{\text{prop}} \pm \frac{t_k}{2}$$

  Where t_k is our skew.
  With the phase delay between data and clock not maintained, delays are unpredictable and different in clock and data. Delays change from one bit transmission to the next, and the transmitter should consider the worst case scenario.
  For the setup time we have two sources of badness:

– Clock arrives early at RX after:

$$t_d^{ck} = t_{\text{prop}} - \frac{t_k}{2}$$

– Data arrives late at RX after:

$$t_d^{data} = t_{\text{prop}} + \frac{t_k}{2}$$

$$t_1^{RX} = t_1^{TX} + t_d^{ck} - t_d^{data} = t_1^{TX} - t_k$$

Since it should be that $t_1^{RX} > t_s$, it has to be that:

$$t_1^{TX} > t_s + t_k$$

Via similar hold time considerations (t_2) we get the frequency constraint:

$$f_{\text{ck}} < \frac{1}{t_s + t_h + 2t_k}$$

15 Lesson 28-10-26

Last lesson we talked about *serial* and *parallel* interconnects. We hinted at the fact that parallel is more performant (as we can transmit entire words per clock cycle).

15.0.1 Serial vs. parallel

The thing is that, performance of serial transmission is not affected by skew as the transmitted signal actually always shows a delay with some skew: but data and clock, being on the same wire, are affected in the same way. As long as the phase delay between data and clock is maintained, if the skew is significantly higher than setup and hold times, the clock frequency of a serial transmission can be significantly higher than the clock frequency of a parallel transmission. This means that serial can have higher performance than parallel when:

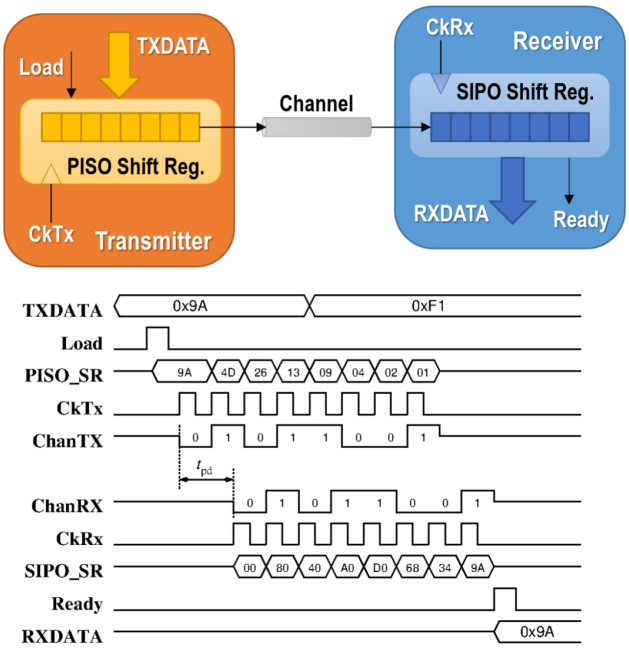
$$f_{\text{ckser}} > n \cdot f_{\text{ckpar}}$$

This means that today most high performance interconnections are *serial*, especially for long wires (high skew). Parallel busses survive outside ICs only for *memory interconnections*, which is high performance, and needs extreme care in design, and low performance communications with short interconnections.

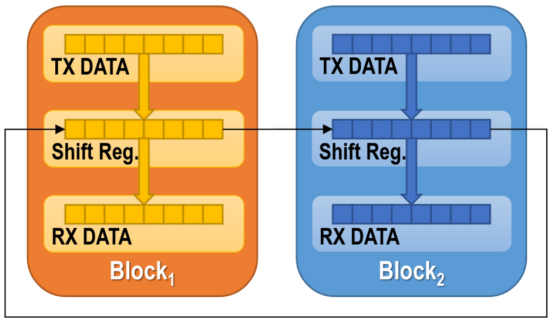
15.1 Basic serial transmission

Let's start looking at the structure of a serial **transmitter** sending to a serial **receiver**.

- The signal **transmitted** is a sequence of symbols, loaded into a register. Each symbol carries a single bit or multiple bits. Data at the transmitter is copied into a **PISO SR** (*Parallel Input, Serial Output Shift Register*), with the load signal activated.
- Data is then transferred from the transmitter to the **receiver** through a communication channel. The receiver has a **SIPO SR** (*Serial Input, Parallel Output Shift Register*). Th transmit and receive clocks are synchronized, so that they maintain a known phase relationship. When data at receiver is ready, a data valid signal is raised.



To enable full duplex transmission we may be tempted to have a transmitter and a receiver on each side, but we can get away with connecting the shift registers in a loop, such as:



The pipeline each block (receiver/transmitter) follows is as follows:

1. Data to be transferred is in the TXDATA register of each block, respectively, and data from TXDATA is transferred to the Shift Register. Blocks make this transfer without synchronizing.
2. Shift Registers of both blocks are clocked synchronously. Data is shifted bit by bit from one shift register to the other. After n clock cycles (for n bit shift registers), data has been transferred completely;
3. The contents of each Shift Register are transferred to the RXDATA register. Blocks make this last transfer without synchronizing.

15.1.1 Clock Sync.

Proper clock transmission allows **bit** synchronization, which guarantees correct sampling of single bits (when loading in the receiver shift register). This needs to synchronzie the clock. Higher protocol levels allow *character* (endianess, ...) and *message* synchronization message delimiters, ...).

The clock can be generated at the transmitter or receiver side, then sent to receiver or transmitter on a parallel line with respect to data. This is limited by skew, but at least (as we said) the skew is mostly the same between clock and data.

If sent by the receiver, the clock is also limited by transmission delay.

Clock can also be *embedded* (*Clock Data Recovery*, **CDR**): generated by transmitter, encoded in the data (presence of transitions), extracted by the receiver. This means data changes periodically (define maximum interval between transitions), and continuously (to allow synchronization).

Lastly, we talk about *isofrequential* clocks: in this case the transmitter and receiver have separate free running isofrequential clocks. Of course these need to be periodically synchronized.

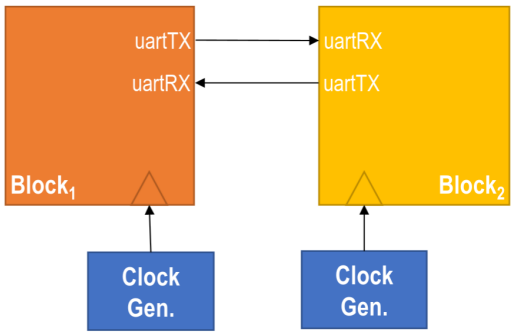
15.2 UART serial transmission

Let's now talk about a real example of serial communications, which is **UART** (*Universal Asynchronous Receiver Transmitter*).

UART consists of a point-to-Point serial communication, with:

- Isofrequential independent clocks;
- Two serial wires;
- One for transmission from Block1 to Block2.
- One for transmission from Block2 to Block1.

Ground is of course included but not shown.



As with most isofrequential systems, there's a periodic synchronization of the clocs. After the synchronization the clocks run free.

15.2.1 Character format

Transmission in UART is character based. Each transmitted bit lasts one clock cycle, and each transmission consists of a *frame*, which contains a character of n bits.

A frame has a defined format:

- START BIT (1 bit);
- Character (n bits, LSB first);
- PARITY BIT (1 bit, optional, might be absent);
- STOP BIT (1 or 2 bits).

Synchronization between clocks happens at the START BIT only.

We also need to talk about frame synchronization.

In the UART transmission format, an UART frame can start at any time. Each bit nominally lasts one clock cycle, so:

$$t_{\text{bit}} = \frac{1}{f_{\text{ck}}}$$

Character and parity bits are transmitted using standard NRZ encoding, so there are no guaranteed transitions. How do we guarantee a transition in each frame (so that the receiver can notice when a transmission starts)?

The UART line is kept at logic 1 when idle, and STOP BITs are always at logic 1. The START BIT is always a logic 0. This means that either after an idle phase, or following the previous frame, the START BIT of a new frame always causes a 1 to 0 transition.

This START BIT transition allows synchronizing clocks at the beginning of a frame. There is always at least a 0 to 1 transition (the first high data bit), as well, but its position is not known a priori, so it cannot be used for synchronization.

15.2.2 Transmission parameters

Transmitter and receiver should also agree on some parameters.

- **Baud rate** (symbols per second): it is the nominal clock frequency f_{ck} of the independent clocks. It corresponds to bps (bits per second) in this context, but symbols in general may represent more than one bit;
- **Number** n of bits in a character;
- Presence and kind of **parity** bits (as we said the parity bit can be not present);
- Number of **stop** bits (either 1 of 2).

Note that the configuration of these parameters is outside of the UART protocol.

15.2.3 Overhead

We saw that UART transmits some bits (START, STOP, eventual parity...) which are not directty useful by themselves. These are **overheads**. The minimum overhead is given by the STOP - START transmission on frame start, so 2 bits.

We have that the maximum transmission rate depends on both:

- **Baud Rate**: The higher the baud rate, the higher the performance. There are f_{ck} bit times in a second;
- **Overhead**: START BIT, Parity and STOP BITs do not carry information. AS we said, the minimum overhead is 2 bits (START BIT, 1 STOP BIT). The maximum overhead, on the other hand, is 4 bits (START BIT, Parity, 2 STOP BITs.

The equation for the ffective useful bit rate is:

$$\text{bit rate} = \frac{n}{n + o_h} f_{\text{ck}}$$

Increasing f_{ck} and n might increase performance, but there are limits: sampling is performed within a certain window where data is stable.

The window is between two active clock edges, where data potentially changes. The window is defined by a set of parameters:

- Voltage levels (V_{OL}, V_{IL}, V_{OH}, V_{IH});
- Noise (NML, NMH);

- Timing (setup t_{su}, hold t_h, rising t_r and falling t_f, slew rate, bandwidth).

Increasing the clock frequency makes the window smaller. In addition to this clock drifting moves the sampling point outside the window after a certain number of bits without synchronization. Once windows parameters are set, maximum n and f_{ck} can be computed

15.2.4 UART performance

Let's talk UART Performance.

For **long interconnections**, performance is limited by attenuation, distortion, noise. These affect the vertical axis in the *eye diagram* (see later).

Performance is also limited by unpredictable delays These affect the horizontal axis in the eye diagram (see later). It is improved by large voltage swings.

For **short interconnections**, erformance is limited by clock jitter and tolerances. These affect the horizontal axis in the eye diagram (see later).

Note that interconnection length and speed are conflicting parameters: we can have a short and fast link, or a long and slow link.

15.3 Eye diagrams

An **eye diagram** defines the sampling window via signal waveforms at the receiver. It is built considering all possible transitions between transmitted bits, and showing all transitions superimposed on the same graph.

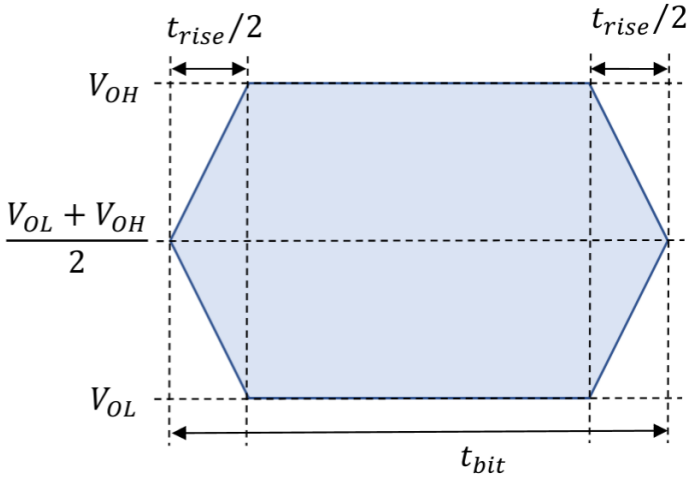
Let's draw an eye diagram ourselves. Let's start with a time interval equal to t_{bit} and a voltage range between V_{OL} and V_{OH} for the given technology, out of the typical 0 to V_{cc} range. Let's now assume for simplicity that $t_{\text{rise}} = t_{\text{fall}}$.

We start with ramps at the left side which start at $\frac{V_{OL}+V_{OH}}{2}$ and go toward V_{OL} and V_{OH} respectively, in time t_{rise} (the down ramps at the rights side will be mirrors of this if $t_{\text{fall}} = t_{\text{rise}}$). For large signals, we can use the **slew rate** S_R:

$$t_{\text{rise}} = \frac{V_{OH} - V_{OL}}{S_R}$$

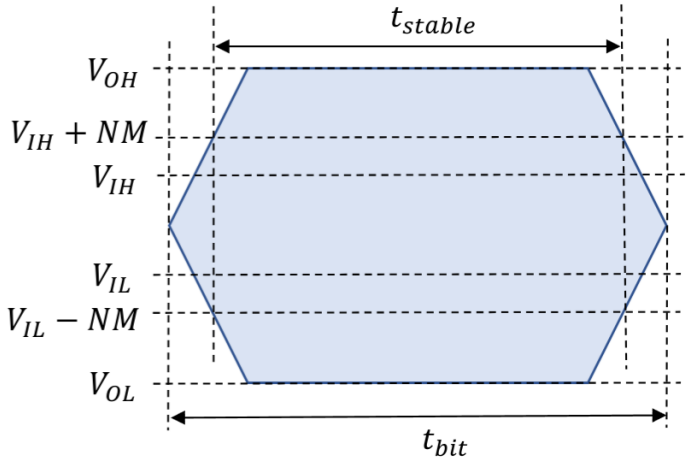
We then do the same thing on the right side, ensuring the total time away from $\frac{V_{OL}+V_{OH}}{2}$ is t_{bit}.

The result is shown below.



Let's note the noise margins in the graph and see where, in both sides (up and down) the signal is considered stable.

- We identify the value as high for a time t_{stable} if the graph is above $V_{IH} + NM$.
- We identify the value as low for a time t_{stable} if the graph is below $v_{IL} - NM$.

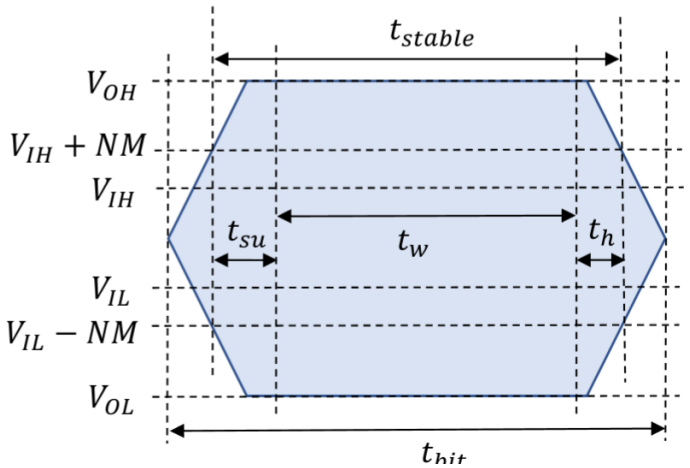


This is still not enough, as the value has to be stable for enough time for the register to setup (t_su) and to hold (t_h). This gives us the actual sampling window t_w, where we always have:

$$t_w < t_{\text{stable}} < t_{\text{bit}}$$

Sampling inside the sampling window t_w is always correct as:

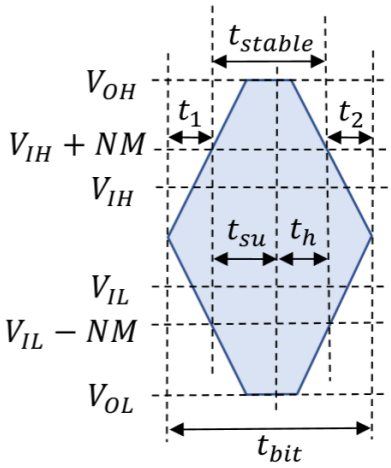
- The signal is stable;
- The signal has proper voltage levels;
- Setup and hold time are satisfied.



Now, immediately after synchronization, the receiver determines the best sampling instant. We assume it is at the center of the sampling window. The main isse is that clock drift eventually might get the sampling instant outside the sampling window.

This means we have 2 limitations:

- *Increasing f_{ck} decreases the t_w window, making the clock drift out faster.*
  With perfect isofrequential clocks, we synchronize clocks on START BIT and always assume sampling happens within the sampling window for all successive bits. So there is no limit to the number n of consecutive bits, but here is still a limit to the maximum clock frequency.
  With real rising and falling edge, we compute the window t_w as:



$$t_1 = \left(\frac{V_{IH} + NM}{V_{OH} - V_{OL}} - \frac{1}{2}\frac{V_{OH} + V_{OL}}{V_{OH} - V_{OL}}\right) t_{\text{rise}}, \quad t_2 = t_1$$

$$f_{\text{ck}} = \frac{1}{t_{su} + t_h + t_1 + t_2} = \frac{1}{t_{su} + t_h + 2t_1}$$

  The maximum clock frequency limits the maximum performance.
- *Increasing n makes the clock to drift out towards the end of the transmission.*
  Let's assume nominally isofrequential clocks with tolerances. This means there's a relative error ε (in percentage) from the nominal clock frequency at the receiver:

$$\begin{cases} f_{\text{ck}} &= f_{\text{nominal}} \pm \delta_f \\ t_{\text{bit}} &= t_{\text{nominal}} \pm \delta_t \end{cases} \rightarrow \begin{cases} f_{\text{ck}} &= f_{\text{nominal}} \pm \varepsilon_f \% \\ t_{\text{bit}} &= t_{\text{nominal}} \pm \varepsilon_t \% \end{cases}$$

  With relative errors:

$$\varepsilon_f = \frac{\delta_f}{f_{\text{nominal}}}, \quad \varepsilon_t = \frac{\delta_t}{t_{\text{nominal}}}$$

  Because of this relative error, the sampling point from one bit to another differs by an amount:

$$\Delta t = t_{\text{bit}}^{RX} - t_{\text{bit}}^{TX}$$

  In the worst case:

$$\Delta t = t_{\text{nominal}} \cdot 2\varepsilon$$

  Considering equal tolerances at transmitter and receiver, but in opposite directions.
  As we were saying, drift may cause sampling outside the sampling window after a certain number of consecutive bits without re-synchronization.

In the eye diagram there is one degree of freedom: once we determine (estimate or measure) all parameters, we keep the clock frequency f_{ck} and the number n of consecutive bits unknown. Fixed the clock frequency and its tolerance, we compute the maximum number of consecutive bits: t_{bit} is known and equal to t_{nominal} when drawing the eye diagram.

We can also do the reverse: fix the number of consecutive bits and the clock tolerance, compute the maximum clock frequency. This leaves t_{bit} as an unknown when drawing the eye diagram.

16 Lesson 29-10-26

16.1 Synchronous protocols

In the last lesson we saw serial protocols, more specifically UART. UART uses separate clocks at RX and TX, with synchronization on the START bit.

We know want to see protocols, either character or bit oriented, where clock recovery uses transitions of the data signal, in each bit (synchronous carrier), so periodically.

First some constraints: The *channel bandwidth* requirement is that it contains at least the highest fundamental frequency in the transmitted signal. The lower, the better, given a certain Bit Rate f_{ck}, as it makes for a cheaper channel. This means we need to find the worst case depending on the bits transmitted.

For the number of *voltage levels* used in the channel, we stick with standard logic which uses 2: $0\,\text{V}$ and V_{dd}.

16.1.1 NRZ encoding

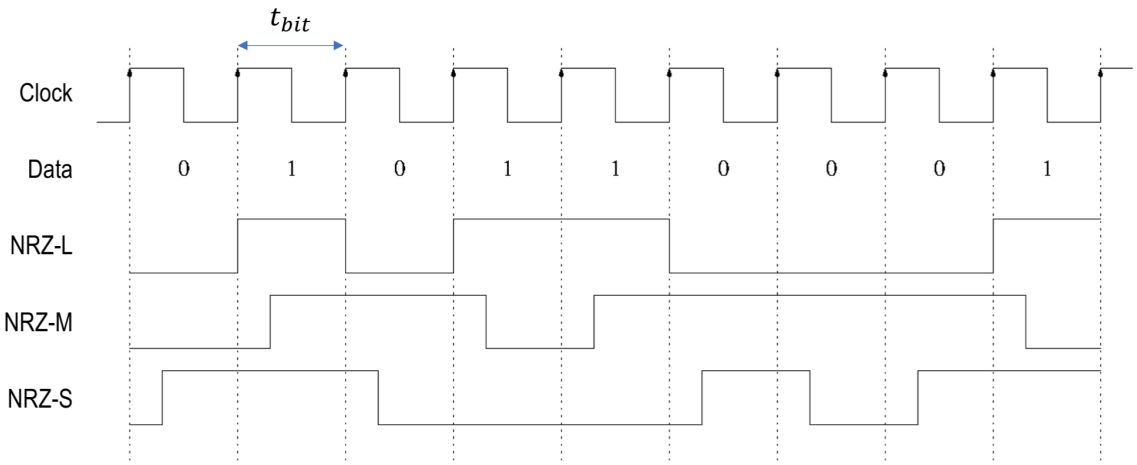
A first approach is **NRZ** (*Non Return to Zero Encoding*). Several types:

- **NRZ-L** (*Level*):
  - Logic 1: high state
  - Logic 0: low state
- **NRZ-M** (*Mark*, also known as **NRZ-I**, *Inverted*):
  - Logic 1: represented by a transition
  - Logic 0: no change
- **NRZ-S** (*Space*, also known as **NRZ-I**, *Inverted*):
  - Logic 1: no change
  - Logic 0: represented by a transition

Note that in none of these transitions are *guaranteed*! We might not have them, which prevents us from synchronizing on that clock cycle.

Also, since at most we change value once per clock, the maximum frequency for NRZ is:

$$f_{\text{max}} = \frac{f_{\text{ck}}}{2}$$



16.1.2 RZ encoding

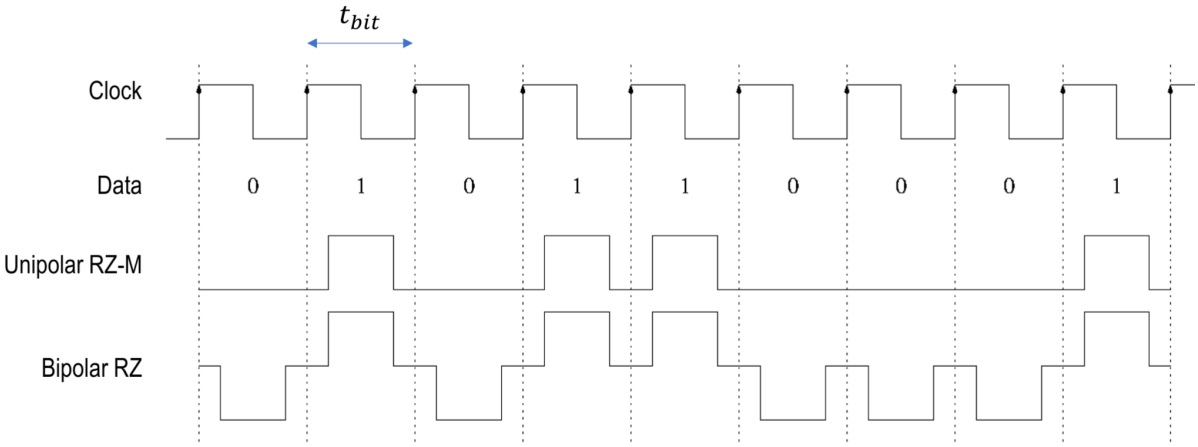
The alternative is **RZ** (*Return to Zero Encoding*). Several types of this too:

- Unipolar **RZ-M**
  - Logic 1: high pulse
  - Logic 0: low state
- Bipolar **RZ**
  - Logic 1: positive pulse
  - Logic 0: negative pulse

Transitions are not guaranteed in unipolar, but they are in bipolar (good!), tough it needs a ternary code with better SNR.

For the bandwidth, we can change the value two times per clock, so the maximum frequency for both RZ types is:

$$f_{\max} = f_{\text{ck}}$$



An alternative that exists is **AMI** RZ (*Alternative Mark Inverted Return to Zero*). This involves marking:

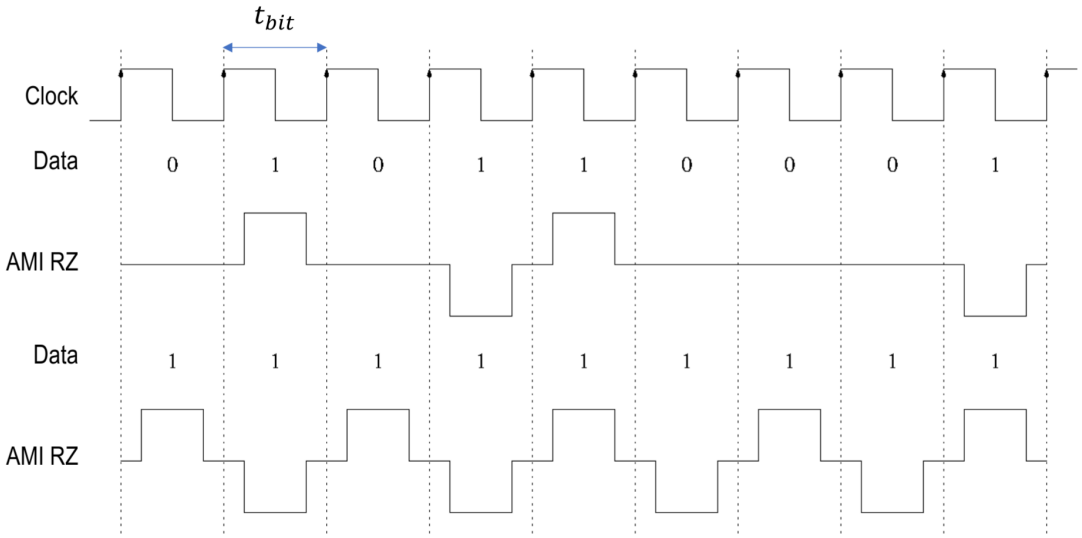
- Logic 1: alternate level pulse
- Logic 0: no pulse

Where, by alternate level pulse we mean a pulse opposite of the previous one. We consider this because we find that the maximum frequency is in fact reduced:

$$f_{\max} = \frac{f_{\text{ck}}}{2}$$

This is because the worst case scenario, where we transmit a high value at each cycle, gets us basically a sinusoidal wave at frequency $\frac{f_{\text{ck}}}{2}$. Also, the DC component is absent on average, which is an advantage in long connections.

The tradeoff is that this clearly doesn't guarantee transitions.



16.1.3 DICODE and DUOBIN encoding

These are based on converting input transitions (in the data to transmit) to output pulses.

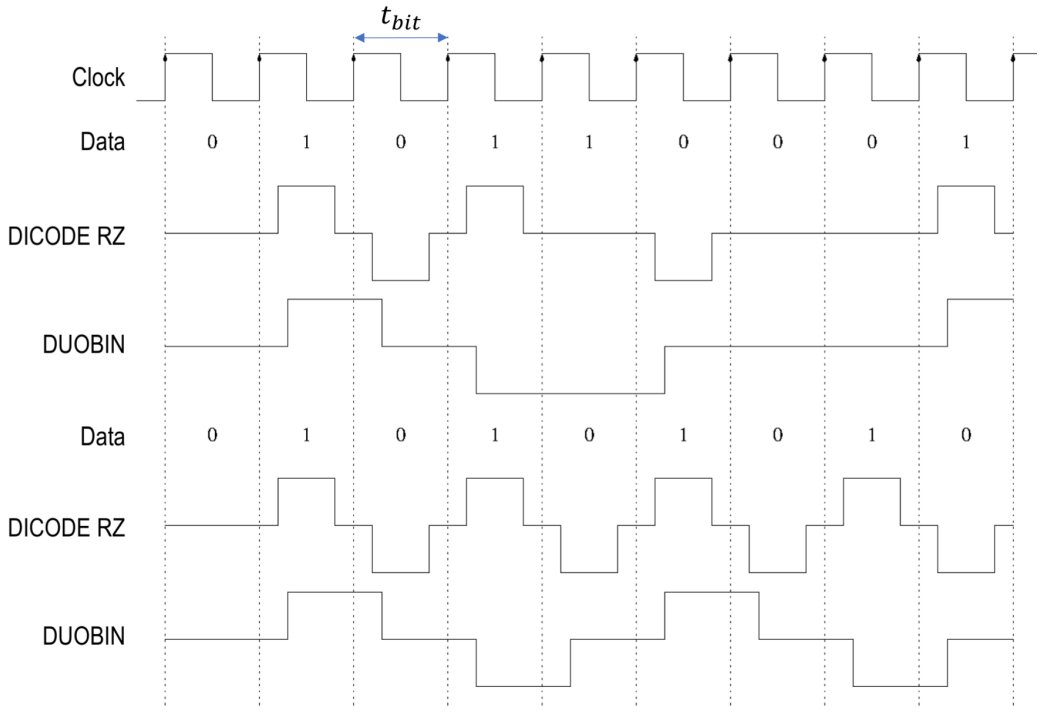
- **DICODE RZ**: each input transition generates an output pulse, with alternate polarity. If there is no input transition, the output remains stable;
- **DUOBIN** (*DICODE NRZ*): each input transition generates an output transition, with alternate polarity. If there is no input transition, the output remains stable.

These clearly don't guarantee transistions, but are really efficient in terms of bandwidth as:

$$f_{\max} = \frac{f_{\text{ck}}}{2} \quad \text{(DICODE)}$$

$$f_{\max} = \frac{f_{\text{ck}}}{4} \quad \text{(DUOBIN)}$$

in the best case.



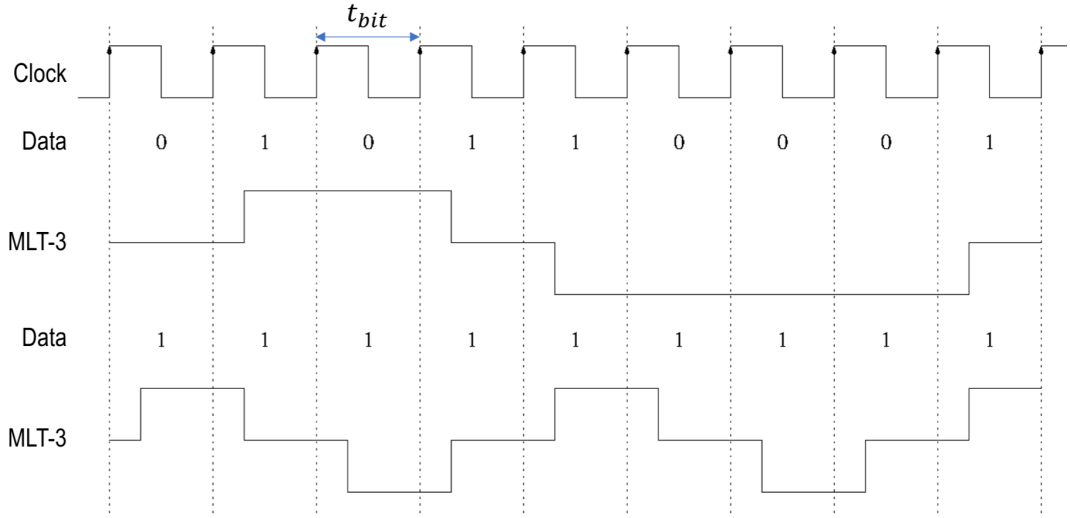
16.1.4 MLT encoding

This is significant because it used in Ethernet 100Base-TX (I would guess one of the thicknet standards). For instance, MLT-3 has three logic levels:

- Logic 0: no change
- Logic 1: output transition with alternate polarity

The bandiwdth is good as we get:

$$f_{\max} = \frac{f_{\text{ck}}}{4}$$



16.1.5 Manchester codes

Also known as IEEE 802.3, **manchester codes** encode data via only transitions. They can be obtained from NRZ-L (see 16.1.1) by XOR'ing with the clock.

- Logic 1: low to high transition (L $\rightarrow$ H)
- Logic 0: high to low transition (H $\rightarrow$ L)

For the opposite encoding:

- Logic 1: high to low transition (H $\rightarrow$ L)
- Logic 0: low to high transition (L $\rightarrow$ H)

Transitions are guaranteed (which is good), but bandwidth of course suffers:

$$f_{\text{max}} \leq f_{\text{ck}}$$

There are some variations.

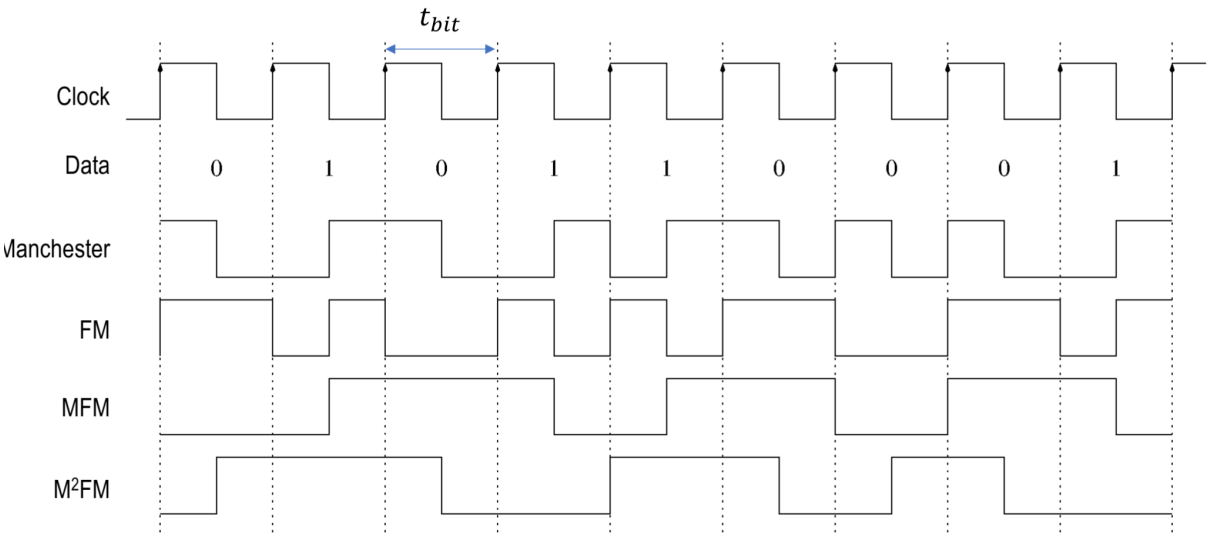
- First off, **FM** (*Frequency Modulation*). Also known as BiPhase or Differential Manchester Encoding:
  - Logic 1: a transition at the beginning of the bit, followed by a transition in the middle of the bit
  - Logic 0: a transition at the beginning of the bit
- **MFM** is *Modified Frequency Modulation*. The idea is to have different transition periods based on the value transmitted:
  - Logic 1: a transition in the middle of the bit
  - Logic 0: no transition if preceded by a 1, a transition at the beginning if preceded by a 0
- Lastly we have **M2MF**, which is somehow named *Modified Modified Frequency Modulation*. In particular, a clock pulse is only inserted between a pair of adjacent 0-bits if the first bit of the pair did not have a clock pulse inserted before it.

  This is wildly non standard as it seems the professor's slides interpret it differently (?).

All of these guarantees transitions.

For bandiwdth, as we said, manchester is f_{ck}. FM is also f_{ck}, whereas MFM and M2FM are:

$$f_{\text{max}} = \frac{f_{\text{ck}}}{2}$$



16.1.6 Forcing transitions

Some encodings show excellent characteristics, such as simplicity, low bandwidth, two voltage levels, but do not guarantee transitions.

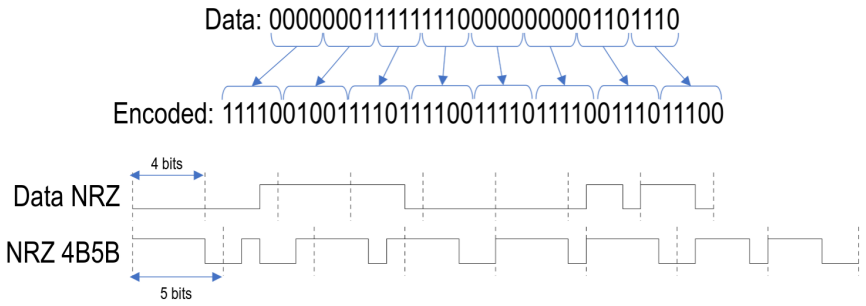
We can solve this by forcing transitions into our data before encoding:

- **xByB**: every x-bit block is replaced by a y-bit block, with $y > x$. Because of how the y blocks are constructed this imposes the presence of transitions, is deterministic, but imposes a fixed overhead.

  Redundancy is used for transition insertions, but not at every bit: instead, they are guaranteed every n bits. xByB can also be used for DC and low frequencies removal.

4B	5B
0000	11110
0001	01001
0010	10100
0011	10101
0100	01010
0101	01011
0110	01110
0111	01111

4B	5B
1000	10010
1001	10011
1010	10110
1011	10111
1100	11010
1101	11011
1110	11100
1111	11101



There are a number of xByB codes, such as 3B4B, 4B5B, ecc... with different characteristics. They can all easily be implemented via LUTs.

- **Bit stuffing** involves, every n bits that don't cause a transition, to insert a bit that causes a transition.

  The value of n is normally between 5 and 8, as the number of acceptable consecutive bits without a transition determined using the eye diagram. It differs depending on the protocol, with transmitter and receiver clocks re-synchronized at transitions.

  Of course at the receiver we have to notice if a bit that causes a transition is an actual data bit or a bit inserted via bit stuffing. This can be done simply by noticing the maximum number of hi (or low) bits was encountered, and the next bit as to be a 0 (or 1) of bitstuffing. If it isn't, it's likely an error or some kind of flag.

  The main disadvantage of bit-stuffing is that the code rate is unpredictable; it depends on the data being transmitted.

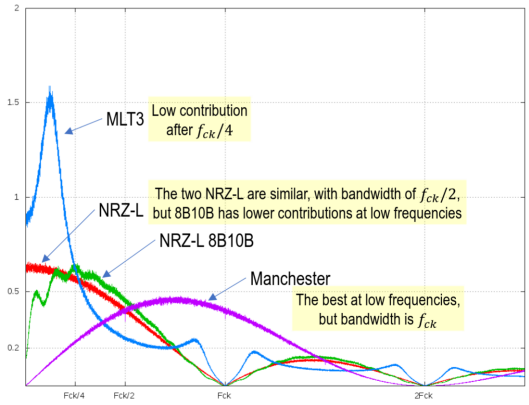
17 Lesson 30-10-26

17.1 Spectrum of serial protocols

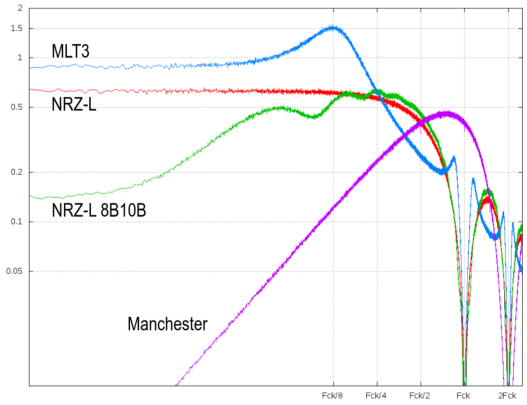
Let's look at some *waveform* and *spectrum* examples for serial communications protocols.

DC and low frequency components may be hard to transmit on most channels, especially if there are galvanic or optical transformers in the transmission chain. Some encodings guarantee the absence of DC and low frequency components, such as *Manchester* and *8B10B*, which guarantees an equal number of 0s and 1s in the encoding in the long run (we might have said something about this in the past lesson).

The sepctra of serial data line waveforms shows a superimposed sinc trend (Fourier transform of a box function), which is zero at f_{ck} and multiples. Regarding the **bandwidth**, this means there are still major contributions to the signal above the bandwidth if taken at f_{ck}.



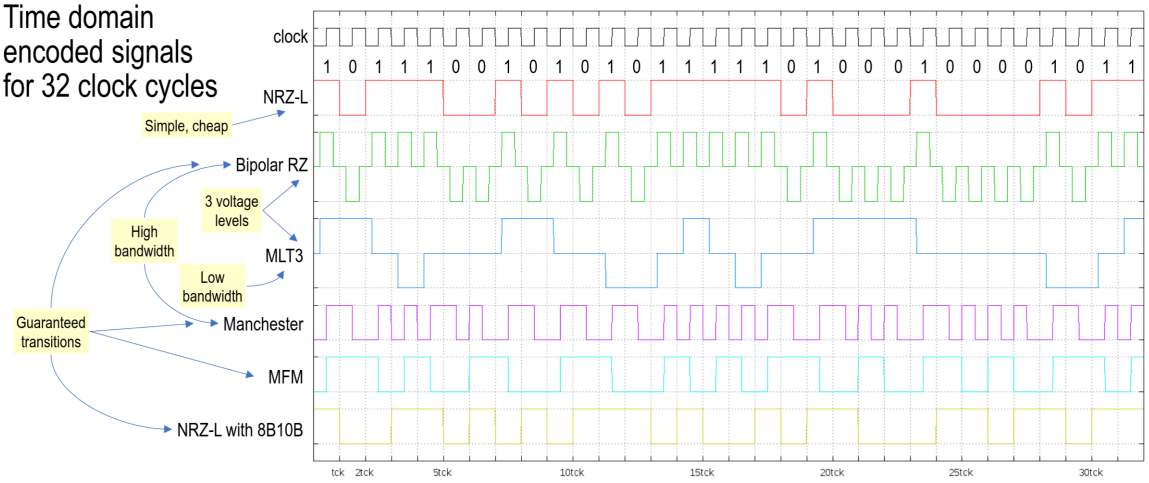
Linear scales



Logarithmic scales (similar to Bode)

Waveform and spectrum examples

- Time domain encoded signals for 32 clock cycles



17.2 SPI and I2C

These are standards used to connect integrated circuits, on the same board, or on separate boards (over short distances), organized as *networks* of nodes.

Master and **slave** devices are identified: they have different roles, but they can both be receiver and/or transmitter.

They are of course *serial*, but usually offer a dedicated clock line. The separate wire for clock transmission is a skew problem, as we cannot achieve very high performance (from hundreds of Kbit/s to a few Mbit/s), and the length of the interconnection is limited. The fact that multiple nodes can drive the single serial line, on the other hand, save costs and allows to have less wires.

17.3 SPI

SPI (*Serial Peripheral Interconnect*), also known as the 3 wires protocol, is a serial transmission protocol supporting:

- Single master, multiple slaves, where the number of slaves is in principle unlimited (in practice usually limited to 4 or 5);
- It allows for **full duplex**, with one wire for data from master to slaves (**MOSI**, *Master Out Slave In*), and one wire for data from slaves to master (**MISO**, *Master In Slave Out*).

  Master and slaves are always both RX and TX.

  So, we need at least 3 wires, but often we find 4 or more:

- **SPICLK**, the clock, always generated by the master;
- **MOSI**, Master Out Slave In, the output data line of the master, and input data line of slaves;
- **MISO**, Master In Slave Out, the input data line of the master, and output data line of slaves. It can have multiple drivers, so it needs to be tristated, and there are risk of dangerous conflicts;
- **SPISS**, not strictly necessary, it's the *SPI Slave Select*. It is connected to the *Chip Select* input of slaves, and driven by the master.

There might be multiple SPISS when connecting to several slaves, or it might be absent when the master is connected to a single slave. The Chip Select of a slave is always active (i.e. connected to GND), and this overrides it.

If it's the master that orchestrates slave communications, then there is no (or at least less) risk of two slaves deciding to talk at the same time. This terminology is gross.

On data transmission, a predetermined number n of bits is transmitted. All blocks in the SPI network have internal shift registers of size n to allow this, and one bit per clock cycle is transmitted. *NRZ encoding* is used, as its simple and cheap. No transition is guaranteed, but this is fine as the separate clock provides the required synchronization. SPICLK is idle when there is no transmission, at a constant level defined by the *clock polarity* **CPOL**.

Let's look at a transmission

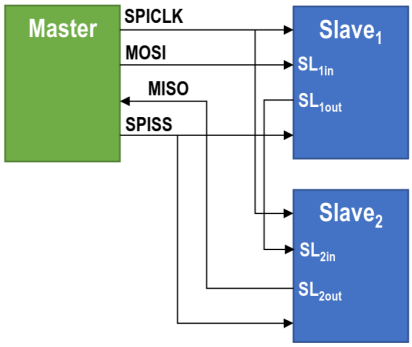
1. The Slave Select of the target slave is activated during the entire transmission, and on all other slaves the SS is not active. These keep their output (on MISO) in high impedance, and ignore what happens on MOSI and SPICLK.
2. The master generates n clock cycles: the first clock transition denotes the beginning of a transmission. Data is transmitted MSB first, changes on a clock edge and is sampled on the opposite clock edge, according to the *clock phase* **CPHA**.
3. The clock returns idle at the end of the transmission;
4. Slave Select is deasserted to complete the transmission.

Notice that CPOL and CPHA define three parameters:

- Idle clock level
- Active edge for data output
- Active edge for data sampling

CPOL	CPHA	Idle CLK	Data output	Sampling
0	0	0	Falling	Rising
0	1	0	Rising	Falling
1	0	1	Rising	Falling
1	1	1	Falling	Rising

SPI supports **daisy chaining**. SPISS not strictly needed, and there is no shared MISO: all slaves get the clock, but only one gets MISO. The other slaves talk one to another, and only the slave which gets the MISO talks directly to the master. Again, gross.



Note that every transmission involves all slaves in this configuration, as values have to propagate through slaves to the slave connected to MISO, and thus the master.

To summarize SPI:

- **Arbitration** is not needed, as there is only a single master, and slaves cannot initiate a transmission. All arbitration is done by the master;
- **Addressing**: the target slave is chosen using Chip Select signals,o ne separate signal for each slave, expensive and not easy to change after fabrication. In daisy chain connection all slaves are always involved;
- **Bit synchronization**: clock always generated by the master, configurable polarity and active edges. A slow slave cannot stop the master.

18 Lesson 03-11-26

18.1 I2C

We know move from *SPI* to **I2C** (*Inter Integrated Circuit*, also known as IIC).

I2C allos for multiple masters, multiple slaves, where:

- Only masters can initiate a transfer;
- Multiple masters require arbitration.

It also requires only 2 wires:

- **SCLK** for the clock;
- **SDATA** for data: a single data line, for half duplex communication.

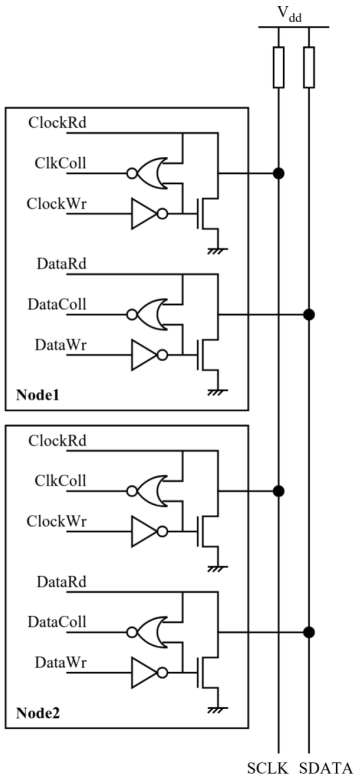
As always, the 2 wire solution (SPI was 3, but still separate clock) introduces distance limits due to skew. Also, no select signals for slaves means the protocol requires addressing: this introduces a certain level of *overhead* in the system.

All devices can read and write the SDATA and SCLK lines:

- The SCLK line mainly controlled by masters, slaves can only stretch the clock duration if they are slow (cooperative clock).
- The SDATA line controlled by a single transmitter (master or slave) at any given time

The protocol should avoid collisions: an *Open Drain* (pull-up) architecture is used for both SCLK and SDATA. If lines are:

- Logic 0: the line is driven to GND by the low side output transistor of a node;
- Logic 1: the low side transistor is OFF, the line is left floating. An external pull-up resistor brings the line to the high voltage.



This means that when I2C is idle, both lines are normally high.

The architecture allows for detecting collisions on SCLK and SDATA.
A collision happens when a node generates 0, another generates 1.

- The output nMOS transistor of the node generating 0 is ON.
- The output nMOS transistor of the node generating 1 is OFF.

There is no short circuit, no damages to the system, and the line just goes to 0 (the nMOS outputting 0 wins).

All nodes constantly monitor the SCLK and SDATA lines:

- The device generating the 1 detects the collision;
- The device generating the 0 does not notice.

Two kinds of collisions, actually features:

- On the SCLK line we have *clock stretching*;
- On the SDATA line we have *arbitration*.

18.1.1 I2C addressing

Each slave in an I2C network is assigned a unique **address** on 7 bits. This means we have a maximum of 128 slaves, but typically much less. Nonetheless, there exists a protocol extension to 10-bit addresses.

Some addresses are reserved, and masters do not need an address.

The address is often hardwired in a slave by connecting pins to V_{dd} or ground. Some bits are not configurable (usually the MSBs) to save on the number of pins on the slave. The master sends the address of the target slave at the beginning of a transmission, to address it.

18.1.2 I2C protocol

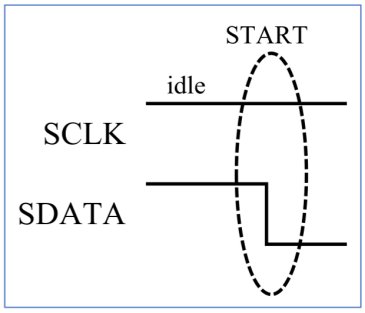
SCLK and SDATA lines are normally high when there is no transmission: this just means that all low side nMOS output transistors are OFF.

The main rule is that the SDATA line is allowed to change only when SCLK is low. Two exceptions mark frames:

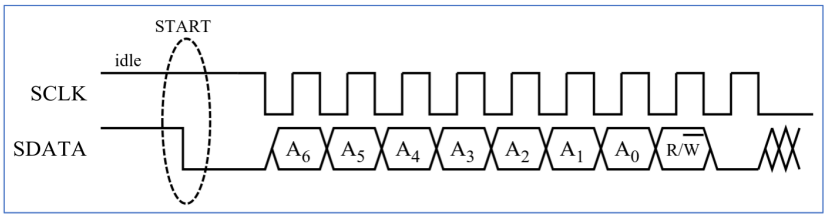
- **START** configuration: SDATA makes a high to low transition while SCLK is high;
- **STOP** configuration: SDATA make a low to high transition while SCLK is high.

Now:

1. All communications begin with a START configuration;

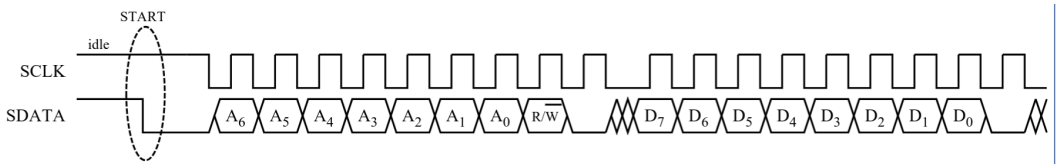


2. An addressing phase follows: the master activates the clock and generates on SDATA the target slave address on 7 bits, MSB first, one bit per clock cycle.
   One more bit is needed for the R/W flag (R when high). Then we get one low bit from the slave before releasing the SDATA line (the ACK).
   SDATA usually changes only when SCLK is low, shortly after the falling edge of SCLK. Slaves samples SDATA immediately after the rising edge of SCLK.

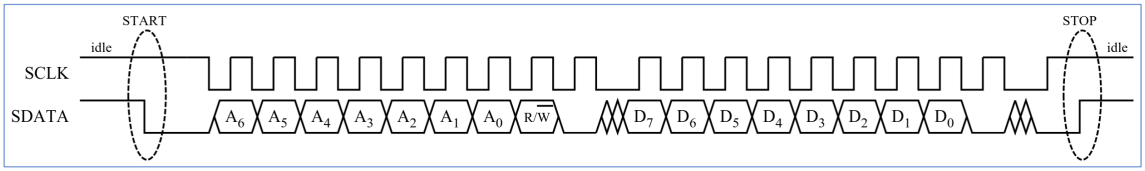


3. A data exchange phase in a single direction is next. We said that the master generates 9 clock cycles: 8 bits for data, 1 bit for ACK.
   Now the situation changes depending on the type of operation the master initiates:
   - *Write*:
     - The master regains control of SDATA and generates 8 data bits, MSB first;
     - The slave generates ACK on the last bit (SDATA=0).
   - *Read*:
     - The slave keeps control of SDATA and generates 8 data bits, MSB first;
     - The master generates ACK on the last bit (SDATA=0).

   This way, only a byte is transferred. Multiple bytes can be transferred one after the other, with no limit;

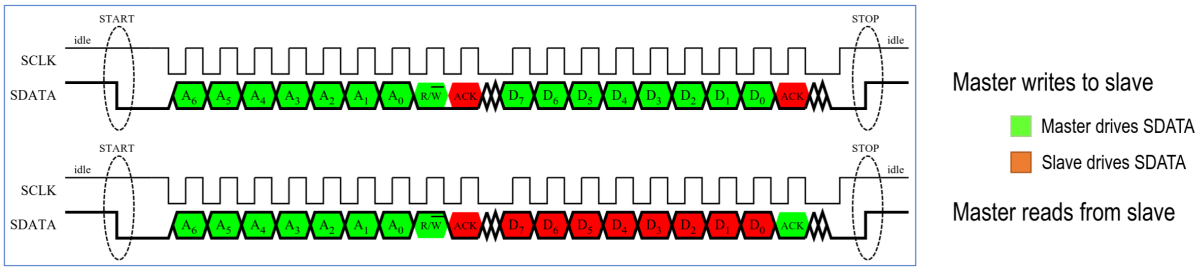


4. A STOP configuration concludes the communication.
   After the last data byte, the master (or the slave, on writes) keeps SDATA low (the ACK).
   Then, the master itself keeps SDATA low while releasing the SCLK: SCLK goes high. When the master then releases SDATA as well, SDATA makes a low to high transition, which signals the STOP configuration.



The bus is now idle again. Hooray.

Lastly, let's note the difference between a read and a write operation performed by the master:



Some other rules:

- The first 8 bits are driven by the transmitter (TX), always a master for the addressing phase, a master or a slave in the data phase;
- The ACK bit is driven by the receiver (RX), always a slave for the addressing phase, a master or a slave in the data phase;
- Before and after ACK the SDATA line may show oscillations while control passes from TX to RX and viceversa. Oscillations only happen while SCLK is low, and SDATA must become stable before the rising edge of SCLK;
- An ACK after the addressing phase means the slave is ready for transmission. If the ACK is missing (i.e. SDATA=1), the transmission is immediately aborted;
- An ACK after the data phase, writing to slave means the slave received the data. It doesn't mean it was received correctly, as there is no error detection on bits. If the ACK is missing, the data definitely was not received correctly;
- An ACK after the data phase, reading from slave means the master is ready to receive an additional byte. If the ACK is missing, no more bytes are requested by the master.

The opposite of an ACK (leaving the SDATA rail up) is called a NACK.

A **RESET** configuration is needed as sometimes a master needs to write a command to a slave, and then immediately read an answer.

This normally requires two transmissions, the first in writing, the second in reading. If the master doesn't want other masters to gain control of the bus before the second (reading) transmission, at the end of the first (writing) transmission, it can:

- Not generate a STOP configuration;
- Generate a START configuration, again.

This means that the master sets SDATA high, releases SCLK, sets SDATA low, which signals the RESTART configuration. The bus never becomes idle, thus no other master can gain control.

18.1.3 Clock stretching

Let's have a look at the first case in which we use collision detection. We focus on collision detection on the SCLOCK line, that is **clock stretching**.

Normally the clock on SCLK is generated by a master, for all bits: a slave should be sufficiently fast to operate at the clock frequency generated by the master. In this case, the slave only receives the clock and never writes it.

A slow slave may extend the time at which the clock is low, simply by activating its own nMOS transistor on the SCLK line, while it is already forced low by the master. When the master releases SCLK, the line remains low: the master detects the collision and waits for the SCLK line to be released by the slave, as well.

The communication then proceeds as usual.

If clock stretching is used, individual bits may have different bit time. Note that not all masters and slaves support clock stretching.

18.1.4 Arbitration

This is the second mechanism in which collision detection is used. We focus on collision detection on the SDATA line, that is **arbitration**.

Multiple master may wish to start a transmission at the same time. If the bus is busy, a master shall not start a transmission. If two masters Master1 and Master2 (or more) concurrently generate a START configuration, they won't notice it, as they are both driving the same line low: each master believes he is the only owner of the bus!

We assume in the following time that they generate the same clock, and during the addressing phase, they (probably) generate two different addresses. If at a certain bit Master1 generates a 0 and Master2 generates a 1, that address line goes low: Master1 does not notice any problem, Master2 detects a collision on SDATA. At this point Master2 has lost the contention and stops the transmission, so that Master1 can complete the transmission.

19 Lesson 04-11-26

19.1 CAN

CAN (*Controller Area Network*) is a protocol aimed to increase communication rate, and increase interconnect length.

Defined by Bosch in 1980, it supports a *multi-node* topology, with multiple blocks on the same bus, and all blocks acting as masters (one-to-many with node arbitration).

It was developed for automotive applications, but also used in other contexts. It's designed for noisy environments.

The protocols defines several aspects of the communication:

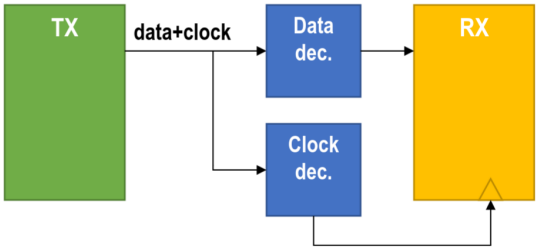
- **Physical** (electrical) level
- **Transaction** (frame format) level
- **Software** (application) level

It is a serial communication with an embedded clock, which means it has a high clock frequency, not affected by skew (though it isn't suited to short interconnections).

Of course, it requires special encodings and circuits, such as clock decoders at the receiver, ro extract the clock information and the data stream.

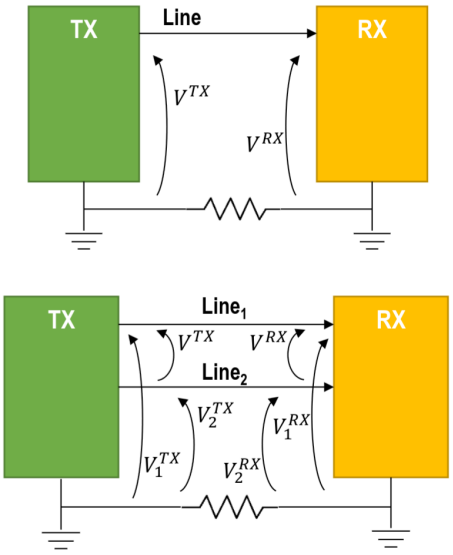
As for increasing interconnection length, skew is very high for long interconnections, so we use a serial communication with embedded clock.

We also use *differential signaling* to reduce the effects of noise.



19.1.1 Differential signaling

Now, in a real communication system, if we transmit signals via 2 wires (data and ground), we never get exactly the same voltage at RX and TX $V^{TX} \neq V^{RX}$. This is because of ground differences, making the ground not a good reference for the data line (which suffers from interference of its own).



What we do is encode information as the voltage difference between two wires. Assuming V_{CM} is equal to the difference between the two grounds, and interference carried

on both the data wires (*Common Mode Noise*, **CMD**), we have:

$$V_1^{RX} = V_1^{TX} + V_{CM}, \quad V_2^{RX} = V_2^{\text{ő}X} + V_{CM}$$

And we can recover, via operational amplifiers at both ends:

$$V^{RX} = V_1^{RX} - V_2^{RX} = V_1^{TX} + V_{CM} - V_2^{TX} - V_{CM} = V_1^{TX} - V_2^{TX} = V^{TX}$$

This both RX and TX see the same voltage, whatever the ground differences, losses, and other interference.

Let's focus on what we mean by interference: any other unwanted interference that equally affects both wires induces *common mode noise*, which is cancelled by differential signaling.

This means we don't want wires affected by different noise (*differential mode nosie*), that is noise source close to one wire, for instance.

To aid in this, we *twist* the pair of wires so that the interference alternatively affects one wire and the other. Assuming the interference is uniform along multiple twists, the noise becomes common mode, and is cancelled by differential signaling. We can also *shield* the pair to reject interferences.

This gives a reason behind the wires we are used to:

- **STP** (*Shielded Twisted Pair*);
- **UTP** (*Unshielded Twisted Pair*).

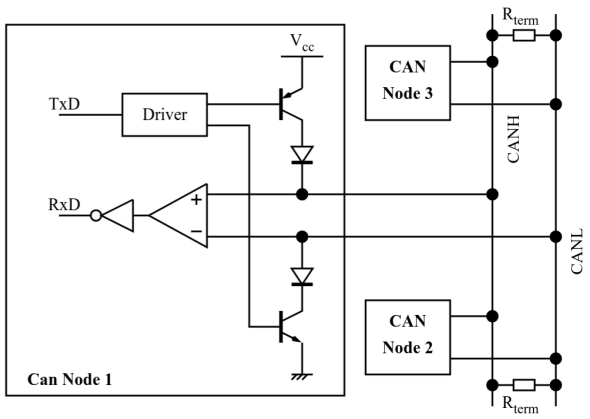


So, we have that the CAN protocol does differential signaling on two wires: **CANH** and **CANL**.

The two wires are used for data transfer, not one for data and the other for clock, like in I2C: there is no dedicated wire for the clock, so we don't suffer the problem of skew.

Data is in fact encoded as the difference CANH - CANL, which allows for long interconnections through embedded clock encoding. The clock is recovered from data.

Let's look at the CAN bus itself.

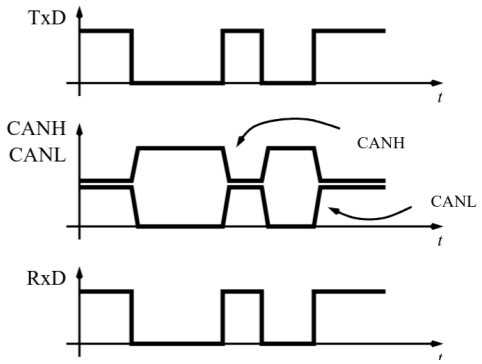


We notice open collector driving of CANH and CANL, driving on the high side (through a PNP) for CANH, and on the low side (through an NPN) for CANL. Reading, as we said, is done through a differential amplifier $V_{out} = CANH - CANL$, with an invertd output.

We have a terminating resistances between CANH and CANL, with value $120\,\Omega$, meaning the resistance between the two CAN lines is $60\,\Omega$. This resistance is used to equalize the two wires voltage values when floating, and used to avoid reflections on the transmission line, especially for long interconnections.

If each side of the can bus can only be driven in one direction (through open collector signaling), we wonder how logic levels are represented. To solve this, we use a system that distinguishes floating (*recessive*) values from driven ones:

- **Logic 0** (*dominant*): $CANH = 5\,\text{V}$, $CANL = 0\,\text{V}$, difference in reading is high, output of the input amplifier is low;
- **Logic 1** (*recessive*): $CANH = HiZ$, $CANL = HiZ$, difference in reading is 0 (because of the equalizing resistance), output of the input amplifier is high.



Let's talk about bus contention: all nodes have the same interface on the CAN bus. When not transmitting, each node generates a recessive bit, so a high value on receiver, and CANH and CANL are floating. A dominant bit during transmission always wins over a recessive bit, as it means some node is driving the lines. Since each node drives the lines to the same values on a dominant bit (open collector driving), in case of bus contention there are no damages or short circuits.

Since we said that NRZ encoding is used, with no guaranteed transitions we do bit stuffing after 5 consecutive identical bits:

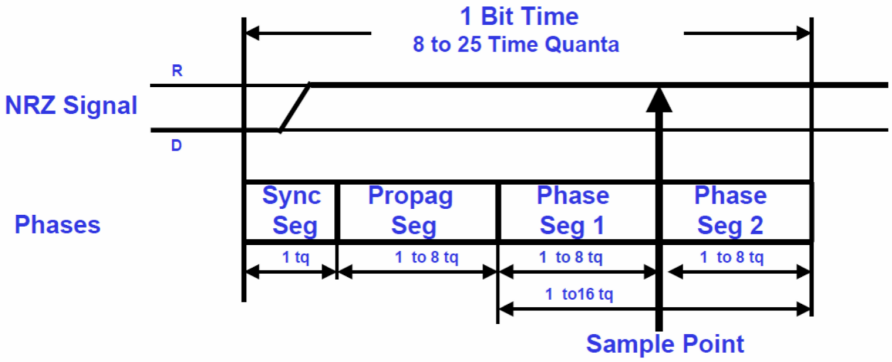
- After 5 dominant (0), insert a recessive (1);
- After 5 recessive (1), insert a dominant (0).

If 5 consecutive identical bits are followed by the opposite value, bit stuffing is applied anyway, so that the receiver can delete stuffed bits when decoding the incoming bitstream.

The eye diagram sees the bit time divided into 4 time quanta (8 to 25):

- Synch segment
- Propagation segment
- Phase segment 1
- Phase segment 2

Sampling is specified to happen between Phase1 and Phase2 segments.

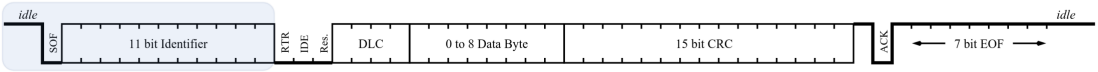


19.1.2 I2C protocol

Enough electronics. Let's talk about the protocol.

Three main problems:

- **Synchronization**: all devices have to be synchronized at each recessive to dominant transition, with transitions assumed to happen at the beginning of a bit time. Each device momentarily adjusts Phase1 and Phase2 lengths to synch. All devices start in low speed, and speed can be modified later at run-time;
- **Addressing**: when transmitting, each node generates an 11 bit address, known as identifier in CAN. It is *not* the address of the destination node;
- **Arbitration**: similar to I2C, the lower identifier wins contention (dominant vs. recessive), as lower identifier means higher priority.



Let's see the full transmission in pratice.

1. At idle, we see a recessive bit on the bus;
2. *Start Of Frame* (**SOF**) is a dominant bit generated by a node. It guarantees a recessive to dominant transition from the idle state. All other nodes synchronize their own clock, and avoid transmitting until the end of the frame;
3. Then the *identifier* is transmitted, on 11 bits (possibly extended to 29 through IDE);
4. The *Remote Transmission Request* (**RTR**) follows.

   If a node wants to transmit data to other nodes, it is dominant to signal for data frames.

   If a node requires data from another node it is recessive and the DLC indicates the number of requested bytes of data. Of course in this last case there is no data field.
5. As we said, there's an **IDE** (*ID Extension*) mode, flagged by the IDE bit (dominant for 11 bits);
6. A reserved dominant bit is included in this flagging section;
7. The *Data Length Code* (**DLC**) follows, including the number of subsequent data bytes. It's a field on 4 bits, with acceptable values 0 to 8;
8. We can finally transmit data. The *data* field is 0 to 8 bytes (depends on DLC, 0 to 64 bits), but always a multiple of 8 bits;

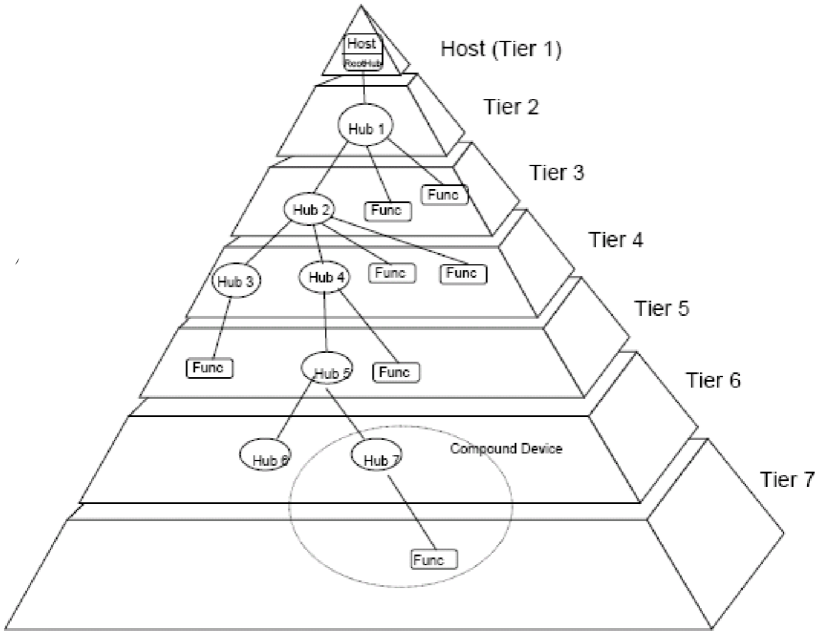
9. A *Cyclic Redundancy Code* (**CRC**) is performed on 15 bits. One recessive bit flags the end of it;
10. There's an acknowledge (**ACK**) bit: transmitter generates recessive, receivers generate dominant. This is also delimited by a recessive bit;
11. Then we have the *End Of Frame* (**EOF**), given by 7 consecutive bits, all recessive. Note that this is an intentional violation of the bit stuffing rule: bit stuffing is otherwise applied to all fields, except CRC delimiter, ACK and EOF.
12. Now the bus returns idle, that is the always recessive. Other nodes can now start a transmission. Good for them.

Notice that CAN defines higher protocol levels. At the electrical level, all nodes on the bus are masters: any node can start a transmission, at any given time. Arbitration on identifiers used to manage priorities and reception masks.

At higher levels of the protocol, some node have additional tasks: for instance to configure the bus and the other nodes, change the communication speed... Just to summarize, several different higher level protocols can be dfined on top of the CAN physical layer.

19.2 USB

We all know **USB** (*Universal Serial Bus*). It was designed to substitute several kind of legacy connections, such as PS/2, RS232, ... It supports Hot Plug&Play, and different speeds (three in USB 2.0).



The USB Network has a tree structure: there's a single master (known as the *host*), and up to 127 slaves (known as *functions* or *devices*), organized on up to 7 levels. Always point-to-point communication between the single master and a slave; *hubs* virtualize connections.

Since the USB Protocol defines a single master, there's no need for arbitration. Each function (FUN) is assigned a unique address between 1 e 127, assigned when it is initially plugged in to the network. Each FUN provides a maximum of 32 (16+16) end-points,

with indexes 0 to 15. For each index, there can be an end-point for reading, and another for writing. Each end-point is unidirectional, with end-point 0 being always present

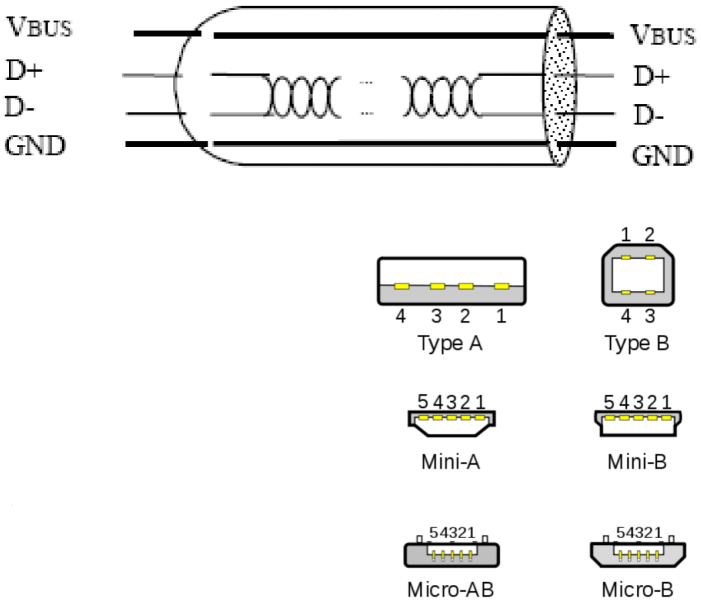
The USB standard is huge. It defines the behavior at several abstraction levels:

- Mechanical
- Electrical
- Protocol
- Transactions
- Multiple transfers
- Driver
- Application and managing software

The *mechanical* standard is based on 4 wires, two for power:

- V_{BUS} at $5\,\mathrm{V}$, $500\,\mathrm{mA}$;
- The ground wire.

And a differential pair for data: D^+ and D^- on STP.



There are several kinds of physical connectors, that can be different between host and function.

USB does full differential signaling, with different voltage levels according to standard and speed configuration. It features error checking and NRZI coding:

- **Logic 0**: transition (at the beginning of a bit);
- **Logic 1**: maintains previous value.

19.2.1 USB pipes

At the *transaction* level, communication on USB based on **pipes** (logical channels). A pipe can be of stream or message type:

- Stream pipe:
  - **Isochronous**: guaranteed bandwidth, possibly loosing data (audio, video, ...);

  - **Interrupt**: quick answer and guaranteed latency (keyboard, mouse, ...);
  - **Bulk**: large sporadic transfers, served using a best-effort bandwidth (file transfers, ...).
- Message pipe:
  - **Control**: short commands to devices and status messages (configuration on endpoint 0, ...).

20 Lesson 06-11-26

20.1 Processor peripherals

Let's connect the peripherals we have seen to a processor core. Different connections to the core are possible:

- Within the pipeline of the processor;
- Embedded on the same integrated circuit, with an On-Chip Bus;
- External circuit mapped in a shared memory;
- External circuit with communication protocol.

20.1.1 In-pipeline

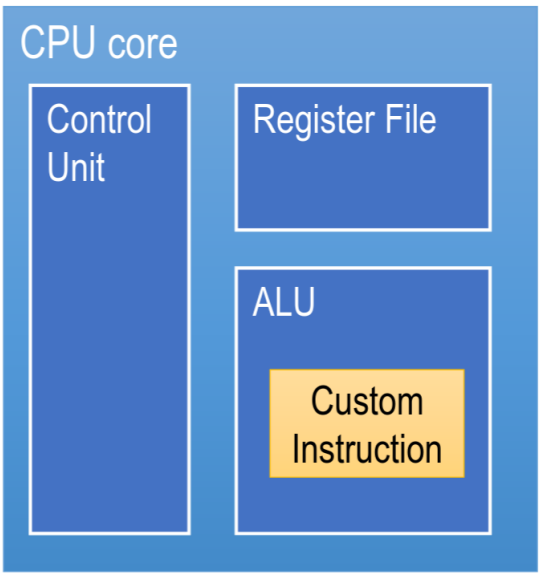
Peripherals *within* the processor pipeline are the absoulte fastest. They are defined as ustom instructions (not really a peripheral), so instructions dedicated to a specific application domain.

This gives a device direct access to the processor register file, and the lowest communication overhead. Note that this can only transfer a limited amount of data.

For specific needs, the processor must also be configurable (and for hard processors this means refabricating). *Soft processors*, that is processors fully described in a hardware description language, are easier for this soft Processors (*Nios II*).

Other kinds of processors can benefit for this, such as:

- Embedded FPGA (*XiRisc*);
- Soft Processor targeting hardware (*Tensilica Xtensa*).

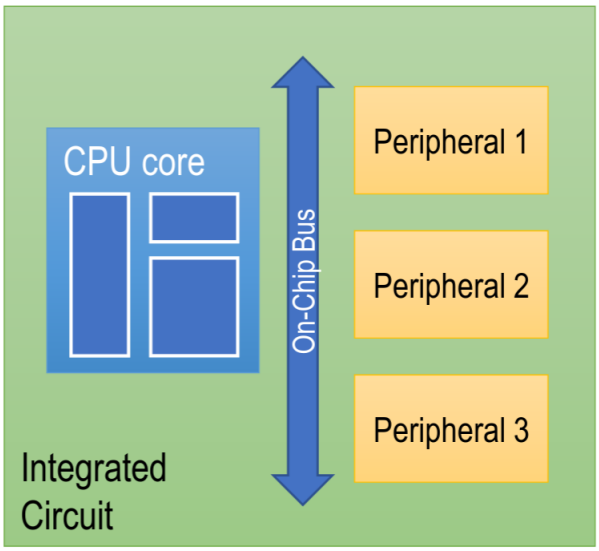


20.1.2 On-chip

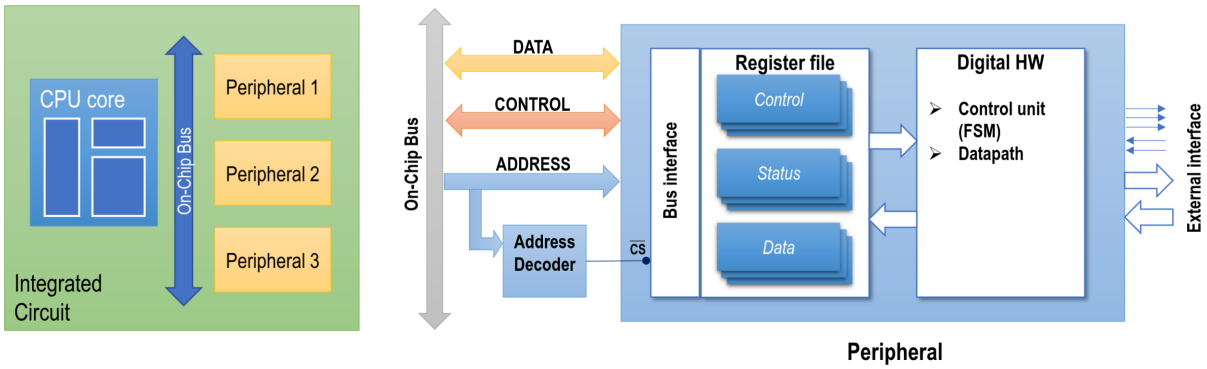
The next best solution (in terms of speed) is peripheral on the same chip, but outside the core. This means the peripheral is embedded in the same integrated circuit (**SoC**, *System on Chip*).

The peripheral is connected to an On-Chip Bus (such as *AMBA, Avalon, AXI, Network on Chip (NoC)*). The bus is accessed by the processor, either I/O Mapped or memory mapped.

There is low communication overhead, as data does not travel outside the integrated circuit (on pins). Of course a set of peripheral registers to exchange data with the processor core must be defined.



We want focus on this.



The typical architecture of single SoC peripheral is as shown above. The processor seems a number of peripherals on his on-chip-bus (which may be distinct from the local bus of the CPU core).

The peripheral takes from the bus the lines:

- **Data**,
- **Address**,
- **Control**.

Communication between processor core and peripheral only through the peripheral register file. The only exception is the address line, which is decoded and functions as an enable for the peripheral (at least its highest part, the low part decides among registers). The other lines are used differently, accessing the register file, based on read/write accesses:

- **Data**: gives data provided by the peripheral on read, takes data from processor on write;
- **Control**: gives status register on read, takes control bits on write.

Internally, the peripheral is designed in whatever way (usually FSM controlling a datapath).

The processor core is usually the master of the on-chip bus. A master, whatever the standard, can initiate a bus transaction to transfer data. The peripheral is a slave, it cannot generate transactions (we saw things similar to this for SPI and I2C).

So, the processor core generates bus transactions to access the peripheral register file. The address decoder enables accessing the peripheral when necessary, driving the chip select, and then selects registers within the file. Multiple bus transactions are required to access multiple registers.

Note that the On-Chip Bus is a shared resource: multiple blocks may drive the same wires concurrently, so there's risk of conflicts and possibly short circuits.

We need to prevent concurrent driving of wires.

This is be done through:

- **Arbitration** between *masters*: only a single master can access the bus at any given time instant;
- **Selection** of *slaves*: using addresses and possibly other control signals, feeding ddress decoders driving chip select.

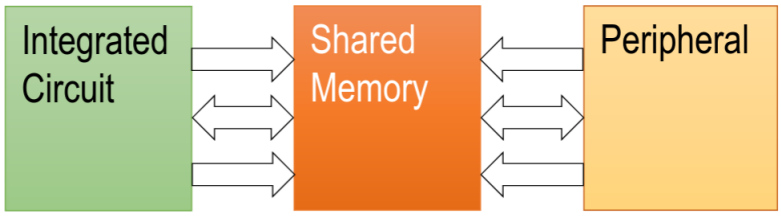
Different driving techniques are used electrically.

- For instance, all drivers can use tri-state output buffers: a single one is enabled, the others are in tri-state.
- Another solution is that a multiplexer drives the shared resource, a single driver output is selected. This way there's no need for tri-state outputs.

20.1.3 External-chip, shared memory

This is the case of a peripheral on an external chip with shared memory. We need an external integrated circuit or FPGA. It's the case of oprocessors (e.g., *JPEG or MPEG codec*), application specific digital circuits designed to work alongisde a procssor.

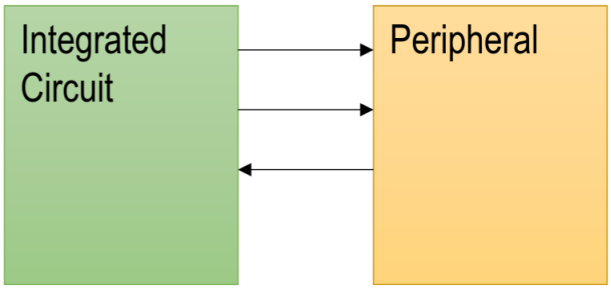
Shared memory can be used for communication, usually large, on the same chip of the peripheral, and multiport. The processor core often needs a memory controller (embedded peripheral) to access the shared memory. There his high communication overhead, usually SRAM is used, and peripheral usually implements a costly and complex function to make the communication overhead acceptable.



20.1.4 External-chip, no shared memory

The same thing can be done without shared a memory. This time, instead of a memory access protocol to touch the peripheral's memory, we need a standard communication protocol such as *SPI, I2C, I2S, UART, USB, PCI, PCIExpress, SATA*. Sometimes even a custom communication protocol can be used.

There is high communication overhead, a custom parallel protocol is sometimes used to increase communication performance.



21 Lesson 11-11-26

21.1 I/O mapping vs memory mapping

bus transaction is characterized by the address generated by the bus master (i.e. the processor core). The Address Bus of the On-Chip Bus has a certain number of bits, where 32 bits is a typical value, for a maximum addressable space of 4GB. The amount of memory

in the system is generally less than the maximum addressable space: a system may have only 128MB out of 4GB of addressable space.

Despite this, the master can generate any address: also addresses where there is no memory. This can be exploited to access the peripheral registers

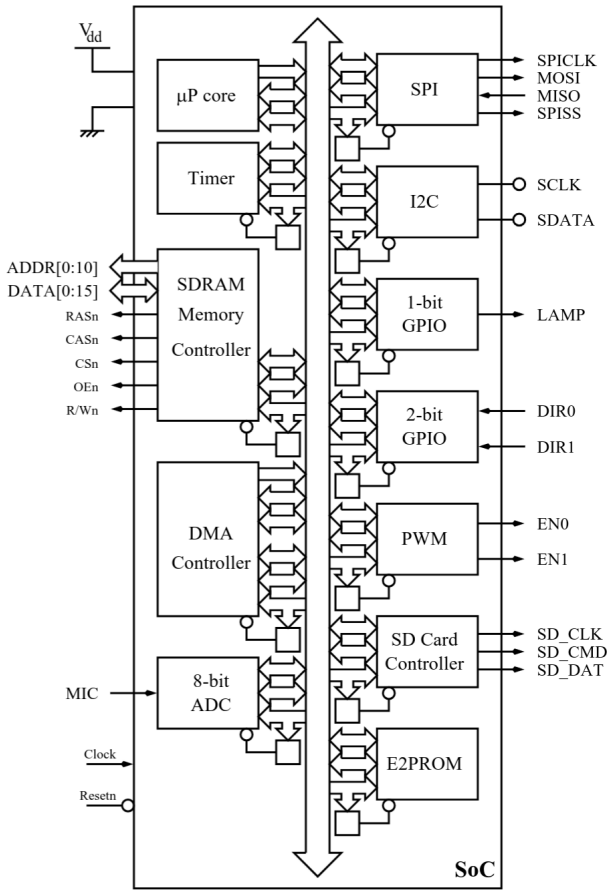
So, we know the two big categories of peripheral mapping in a processor's addressing space:

- **I/O mapping**: registers in a peripheral are assigned a contiguous segment of addresses within a dedicated addressin space, called the *I/O space*. This doesn't exploit our previous property. Peripheral addresses can overlap memory addresses: what's important is that a peripheral address range cannot overlap the range of another peripheral. To distinguish between memory and peripheral (I/O) operation, a special signal of the On-Chip Bus is used. The addressable memory space for I/O operations is often just a subset of the available addressable space (only some address wires are used, rather than all of them as when targeting a memory).
- **Memory mapping**: registers in a peripheral are assigned a contiguous segment of addresses, similar to I/O mapping. The difference is that peripheral addresses should not overlap memory addresses: no need to distinguish!
  - A peripheral address range cannot overlap the range of another peripheral or memory
  - A memory address range cannot overlap the range of another memory or peripheral

  Peripheral addresses should not be cached, if a cache is present. Registers are modified by the peripheral, as well as by the processor A special instruction might be used to bypass the cache (Nios II). The GNU compiler collection, for instance, offers specific flags, and the C language itself has `volatile`.

21.2 SoCs

We can now talk about **SoC**s (which stands for *System on Chip*).



The processor core has no external pins on the SoC: the chip includes DMA Controller, E2PROM,timer and other peripherals. Processor core and DMA are the only masters on the On-Chip Bus: DMA is also a slave (with two separate ports on the bus).

Slave interfaces include address decoders, which drive the peripheral Chip Select signals of the respective bus interfaces.

Clock and Reset go to all blocks in the system.

We said that the processor is a master on the On-Chip Bus, peripheral is a slave (except DMAs). This means the master can generate addresses, and it determines the kind of operation (read, write, ...).

How can a peripheral request attention to the processor? 3 main ways:

- **Polling** (high latency, low performance), might loose some data, as its not real time;
- **Interrupt** (medium latency, medium performance), needs hardware in the processor (which usually does exist);
- **DMA** *Direct Memory Access* (small latency, high performance), we need a special peripheral with master interface on the On-Chip Bus.

22 Lesson 12-11-26

22.1 Peripheral examples

Let's now look at a few peripheral examples that we might find in actual SoCs.

22.1.1 GPIO

GPIO stands for *General Purpose Input Output* controls or checks the logic state of a set of pins, used to control a variety of devices.

We have a set of **control registers** for a variety of options per pin:

- Direction (for bidirectional GPIOs);
- Edge Detection;
- Interrupt generation;
- Internal pull-ups/pull-downs.

From the **status registers** we can obtain info about *edges detected*. From the **data registers** we can fetch the actual *pin value*.

This can be used to control lamps, piezos, relays, ... on the output side, and buttons, switches, ADCs, ... on the input side. Of course electrical considerations apply (resistances when controlling LEDs or BJTs, ...).

22.1.2 Timers

A **timer** is used to count clock ticks. They use the same processor clock, fed to a counter. Number of bits of the counter and clock frequency determine both the *resolution* of the timer, and the *range*. We can trade resolution for range by using a **prescaler** which dides the processor clock frequency by an arbitrary integer value, not necessarily a power of 2. The terminal Count (**TC**) of the pre-scaler drives the enable of the counter. The number of bits of pre-scaler determines the maximum divisor.

At this point, different types of timers are available:

- **Input capture**: we set an external source (edge or level detected), start the timer. When the external source is detected we read the counter value;
- **Output compare**: we set a count and start the timer. When the count is finished, we drive a line high or pulse it.
- **Function generator**: output compare timers can automatically reset themselves: this means they can toggle a line periodically, building square wave oscillators;
- **Interrupt generator**: often timers raise interrupt lines when they finish counting.

Counters are similar to timers, but they count **events** (not *clock ticks*). Usually events are transitions on an external pin. If a timer and a counter are combined, we can compute both the number of events and their frequency over time.

The peripheral registers are the same as a timer, where we configure the input source.

A **watchdog** timer is a down counter, initialized to some known value. Clock signal as input, it measures time (as any timer does).

When it reaches 0, it generates a non-maskable reset to the system: the software shall reload the initial value before the deadline expires.

This is used to restart a system in case it hangs up due to an error or transient failure. It is useful to recover when no user intervention is possible, so it often is present on SoCs for embedded systems.

22.1.3 PWM modulators

A **PWM** (*Pulse Width Modulation*) modulator allows to send signals "close to analog" without having actual analog outputs (DACs). This is done by quickly alternating a pin

between the two voltage levels V_H and V_L, with a high period (duty cycle) proportional to the clock frequency by the analog value we want to output.

In symbols:

$$f = \frac{1}{T}, \quad T = T_H + T_L, \quad \delta = \frac{T_H}{T} \text{ (duty cycle)}$$

The output signal has an average δV_H, which we hope to get close to, with little *ripple* (high frequency noise), when driving an external circuit with some capacitance (which makes it act as a low pass filter).

If δ is slow compared to f, we can use an actual filter to extract the waveform of the average value of the PWM square wave. The average value follows the modulation signal of the duty cycle δ.

How is this implemented?

- A PWM generator consists of a counter, a register and a comparator.
- The counter increments up to a configurable `MAX` value, then resets to 0. A pre-scaler divides the processor clock frequency f_{ck} by an integer divisor.
- The register stores a configurable value $\texttt{reg} \in [0, \texttt{MAX}]$. The output is:

$$\text{PWM}_{out} = \begin{cases} 1 & \text{if } \texttt{cnt} < \texttt{reg} \\ 0 & \text{if } \texttt{cnt} \geq \texttt{reg} \end{cases}$$

- The output duty cycle is:

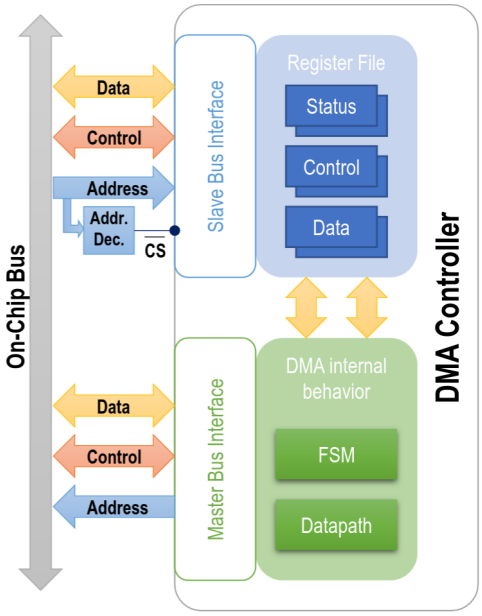
$$\delta = \frac{\texttt{reg}}{\texttt{MAX} + 1}$$

22.2 DMA

Other peripherals we are interested in involve **DMA**. First, we need to distinguish between the three main peripheral access modes in a system:

- **Polling** (*busy waiting*): the CPU continuously checks the peripheral's status until it is ready, wasting CPU cycles;
- **Interrupt**: the peripheral notifies the CPU when it requires attention. The CPU can perform other tasks in the meantime and execute an interrupt handler when notified;
- **DMA** (*Direct Memory Access*): a dedicated controller transfers data directly between memory and peripherals, without requiring the CPU to handle each individual transfer.

DMA is clearly the fastert, but it requires a **DMA controller** to be present on the bus.

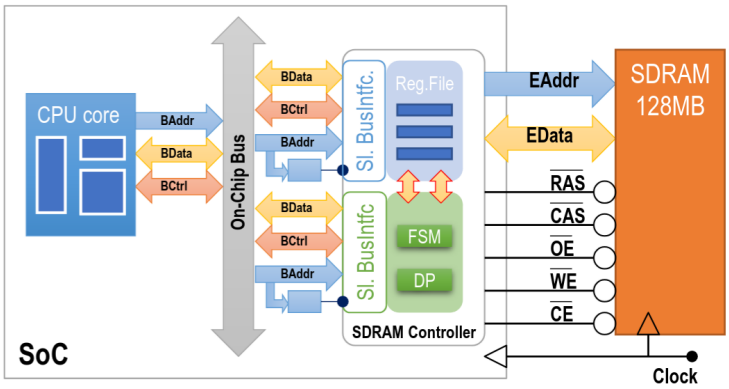


As we can see, a DMA Controller has 2 interfaces:

- A **Slave** Bus Interface to access the peripheral registers, the same as a standard peripheral. This is used by the microprocessor core to:
  - Configure the DMA controller
  - Check its status
- A **Master** Bus Interface to generate bus transactions. It is used by the DMA controller internal behavior. It competes with other masters for the On-Chip Bus ownership (meaning we need arbitration on the On-Chip Bus). A DMA often has higher priority than a microprocessor core.

22.2.1 External memory

Similar to DMA (as in, it has 2 interfaces) is an **external memory controller**.



An External Memory Controller allows access to an external memory, such as *asynchronous memories* (SRAM, E2PROM, Flash) or *synchronous memories* (SDRAM, DDR). It

generates the proper access protocol, different depending on the memory kind. For this reason, we need multiple controllers for different memory kinds.

Note that the controller responds to 2 distinct sets of addresses:

- The address range assigned to the peripheral registers (green);
- The address range assigned to the external memory (blue).

To configure or read the state of the memory controller, the microprocessor core accesses the memory controller peripheral registers starting at the base address. This is identical to a standard peripheral access, and allows access to control registers:

- Type of memory to be driven
- Type of access (Page Mode, EDO, ...)
- Number of banks
- Word size
- Burst size
- Timing parameters (latency)

Then, o access the external memory the microprocessor core generates addresses assigned to the memory. The transaction is initiated on the On-Chip Bus, and the same external memory controller detects the transactions. It generates the proper external signals to access the memory, following memory protocol, and address translations.

It then reports on the On-Chip Bus the results: the microprocessor core only sees a standard On-Chip Bus transaction, possibly with some delay.